

Tunza agent context pack

Self-contained. Paths are labels. The source follows each heading.

Tunza agent context pack

Generated 2026-10-07T03:20:40Z. Branch `clinical-contract-wave1`. HEAD `8228512`. This pack snapshots the working tree, including uncommitted files. It is not origin/main.

You are not on the machine that wrote this

This document is the context. Each path below is a label for source that is pasted in full under that same heading. The bytes are in this file.

Answer from those bytes. If a fact is not in this file, it is absent. Say absent. Do not search your disk for `~/code/Tunza`, and do not treat a path as a file you can open.

A filename is not a capability. The capability map embedded below is the reconciliation of README claims against code. The embedded code wins if a sentence and a file disagree.

What you can decide from this pack

- The public app is a Next.js demo. `decide()` in `lib/assessment.ts` is the only approved decision path. The file says it is not clinically validated. Keep that label.
- The store still calls `decide()` when the question flow stops. `runClinicalPipeline` exists and no surface imports it.
- The contract has ten sections. They stay separate. `abstain` has no user-visible headline.
- A candidate below the floor, or an unapproved higher candidate, is rewritten to the floor. `fail\_open` is false.
- There is no owned model, no training set, and no retrieval corpus in the embedded tree.
- Promotion stays blocked. Green tests are not a release.
- Training, retrieval ingestion, guideline thresholds, JEV, the private `medical-triage` route, and cross-encounter persistence stay closed. The opening tests are in the embedded wave-2 ledger. This pack does not supply that evidence.

Working tree at pack time

...

M evals/manifest.json
M evals/runner.ts
M lib/copy.ts
M vault/INDEX.md
M vault/decisions.md
?? agent-context/

```
?? evals/adversarial/
?? evals/gates.ts
?? evals/reports/wave1-pipeline.json
?? lib/clinical/pipeline.ts
?? lib/clinical/reconcile.ts
?? lib/clinical/state.ts
?? specs/
?? tests/clinical-pipeline.test.ts
?? tests/clinical-reconcile.test.ts
?? tests/clinical-state.test.ts
?? tests/eval-gates.test.ts
?? vault/sessions/2026-10-06-clinical-contract-wave1.md
?? vault/sessions/2026-10-06-receipt-workflow.md
?? vault/tickets/
?? vault/wave1-handoff.md
?? vault/wave2-ledger.md
?? vault/workflows/
````
```

## ## Index

- `lib/copy.ts` - Every user-visible string, English and Kiswahili. A missing Kiswahili string is a compile error. SHA-256  
`d94cbfb03fe3c34d0a434f9d6ea1a4e1478a65c7e67f6d90209848eb64d0ae1c`.
- `DESIGN.md` - The product contract: one next action, three surfaces, the referral spine. SHA-256 `20241fbae5a7d68a9a11c8fba27fe104a3026ccb67c26a6291183e5d396a3524`.
- `lib/types.ts` - Roles, the four decision kinds, the referral lifecycle, the encounter shape. SHA-256  
`a2d0068bde83840c300f06a03d8050fd70eb6b644b23aa2b14f8606b2a29f738`.
- `lib/assessment.ts` - The demo floor. `decide()` is the only approved path. Not clinically validated. SHA-256  
`e9a2cdeaed47f534d16163063b1c159b4040fc7eba07deed655c1d247921df87`.
- `lib/clinical/contract.ts` - Ten-section contract, disposition table, validators. No new user-visible headline. SHA-256  
`da15f6167c3e6aad9e2ca7ea6e7a07ba54ae8e60fd7232b825942864090f9976`.
- `lib/clinical/state.ts` - Append-only encounter projection. No single temperature. No clinical cutoff. SHA-256  
`2c0a721d5f4450aa50f7f13f529ef33bd0ceaa19507da1a61df7bfa5159cd218`.
- `lib/clinical/reconcile.ts` - Floor wins. A lower or unapproved candidate is rewritten up. `fail_open` is false. SHA-256  
`67626c656def0b9a06db426f5b19a1e490fd0c4b9dcdb5242156dae30f9b23bc`.
- `lib/clinical/pipeline.ts` - `decide()` then reconciler then contract. The UI does not call this yet. SHA-256  
`25a2b5e1ad7cfe82991f4841d49216e0bfb5de652b3e7643b1675cdf104f26a7`.
- `lib/store.tsx` - The reducer. The answer path still calls `decide()` directly and persists that Decision. SHA-256  
`cc7c7341b7c6d0f5b38947b0c462deee351bf28f6d3c8d1e2608cbab64670cf7`.
- `lib/referral.ts` - The only place referral state becomes words. SHA-256  
`43bf831d61cd67a06f0271e61dc5f8046646ec440c0fccd961c00adb5a72672a`.
- `components/DecisionResult.tsx` - The verdict component. One headline, one action.

SHA-256 `50e107b3edae24262ed1ad57cc829f1e6171cf8a7977ebd2072eea105df0d91d`.

- `components/surfaces/HouseholdSurface.tsx` – Household renders the stored Decision, not the pipeline. SHA-256 `9ca3625c201d79fe39de194bf8b46638948118e63c2e3c299771cf274294037f`.
- `components/surfaces/ChpSurface.tsx` – CHP uses the same DecisionResult. SHA-256 `652906d914bae7973485bf432cf2800acca21f236ee5b07fdc8358b35ce377f2`.
- `components/surfaces/FacilitySurface.tsx` – Facility speaks referral state, not a second clinical paragraph. SHA-256 `7bb2637971117ac311d51c8799ff3005f1630d1e462e3e97c5cea6bcec74ccc4`.
- `components/CarePath.tsx` – Referral presentation goes through describeReferral. SHA-256 `75ce8f94c75430458223d844a74468fe27fc9e12505ce10b650bef60dc0d4f7e`.
- `evals/gates.ts` – Release machinery. Promotion is blocked. This file cannot authorize a release. SHA-256 `41c4487ad7df83b5e2de5129e3bfce76e9bbb48cc47a7218fe0707590148d237`.
- `vault/architecture.md` – Module map as written in the repo vault. SHA-256 `ec884e3b547fdd5120744ebbcbeed887387ae25fccdb7180a5a5409fe9832876`.
- `vault/decisions.md` – Dated decisions, including why wave 1 is a contract and not a model. SHA-256 `c166b008cfe2238a1b213a7f9f5ee038f7e0663a5d991dbbd9d8ca8d09c486e4`.
- `vault/capability-map.md` – What exists, what is missing, and which README claims are not code. SHA-256 `5d12d4a47322bd9ded6098ccc774e4156aab050f447b76fe05e89ae1fada9fcc`.
- `vault/wave1-contract.md` – Wave-1 scope, deferrals, and the files that must not be edited. SHA-256 `4ae6ae98b3f4b11748768629a7d64924b84015a9ff31caf5d2a7ca7c63eadec8`.
- `vault/wave1-handoff.md` – What was written, what was verified, and what is still blocked. SHA-256 `3e4802111b72eeb51bd72e3640e47bcbe430c2eaafff8e972e0b322f5ab47197d`.
- `vault/wave2-ledger.md` – Closed doors and the evidence required before any of them open. SHA-256 `dde8bab8d704e71c3f4339f5915ed4cd1b5d265f04bb42469baa2d87c9046686`.

## Embedded sources

## lib/copy.ts

Every user-visible string, English and Kiswahili. A missing Kiswahili string is a compile error.

SHA-256 d94cbfb03fe3c34d0a434f9d6ea1a4e1478a65c7e67f6d90209848eb64d0ae1c · 729 lines

```
export type Locale = "en" | "sw";

export const LOCALES: Locale[] = ["en", "sw"];
export const DEFAULT_LOCALE: Locale = "en";

/**
 * Every user-visible string in the product, in both languages.
 * A screen is not finished until it fits in both – `sw` is typed against the
 * English table, so a missing translation is a compile error, and `t` requires
 * an explicit locale so nothing silently ships in one language.
 */
const en = {
 // Home – the landing surface, carried over from the earlier Tunza app
 homeTagline: "Healthcare, anywhere, anytime.",
 homeGetStarted: "Get started",
 homeCardAssessTitle: "Check symptoms",
 homeCardAssessDesc: "Text, voice or photo",
```

```

homeCardNearbyTitle: "Nearby facilities",
homeCardNearbyDesc: "See your area",
homeCardContinueTitle: "Continue",
homeMission:
 "Built for community health workers where the nearest doctor may be hours away.",
homeHealthWorker: "Are you a health worker?",
gateHeading: "Sign in to continue",
gateBody:
 "This area is for registered health workers. Choose your role and enter your access code.",
gateChooseChp: "I'm a community health promoter",
gateChooseFacility: "I work at a facility",
gateCodeLabel: "Access code",
gateSubmit: "Sign in",
gateChecking: "Checking...",
gateWrongCode: "That code did not work. Check with your coordinator.",
gateDemoHint: "Demo codes: CHP-DEMO and FACILITY-DEMO.",
nearbyTitle: "Nearby facilities",
nearbyLocating: "Finding facilities near you...",
nearbyDemoFallback: "Showing demo facilities – live lookup is not available right now.",
facilityUrgentCapable: "Can take urgent cases",
facilityGeneralCare: "General care",
facilityHospital: "Hospital",
facilityClinic: "Clinic",
facilityPharmacy: "Pharmacy",
facilityDistanceTpl: "About {km} km away",
transcribing: "Transcribing...",

// Who needs help?
whoQuestion: "Who needs help?",
whoSelf: "Me",
whoAdult: "An adult at home",
whoChild: "A child",
whoUnknown: "I'm not sure",

// What is happening?
whatQuestion: "What is happening?",
whatHint: "Say it the way you would tell a neighbour.",
whatPlaceholder: "Fever, coughing, not drinking...",
whatContinue: "Continue",

// Entry modes
modeSpeak: "Speak",
modeType: "Type",
modePhoto: "Photo",
entryAria: "How to answer",
typeWhatYouSee: "Type what you see",
tapToSpeak: "Tap to speak",
listening: "Listening...",
speakNotAvailable: "Speak isn't available",
speakUnavailable: "Speaking isn't available on this device. Type instead.",
photoTake: "Take or choose a photo",
photoAttachedDemo: "Photo attached (demo)",
photoRemove: "Remove photo",
photoHint:
 "Use this when a rash, wound, or breathing effort is easier to show than to describe.",
addNote: "Add a short note if you can",

// One question at a time
dontKnow: "I don't know",
back: "Previous",
awakeQuestion: "Are they awake and responding?",
awakeAlert: "Yes, alert",
awakeSleepy: "Sleepy or confused",
awakeNotWaking: "Not waking up",

```

```
breathingQuestion: "How is their breathing?",
breathingFine: "Breathing looks fine",
breathingDifficult: "Fast or hard to breathe",
breathingSevere: "Struggling to breathe",
drinkingQuestion: "Can they drink?",
drinkingQuestionChild: "Can they drink or breastfeed?",
drinkingYes: "Yes",
drinkingLittle: "Only a little",
drinkingNo: "No",
durationQuestion: "How long has this been going on?",
durationToday: "Started today",
durationTwoDays: "1-2 days",
durationLonger: "Longer than that",
mainProblemQuestion: "What is the main problem right now?",
mainBreathing: "Breathing",
mainFever: "Fever",
mainInjury: "Injury or bleeding",
mainStomach: "Stomach or not eating",
mainOther: "Something else",

// The decision
nextAction: "Next action",
goNow: "Go now",
getCareToday: "Get care today",
monitorAtHome: "Monitor at home",
needOneMore: "I need one more answer",
goNowStatus: "This looks urgent. Do not wait.",
getCareStatus: "They should be seen today, not next week.",
monitorStatus: "Stay home for now, and watch these signs.",
needOneMoreStatus: "One question would make this safer.",
prepareFacility: "Get a facility ready",
createReferral: "Create referral",
answerTheQuestion: "Answer the question",
whyThis: "Why this?",
moreDetail: "More detail",

// Danger signs
dangerNotWaking: "Not waking",
dangerSeizure: "Seizure",
dangerNotBreathing: "Not breathing",
dangerBlueLips: "Blue lips",
dangerHeavyBleeding: "Heavy bleeding",
dangerStrugglingBreathe: "Struggling to breathe",
dangerChildNoDrink: "Child cannot drink",

// Watch signs
watchNotWaking: "Not waking or very sleepy",
watchBreathingHard: "Breathing gets hard or fast",
watchCannotDrink: "Cannot drink",
watchSeizureBleeding: "Seizure or heavy bleeding",

// Reasons behind the decision
reasonSleepyHardBreathing: "Sleepy and breathing is hard",
reasonTooLittleKnown: "Too little is known to choose safely",
reasonOneMoreSafer: "One more question would make this safer",
reasonBreathingHarder: "Breathing is harder than usual",
reasonDrinkingLittle: "Drinking only a little",
reasonNotDrinking: "Not drinking",
reasonSleepy: "Sleepy or harder to wake",
reasonChildFever: "Child with fever",
reasonInjurySeen: "Injury that should be seen",
reasonBreathingMain: "Breathing is the main problem",
reasonTooLongHome: "This has gone on too long to only watch at home",
reasonNoDanger: "Awake, drinking, and breathing do not show a danger sign",
```

```
// Missing information
missingWho: "Who this is for is not clear",
missingWhoDetail: "Child, adult, or self changes what we ask next.",
missingWhat: "What is happening was not described",
missingAwake: "Awake and responding is unknown",
missingBreathing: "Breathing is unknown",
missingDrinking: "Whether they can drink is unknown",
missingAskMoreDetail: "The facility asked for this before accepting.",
askMoreCanWalk: "Can they walk into the facility?",

// Referral – stage labels (human words, never the internal names)
referralEyebrow: "Referral",
stageCreated: "Being prepared",
stageSent: "Sent",
stageReceived: "Received",
stageAccepted: "Accepted",
stagePatientMoving: "Patient traveling",
stageArrived: "Arrived",
stageSeen: "Being seen",
stageCompleted: "Visit complete",
stageOutcomeReturned: "Outcome returned",

// Referral – the same event, per role
refCreatedHouseholdHeadline: "A facility is being prepared",
refCreatedHouseholdStatus: "Nothing has been sent yet.",
refCreatedChpHeadline: "Referral prepared – not sent yet",
refCreatedChpStatus: "Ready for {f}.",
refCreatedFacilityHeadline: "A community referral is being prepared",
refCreatedFacilityStatus: "It has not reached this facility yet.",
refSentHouseholdHeadline: "Your referral is on the way",
refSentHouseholdStatus: "{f} has not answered yet.",
refSentChpHeadline: "Referral sent – waiting for the facility",
refSentChpStatus: "{f} has not accepted yet.",
refSentFacilityHeadline: "New referral – not yet reviewed",
refSentFacilityStatus: "Decide whether this facility can take them.",
refReceivedHouseholdHeadline: "The facility has seen your referral",
refReceivedHouseholdStatus: "They have not accepted yet. Stay ready.",
refReceivedChpHeadline: "Facility received referral – not yet accepted",
refReceivedChpStatus: "{f} is deciding now.",
refReceivedFacilityHeadline: "Referral received – decide now",
refReceivedFacilityStatus: "Accept, redirect, or ask for one missing fact.",
refAcceptedHouseholdHeadline: "Facility accepted – you can leave now",
refAcceptedHouseholdStatus: "Go to {f}.",
refAcceptedChpHeadline: "Referral accepted – patient travel not yet confirmed",
refAcceptedChpStatus: "{f} is waiting for them to leave.",
refAcceptedFacilityHeadline: "Incoming referral",
refAcceptedFacilityUrgentHeadline: "Incoming urgent referral",
refAcceptedFacilityStatus: "Prepare to receive them.",
refMovingHouseholdHeadline: "You're on the way. They know you're coming.",
refMovingHouseholdStatus: "Tell them you are the Tunza referral at {f}.",
refMovingChpHeadline: "Patient traveling – arrival not yet confirmed",
refMovingChpStatus: "{f} is expecting them.",
refMovingFacilityHeadline: "Patient en route – prepare to receive",
refMovingFacilityStatus: "Keep the receiving place ready.",
refArrivedHouseholdHeadline: "You've arrived. Tell them you're the Tunza referral.",
refArrivedHouseholdStatus: "Stay until they have seen you.",
refArrivedChpHeadline: "Patient arrived – visit not yet closed",
refArrivedChpStatus: "They are at the facility.",
refArrivedFacilityHeadline: "Patient here – start care",
refArrivedFacilityStatus: "The person has arrived.",
refSeenHouseholdHeadline: "They are being seen now",
refSeenHouseholdStatus: "Stay until the visit is finished.",
refSeenChpHeadline: "Patient being seen – visit not yet closed",
```

```

refSeenChpStatus: "The outcome comes after the visit is closed.",
refSeenFacilityHeadline: "In care – close the visit when done",
refSeenFacilityStatus: "Record what happened when care is finished.",
refCompletedHouseholdHeadline: "Care is finished. Waiting to hear what happened.",
refCompletedHouseholdStatus: "The outcome has not come back yet.",
refCompletedChpHeadline: "Visit completed – waiting for outcome",
refCompletedChpStatus: "Follow-up depends on what the facility returns.",
refCompletedFacilityHeadline: "Visit complete – return the outcome",
refCompletedFacilityStatus: "The community needs to know what happened.",
refOutcomeHouseholdHeadline: "Here's what happened, and what to do next",
refOutcomeHouseholdStatus: "The facility returned an outcome.",
refOutcomeChpHeadline: "Outcome returned – follow-up may be needed",
refOutcomeChpStatus: "Review whether this household needs a visit.",
refOutcomeFacilityHeadline: "Outcome sent to the community",
refOutcomeFacilityStatus: "Returned.",
refQueuedHouseholdHeadline: "Saved on this phone",
refQueuedHouseholdStatus: "You're offline. The referral will send when you're back.",
refQueuedChpHeadline: "Referral queued – offline",
refQueuedChpStatus: "It is on this device. Send when the connection returns.",
refNoResponseHeadline: "No facility response",
refNoResponseHouseholdStatus: "Stay where you can travel when they answer.",
refNoResponseOtherStatus: "Referral sent – the facility has not answered.",
refNoResponseFacilityStatus: "{f} has not responded.",

// Referral – actions
actionSendNow: "Send now",
actionSendFacility: "Send to the facility",
actionSendReferral: "Send referral",
actionAccept: "Accept",
actionWeveLeft: "We've left",
actionTheyLeft: "They have left",
actionWeveArrived: "We've arrived",
actionTheyreHere: "They're here",
actionStartCare: "Start care",
actionCompleteVisit: "Complete visit",
actionReturnOutcome: "Return what happened",

// Referral – numbers
expectedArrivalTpl: "expected arrival {m} min",
travelTpl: "Travel {m} min",

// Failure states (designed, never generic)
failOfflineEyebrow: "Offline",
failOfflineTitle: "You're offline",
failOfflineBody:
 "The assessment still works on this phone. Sending a referral waits until you are back.",
failNoResponseEyebrow: "No response",
failNoResponseTitle: "No facility response",
failNoResponseBody:
 "The referral left this phone. The facility has not answered. Stay where you can travel when they do.",
failRedirectedEyebrow: "Redirected",
failRedirectedTitle: "Redirected to another facility",
failRedirectedBody:
 "This place cannot take them. Another facility is now on the referral.",
failStaleEyebrow: "Old information",
failStaleTitle: "This information may be stale",
failStaleBody:
 "A recorded value is too old to drive today's decision the same way as a fresh one.",
failIncompleteEyebrow: "Incomplete",
failIncompleteTitle: "This assessment is incomplete",
failIncompleteBody:
 "There is not enough information to make this decision safely. One more answer is required.",
failWeakEyebrow: "Weak connection",

```

```

failWeakTitle: "Weak connection",
failWeakBody:
 "This may take longer. The care path stays on this phone until the send is confirmed.",
dangerEyebrow: "Danger sign",
dangerTitle: "Do not wait at home with this sign.",
dangerBody: "Go to care now.",
watchEyebrow: "Watch",
watchTitle: "Go now if this starts",

// Facility card
facilityEyebrow: "Facility",
facilityTravelTpl: "{m} min by typical road",
facilityCanTake: "This facility can take them",
facilityWeCanHandle: "We can handle this referral",
facilityMayNot: "This facility may not be able to take them",

// Handoff
handoffEyebrow: "Handoff",
whyComingUrgent:
 "{who} needs urgent care. Danger signs were flagged in the community assessment.",
whyComingToday: "{who} should be seen today. This is not a wait-at-home case.",
whyComingDefault: "{who} is on a Tunza referral.",
whoSubjectChild: "A child from a demo household",
whoSubjectSelf: "An adult (self) from a demo household",
whoSubjectOther: "A person from a demo household",
handoffWho: "Who",
handoffDescribed: "What they described",
handoffAwake: "Awake",
handoffBreathing: "Breathing",
handoffDrinking: "Drinking",
handoffDuration: "How long",
handoffTemp: "Temperature (demo)",
handoffPhotoOnly: "Photo only (demo)",
handoffNotDescribed: "Not described",
handoffStillMissing: "Still missing",
whoValueSelf: "Adult (self) · demo household",
whoValueAdult: "Adult · demo household",
whoValueChild: "Child · demo household",
whoValueUnknown: "Not specified · demo household",
unknownValue: "Unknown",
freshnessAssessedTpl: "assessed {m} min ago",
freshnessOld: "recorded 8 months ago",

// Outcome
outcomeQuestion: "What happened?",
outcomeTreated: "Seen and treated",
outcomeHigher: "Needed a higher facility",
outcomeNoShow: "Did not arrive",
outcomeUnknown: "Outcome not known",

// CHP and facility surfaces
startEncounter: "Start encounter",
noEncounter: "No active encounter",
noEncounterDetail: "Start one to capture what is happening, then decide.",
noIncoming: "No incoming referral",
noIncomingDetail: "When a household or CHP sends one, it appears here.",
otherActions: "Other actions",
redirect: "Redirect",
askMore: "Ask more",
needsAttention: "Needs attention",
needsAction: "Needs action",
missingInformation: "Missing information",
whoNeedsFollowUp: "Who needs follow-up",
nothingWaiting: "Nothing waiting",

```

```

encounterTitle: "Encounter",
incomingTitle: "Incoming",
fuWatchHome: "Demo household – watch for danger signs",
fuWatchHomeDetail:
 "They were advised to stay home. Check they still can drink and wake.",
fuConfirmTravel: "Travel not yet confirmed",
fuConfirmTravelDetail: "Facility accepted. Confirm whether the household has left.",
fuPostOutcome: "Demo household – follow up after care",
fuPostOutcomeDetail: "Confirm they are improving at home.",

// Chrome
viewingAs: "Viewing as",
roleHousehold: "Household",
roleChp: "CHP",
roleFacility: "Facility",
demoLabel: "Demo",
namedConditions: "Named conditions",
startOver: "Start over",
languageButton: "Kiswahili",
languageAria: "Badilisha lugha kuwa Kiswahili",
disclaimer: "Not a substitute for emergency services or professional medical care.",

contractObservedFacts: "Observed facts",
contractRetrievedEvidence: "Retrieved evidence",
contractDerivedFeatures: "Derived features",
contractHypotheses: "Hypotheses",
contractUncertainties: "Uncertainties",
contractContradictions: "Contradictions",
contractRequiredInformation: "Required information",
contractRedFlags: "Recorded flags",
contractNextAction: "Next action",
contractProvenance: "Provenance",
};

export type CopyKey = keyof typeof en;

const sw: Record<CopyKey, string> = {
 homeTagline: "Huduma ya afya, popote, wakati wowote.",
 homeGetStarted: "Anza",
 homeCardAssessTitle: "Angalia dalili",
 homeCardAssessDesc: "Maandishi, sauti au picha",
 homeCardNearbyTitle: "Vituo vya karibu",
 homeCardNearbyDesc: "Ona eneo lako",
 homeCardContinueTitle: "Endelea",
 homeMission:
 "Imetengenezwa kwa ajili ya wahudumu wa afya ya jamii ambapo daktari wa karibu anaweza kuwa masaa mengi mbali.",
 homeHealthWorker: "Wewe ni mhudumu wa afya?",
 gateHeading: "Ingia ili kuendelea",
 gateBody:
 "Sehemu hii ni ya wahudumu wa afya waliosajiliwa. Chagua nafasi yako na uweke msimbo wako wa kuingia.",
 gateChooseChp: "Mimi ni mhamasishaji wa afya ya jamii",
 gateChooseFacility: "Nafanya kazi katika kituo cha afya",
 gateCodeLabel: "Msimbo wa kuingia",
 gateSubmit: "Ingia",
 gateChecking: "Inakagua...",
 gateWrongCode: "Msimbo huo haukufanya kazi. Wasiliana na mratibu wako.",
 gateDemoHint: "Misimbo ya majaribio: CHP-DEMO na FACILITY-DEMO.",
 nearbyTitle: "Vituo vya karibu",
 nearbyLocating: "Inatafuta vituo karibu nawe...",
 nearbyDemoFallback:
 "Vituo vya majaribio vinaonyeshwa – utafutaji wa moja kwa moja haupatikani sasa.",
 facilityUrgentCapable: "Inaweza kupokea dharura",

```

facilityGeneralCare: "Huduma za kawaida",  
facilityHospital: "Hospitali",  
facilityClinic: "Kliniki",  
facilityPharmacy: "Duka la dawa",  
facilityDistanceTpl: "Takriban kilomita {km} kutoka hapa",  
transcribing: "Inanukuu...",

whoQuestion: "Nani anahitaji msaada?",  
whoSelf: "Mimi",  
whoAdult: "Mtu mzima nyumbani",  
whoChild: "Mtoto",  
whoUnknown: "Sina uhakika",

whatQuestion: "Nini kinaendelea?",  
whatHint: "Sema kama vile ungemwambia jirani.",  
whatPlaceholder: "Homa, kukohoa, hanywi...",  
whatContinue: "Endelea",

modeSpeak: "Sema",  
modeType: "Andika",  
modePhoto: "Picha",  
entryAria: "Jinsi ya kujibu",  
typeWhatYouSee: "Andika unachoona",  
tapToSpeak: "Gusa kuongea",  
listening: "Inasikiliza...",  
speakNotAvailable: "Kusema hakupatikani",  
speakUnavailable: "Kusema hakupatikani kwenye kifaa hiki. Andika badala yake.",  
photoTake: "Piga au chagua picha",  
photoAttachedDemo: "Picha imeambatishwa (jaribio)",  
photoRemove: "Ondoa picha",  
photoHint:  
    "Tumia hii wakati upele, jeraha, au jinsi anavyopumua ni rahisi kuonyesha kuliko kueleza.",  
addNote: "Ongeza maelezo mafupi ukiweza",

dontKnow: "Sijui",  
back: "Rudi nyuma",  
awakeQuestion: "Je, yuko macho na anaitika?",  
awakeAlert: "Ndiyo, yuko macho",  
awakeSleepy: "Ana usingizi au amechanganyikiwa",  
awakeNotWaking: "Haamki",  
breathingQuestion: "Anapumujaje?",  
breathingFine: "Anapumua vizuri",  
breathingDifficult: "Anapumua kwa kasi au kwa shida",  
breathingSevere: "Anahangaika kupumua",  
drinkingQuestion: "Je, anaweza kunywa?",  
drinkingQuestionChild: "Je, anaweza kunywa au kunyonya?",  
drinkingYes: "Ndiyo",  
drinkingLittle: "Kidogo tu",  
drinkingNo: "Hapana",  
durationQuestion: "Hii imeendelea kwa muda gani?",  
durationToday: "Imeanza leo",  
durationTwoDays: "Siku 1–2",  
durationLonger: "Zaidi ya hapo",  
mainProblemQuestion: "Tatizo kuu ni nini sasa hivi?",  
mainBreathing: "Kupumua",  
mainFever: "Homa",  
mainInjury: "Jeraha au kutokwa na damu",  
mainStomach: "Tumbo au kutokula",  
mainOther: "Kitu kingine",

nextAction: "Hatua inayofuata",  
goNow: "Nenda sasa",  
getCareToday: "Pata matibabu leo",  
monitorAtHome: "Angalia nyumbani",

needOneMore: "Nahitaji jibu moja zaidi",  
goNowStatus: "Hii inaonekana ya dharura. Usisubiri.",  
getCareStatus: "Anapaswa kuonwa leo, si wiki ijayo.",  
monitorStatus: "Kaa nyumbani kwa sasa, na uangalie dalili hizi.",  
needOneMoreStatus: "Swali moja lingefanya hili kuwa salama zaidi.",  
prepareFacility: "Andaa kituo cha afya",  
createReferral: "Tengeneza rufaa",  
answerTheQuestion: "Jibu swali",  
whyThis: "Kwa nini hivi?",  
moreDetail: "Maelezo zaidi",

dangerNotWaking: "Haamki",  
dangerSeizure: "Kifafa au degedege",  
dangerNotBreathing: "Hapumui",  
dangerBlueLips: "Midomo ya bluu",  
dangerHeavyBleeding: "Kutokwa na damu nyingi",  
dangerStrugglingBreathe: "Anahangaika kupumua",  
dangerChildNoDrink: "Mtoto hawezi kunywa",

watchNotWaking: "Haamki au ana usingizi mzito",  
watchBreathingHard: "Kupumua kunakuwa kugumu au kwa kasi",  
watchCannotDrink: "Hawezi kunywa",  
watchSeizureBleeding: "Kifafa au kutokwa na damu nyingi",

reasonSleepyHardBreathing: "Ana usingizi na anapumua kwa shida",  
reasonTooLittleKnown: "Kinachojulikana ni kidogo mno kuchagua kwa usalama",  
reasonOneMoreSafer: "Swali moja zaidi lingefanya hili kuwa salama zaidi",  
reasonBreathingHarder: "Anapumua kwa shida kuliko kawaida",  
reasonDrinkingLittle: "Anakunywa kidogo tu",  
reasonNotDrinking: "Hanywi",  
reasonSleepy: "Ana usingizi au ni vigumu kumwamsha",  
reasonChildFever: "Mtoto mwenye homa",  
reasonInjurySeen: "Jeraha linalopaswa kuonwa",  
reasonBreathingMain: "Kupumua ndilo tatizo kuu",  
reasonTooLongHome: "Imeendelea muda mrefu mno kuangalia nyumbani tu",  
reasonNoDanger: "Yuko macho, anakunywa, na kupumua hakuonyeshi dalili ya hatari",

missingWho: "Ni nani anayehusika haijabainika",  
missingWhoDetail: "Mtoto, mtu mzima, au wewe mwenyewe hubadilisha tunachouliza.",  
missingWhat: "Kinachoendelea hakikuelezwa",  
missingAwake: "Kuwa macho na kuitika hakujulikani",  
missingBreathing: "Hali ya kupumua haijulikani",  
missingDrinking: "Kama anaweza kunywa hakujulikani",  
missingAskMoreDetail: "Kituo kiliuliza hili kabla ya kukubali.",  
askMoreCanWalk: "Je, anaweza kutembea hadi ndani ya kituo?",

referralEyebrow: "Rufaa",  
stageCreated: "Inaandaliwa",  
stageSent: "Imetumwa",  
stageReceived: "Imepokelewa",  
stageAccepted: "Imekubaliwa",  
stagePatientMoving: "Mgonjwa safarini",  
stageArrived: "Amefika",  
stageSeen: "Anaonwa",  
stageCompleted: "Ziara imekamilika",  
stageOutcomeReturned: "Matokeo yamerejeshwa",

refCreatedHouseholdHeadline: "Kituo cha afya kinaandaliwa",  
refCreatedHouseholdStatus: "Hakuna kilichotumwa bado.",  
refCreatedChpHeadline: "Rufaa imeandaliwa – haijatumwa bado",  
refCreatedChpStatus: "Iko tayari kwa {f}.",  
refCreatedFacilityHeadline: "Rufaa ya jamii inaandaliwa",  
refCreatedFacilityStatus: "Bado haijafika kituo hiki.",  
refSentHouseholdHeadline: "Rufaa yako iko njiani",

refSentHouseholdStatus: "{f} bado haijajibu.",  
refSentChpHeadline: "Rufaa imetumwa – inasubiri kituo",  
refSentChpStatus: "{f} bado haijakubali.",  
refSentFacilityHeadline: "Rufaa mpya – bado haijakaguliwa",  
refSentFacilityStatus: "Amua kama kituo hiki kinaweza kumpokea.",  
refReceivedHouseholdHeadline: "Kituo kimeona rufaa yako",  
refReceivedHouseholdStatus: "Bado hawajakubali. Kaa tayari.",  
refReceivedChpHeadline: "Kituo kimepokea rufaa – bado haijakubaliwa",  
refReceivedChpStatus: "{f} inaamua sasa.",  
refReceivedFacilityHeadline: "Rufaa imepokelewa – amua sasa",  
refReceivedFacilityStatus: "Kubali, elekeza kwingine, au uliza jambo moja linalokosekana.",  
refAcceptedHouseholdHeadline: "Kituo kimekubali – unaweza kuondoka sasa",  
refAcceptedHouseholdStatus: "Nenda {f}.",  
refAcceptedChpHeadline: "Rufaa imekubaliwa – safari ya mgonjwa haijathibitishwa",  
refAcceptedChpStatus: "{f} inasubiri waondoke.",  
refAcceptedFacilityHeadline: "Rufaa inakuja",  
refAcceptedFacilityUrgentHeadline: "Rufaa ya dharura inakuja",  
refAcceptedFacilityStatus: "Jiandae kumpokea.",  
refMovingHouseholdHeadline: "Uko njiani. Wanajua unakuja.",  
refMovingHouseholdStatus: "Waambie wewe ni rufaa ya Tunza katika {f}.",  
refMovingChpHeadline: "Mgonjwa yuko safarini – kufika hakujathibitishwa",  
refMovingChpStatus: "{f} inamtarajia.",  
refMovingFacilityHeadline: "Mgonjwa yuko njiani – jiandae kupokea",  
refMovingFacilityStatus: "Weka mahali pa kupokea tayari.",  
refArrivedHouseholdHeadline: "Umefika. Waambie wewe ni rufaa ya Tunza.",  
refArrivedHouseholdStatus: "Kaa hadi wakuone.",  
refArrivedChpHeadline: "Mgonjwa amefika – ziara bado haijafungwa",  
refArrivedChpStatus: "Yuko kituoni.",  
refArrivedFacilityHeadline: "Mgonjwa yuko hapa – anza huduma",  
refArrivedFacilityStatus: "Mtu amefika.",  
refSeenHouseholdHeadline: "Anaonwa sasa",  
refSeenHouseholdStatus: "Kaa hadi ziara ikamilike.",  
refSeenChpHeadline: "Mgonjwa anaonwa – ziara bado haijafungwa",  
refSeenChpStatus: "Matokeo huja baada ya ziara kufungwa.",  
refSeenFacilityHeadline: "Anahudumiwa – funga ziara ukimaliza",  
refSeenFacilityStatus: "Rekodi kilichotokea huduma ikiisha.",  
refCompletedHouseholdHeadline: "Huduma imekamilika. Tunasubiri kusikia kilichotokea.",  
refCompletedHouseholdStatus: "Matokeo hayajarudi bado.",  
refCompletedChpHeadline: "Ziara imekamilika – inasubiri matokeo",  
refCompletedChpStatus: "Ufuatiliaji unategemea kile kituo kitakachorejesha.",  
refCompletedFacilityHeadline: "Ziara imekamilika – rejesha matokeo",  
refCompletedFacilityStatus: "Jamii inahitaji kujua kilichotokea.",  
refOutcomeHouseholdHeadline: "Hiki ndicho kilichotokea, na cha kufanya sasa",  
refOutcomeHouseholdStatus: "Kituo kimerejesha matokeo.",  
refOutcomeChpHeadline: "Matokeo yamerejeshwa – ufuatiliaji unaweza kuhitajika",  
refOutcomeChpStatus: "Kagua kama kaya hii inahitaji kutembelewa.",  
refOutcomeFacilityHeadline: "Matokeo yametumwa kwa jamii",  
refOutcomeFacilityStatus: "Yamerejeshwa.",  
refQueuedHouseholdHeadline: "Imehifadhiwa kwenye simu hii",  
refQueuedHouseholdStatus: "Huko mtandaoni. Rufaa itatumwa utakaporudi mtandaoni.",  
refQueuedChpHeadline: "Rufaa imepangwa – nje ya mtandao",  
refQueuedChpStatus: "Iko kwenye kifaa hiki. Tuma muunganisho ukirudi.",  
refNoResponseHeadline: "Kituo hakijajibu",  
refNoResponseHouseholdStatus: "Kaa mahali unapoweza kusafiri watakapojibu.",  
refNoResponseOtherStatus: "Rufaa imetumwa – kituo hakijajibu.",  
refNoResponseFacilityStatus: "{f} haijajibu bado.",

actionSendNow: "Tuma sasa",  
actionSendFacility: "Tuma kwa kituo",  
actionSendReferral: "Tuma rufaa",  
actionAccept: "Kubali",  
actionWeveLeft: "Tumeondoka",  
actionTheyLeft: "Wameondoka",  
actionWeveArrived: "Tumefika",

actionTheyreHere: "Wamefika",  
actionStartCare: "Anza huduma",  
actionCompleteVisit: "Kamilisha ziara",  
actionReturnOutcome: "Rejesha kilichotokea",

expectedArrivalTpl: "anatarajiwa kufika baada ya dakika {m}",  
travelTpl: "Safari ya dakika {m}",

failOfflineEyebrow: "Nje ya mtandao",  
failOfflineTitle: "Huko mtandaoni",  
failOfflineBody:

"Tathmini bado inafanya kazi kwenye simu hii. Kutuma rufaa kutasubiri hadi urudi mtandaoni.",

failNoResponseEyebrow: "Hakuna jibu",  
failNoResponseTitle: "Kituo hakijajibu",  
failNoResponseBody:

"Rufaa imetoka kwenye simu hii. Kituo hakijajibu. Kaa mahali unapoweza kusafiri watakapojibu.",

failRedirectedEyebrow: "Imeelekezwa kwingine",  
failRedirectedTitle: "Imeelekezwa kituo kingine",  
failRedirectedBody:

"Kituo hiki hakiwezi kumpokea. Kituo kingine sasa kiko kwenye rufaa.",

failStaleEyebrow: "Taarifa za zamani",  
failStaleTitle: "Taarifa hizi zinaweza kuwa za zamani",  
failStaleBody:

"Kipimo kilichorekodiwa ni cha zamani mno kuongoza uamuzi wa leo kama kipimo kipya.",

failIncompleteEyebrow: "Haijakamilika",  
failIncompleteTitle: "Tathmini hii haijakamilika",  
failIncompleteBody:

"Hakuna taarifa za kutosha kufanya uamuzi huu kwa usalama. Jibu moja zaidi linahitajika.",

failWeakEyebrow: "Muunganisho dhaifu",  
failWeakTitle: "Muunganisho ni dhaifu",  
failWeakBody:

"Hii inaweza kuchukua muda zaidi. Njia ya huduma inabaki kwenye simu hii hadi utumaji uthibitishwe.",

dangerEyebrow: "Dalili ya hatari",  
dangerTitle: "Usisubiri nyumbani na dalili hii.",  
dangerBody: "Nenda kupata huduma sasa.",  
watchEyebrow: "Angalia",  
watchTitle: "Nenda sasa dalili hizi zikianza",

facilityEyebrow: "Kituo cha afya",  
facilityTravelTpl: "Dakika {m} kwa barabara ya kawaida",  
facilityCanTake: "Kituo hiki kinaweza kumpokea",  
facilityWeCanHandle: "Tunaweza kushughulikia rufaa hii",  
facilityMayNot: "Huenda kituo hiki kisiweze kumpokea",

handoffEyebrow: "Makabidhiano",  
whyComingUrgent:

"{who} anahitaji huduma ya dharura. Dalili za hatari zilibainika katika tathmini ya jamii.",

whyComingToday: "{who} anapaswa kuonwa leo. Hii si hali ya kusubiri nyumbani.",

whyComingDefault: "{who} yuko kwenye rufaa ya Tunza.",

whoSubjectChild: "Mtoto kutoka kaya ya majaribio",

whoSubjectSelf: "Mtu mzima (mwenyewe) kutoka kaya ya majaribio",

whoSubjectOther: "Mtu kutoka kaya ya majaribio",

handoffWho: "Nani",

handoffDescribed: "Walichoeleza",

handoffAwake: "Macho",

handoffBreathing: "Kupumua",

handoffDrinking: "Kunywa",

handoffDuration: "Muda gani",

handoffTemp: "Joto la mwili (jaribio)",

handoffPhotoOnly: "Picha pekee (jaribio)",

handoffNotDescribed: "Hakikuelezwa",

handoffStillMissing: "Bado zinakosekana",

whoValueSelf: "Mtu mzima (mwenyewe) · kaya ya majaribio",

```

whoValueAdult: "Mtu mzima · kaya ya majaribio",
whoValueChild: "Mtoto · kaya ya majaribio",
whoValueUnknown: "Hajabainishwa · kaya ya majaribio",
unknownValue: "Haijulikani",
freshnessAssessedTpl: "ilitathminiwa dakika {m} zilizopita",
freshnessOld: "ilirekodiwa miezi 8 iliyopita",

outcomeQuestion: "Nini kilitokea?",
outcomeTreated: "Alionwa na kutibiwa",
outcomeHigher: "Alihitaji kituo cha ngazi ya juu",
outcomeNoShow: "Hakufika",
outcomeUnknown: "Matokeo hayajulikani",

startEncounter: "Anza tathmini",
noEncounter: "Hakuna tathmini inayoendelea",
noEncounterDetail: "Anza moja kunasa kinachoendelea, kisha uamue.",
noIncoming: "Hakuna rufaa inayokuja",
noIncomingDetail: "Kaya au CHP akituma rufaa, itaonekana hapa.",
otherActions: "Vitendo vingine",
redirect: "Elekeza kwingine",
askMore: "Uliza zaidi",
needsAttention: "Inahitaji umakini",
needsAction: "Inahitaji hatua",
missingInformation: "Taarifa zinazokosekana",
whoNeedsFollowUp: "Nani anahitaji ufuatiliaji",
nothingWaiting: "Hakuna kinachosubiri",
encounterTitle: "Tathmini",
incomingTitle: "Zinazolingia",
fuWatchHome: "Kaya ya majaribio – angalia dalili za hatari",
fuWatchHomeDetail:
 "Walishauriwa kubaki nyumbani. Hakikisha bado anaweza kunywa na kuamka.",
fuConfirmTravel: "Safari haijathibitishwa",
fuConfirmTravelDetail: "Kituo kimekubali. Thibitisha kama kaya imeondoka.",
fuPostOutcome: "Kaya ya majaribio – fuatilia baada ya huduma",
fuPostOutcomeDetail: "Thibitisha wanapata nafuu nyumbani.",

viewingAs: "Unaangalia kama",
roleHousehold: "Kaya",
roleChp: "CHP",
roleFacility: "Kituo",
demoLabel: "Demo",
namedConditions: "Hali zilizotajwa",
startOver: "Anza upya",
languageButton: "English",
languageAria: "Switch language to English",
disclaimer: "Si mbadala wa huduma za dharura au matibabu ya kitaalamu.",

contractObservedFacts: "Mambo yaliyotajwa",
contractRetrievedEvidence: "Ushahidi ulioletwa",
contractDerivedFeatures: "Vipengele vilivyokokotwa",
contractHypotheses: "Mawazo",
contractUncertainties: "Yasiyojulikana",
contractContradictions: "Kinachokinzana",
contractRequiredInformation: "Taarifa zinazohitajika",
contractRedFlags: "Alama zilizorekodiwa",
contractNextAction: "Hatua inayofuata",
contractProvenance: "Chanzo",
};

const TABLES: Record<Locale, Record<CopyKey, string>> = { en, sw };

export function t(key: CopyKey, locale: Locale): string {
 return TABLES[locale][key];
}

```

```
/** Replace {name} placeholders in a copy template. */
export function fill(
 text: string,
 vars: Record<string, string | number>,
): string {
 return text.replace(/\{(\w+)\}/g, (match, name: string) =>
 name in vars ? String(vars[name]) : match,
);
}

export const copyTables = TABLES;
```

## DESIGN.md

The product contract: one next action, three surfaces, the referral spine.

SHA-256 20241fbae5a7d68a9a11c8fba27fe104a3026ccb67c26a6291183e5d396a3524 · 319 lines

### # Tunza design

One obvious next action.

This document is the design contract for the Tunza product. It sits beside the README: the README describes what the system is; this describes what a person holding a phone actually experiences, and the rules that keep that experience simple while the system underneath grows.

The whole product is designed around one idea:

At every moment, Tunza should make the next action obvious.

Complexity is allowed to live underneath Tunza. The person using it should never feel that complexity.

---

The test every screen must pass

Every screen answers one question:

What do I do next?

Each screen gets one dominant answer to that question. One primary action, one clear status, and secondary information only when someone asks for it.

If a screen ends up with five equally important buttons, the problem is not layout. It means a product decision underneath the screen has not been resolved. Resolve the decision, then design the screen.

This is the review question for every screen, in every role, in every state – including the failure states.

---

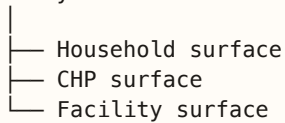
One foundation, three surfaces

Tunza is not three frontends.

The household, the Community Health Promoter, and the receiving facility are looking at the same care path from different angles. If we build them separately, one interface will eventually say a referral is accepted while another still thinks it is pending; translations will drift; offline behavior will be implemented three different ways; and every new workflow will be built three times.

So the structure is:

one shared system underneath



Three very calculated surfaces on top of one foundation.

Each surface exists to do a small number of jobs:

#### Household

- tell Tunza what is wrong;
- answer the few questions that actually matter;
- understand what to do;
- get to the right care if necessary.

#### CHP

- start or continue an encounter;
- collect missing information;
- see what requires action;
- create or follow the referral;
- know who needs follow-up.

#### Facility / clinician

- understand why this person is coming before they arrive;
- decide whether we can handle them;
- accept, redirect, or ask for more information;
- return what happened.

Three views of the same journey. Anything that appears on more than one surface is the same component underneath.

---

#### The household journey

Start with almost nothing.

##### 1. Who needs help?

Me. My child. Someone else. Nothing else on the screen.

##### 2. What's happening?

Three natural ways to answer: speak, type, or add a photo when it is relevant. No form. No menu of body parts.

##### 3. One question at a time.

This is not a chat interface and it is not a medical form. Each step is:

- a clear question;
- a small set of significant choices;
- an obvious "I don't know";
- the ability to speak the answer where useful.

The interaction should feel controlled and intentional. Every question appears because it changes the outcome – never because it makes the product look medically impressive. The missing-information check in the clinical core decides what to ask next; the UI only ever shows one question.

"I don't know" is a first-class answer, not a failure. The system is allowed to say it does not have

enough information (see the README: forced certainty is a defect).

#### 4. The answer.

Radically simpler than symptom-checker output. The top of the screen says one of:

```
Go now.
Get care today.
Monitor at home for now.
I need one more answer before I can safely tell you.
```

Everything else – reasoning, warning signs, what to watch for – sits underneath that. Warning signs appear on every assessment, but they support the decision; they do not compete with it.

#### 5. The transformation.

If the person needs care, the UI fundamentally changes. It stops being an assessment and becomes a care path: where to go, how the facility responded, what to bring, what happens next. This is the moment Tunza stops resembling a symptom checker, and it should look like it.

---

The referral is the spine

There is one authoritative referral object, and it moves through the lifecycle the README defines:

CREATED → SENT → RECEIVED → ACCEPTED → PATIENT\_MOVING → ARRIVED → SEEN → COMPLETED → OUTCOME\_RETURNED

with named failures (NO\_ACKNOWLEDGEMENT, WRONG\_CARE\_LEVEL, SERVICE\_UNAVAILABLE, CAPACITY\_UNAVAILABLE, TRANSPORT\_FAILURE, PATIENT\_DECLINED, REFERRED\_ONWARD, LOST\_TO\_FOLLOWUP).

The UI never invents state and never stores its own copy of it. It translates the canonical state into whatever is relevant to the person looking at it.

The same event, three presentations:

referral.accepted

|           |                                                         |
|-----------|---------------------------------------------------------|
| Household | "Facility accepted – you can leave now."                |
| CHP       | "Referral accepted – patient travel not yet confirmed." |
| Facility  | "Incoming urgent referral – expected arrival 42 min."   |

That translation lives in exactly one place in the codebase: a single module that maps (referral state × role × language) to presentation. No surface hand-writes its own referral copy. That is how the three surfaces stay honest with each other, and how English and Kiswahili stay in step.

The internal state names never appear on screen.

---

The component set

v1 is built from a very small set of reusable components. Every role, every screen, is assembled from these:

##### 1. Assessment Question

One question, significant choices, obvious "I don't know", optional voice answer. Used by household and CHP.

##### 2. Decision / Result

The verdict at the top, support underneath. Household sees the plain-language version; CHP sees the same decision with clinical context.

##### 3. Warning State

The signs that change everything. Present on every assessment, styled to inform, not to alarm decoratively.

#### 4. Referral Status

The current state of the one referral object, translated for the viewer. The same component on all three surfaces.

#### 5. Facility Card

Where to go and why this facility: capability, distance, response. Selection, not browsing.

#### 6. Attention Needed Card

The unit of the CHP and facility work lists. One person, one reason they need attention, one action.

#### 7. Patient Handoff View

What the facility sees before and at arrival: why this person is coming, what is already known, what is missing.

No screen gets a custom one-off component until it has failed to be built from these. This is also how the product compounds: improve the referral component once, and every role benefits from the same improvement.

---

Failure is a designed state

The failure states get the same design attention as the happy path, because in the environments Tunza serves they are normal states, not exceptions:

|                       |                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------|
| Offline               | "Saved on this phone. It will send when the connection returns." The safety path still works. |
| Weak connection       | Sending in the background; the person is never blocked on a spinner to be safe.               |
| No facility response  | Time-boxed. Then: what to do instead, stated plainly.                                         |
| Redirected            | The new destination is the dominant information, with the reason underneath.                  |
| Stale information     | Values carry their age ("measured 8 minutes ago"). Old data is shown as old, never as live.   |
| Incomplete assessment | "I need one more answer before I can safely tell you" – a state, not an error.                |

No generic error screens. Every failure state answers the same question as every other screen: what do I do next?

---

Type

One typeface: Inter (variable).

It is extremely readable on small screens, has a high x-height, clear numerals for vitals and times, handles our languages cleanly, and the variable file means we do not ship a stack of separate weights.

```
font-family: "InterVariable", Inter, Roboto, system-ui, sans-serif;
```

The fallback is designed, not accidental: if the font fails to load, or we choose not to load it in a constrained environment, Android falls to Roboto and nothing looks broken.

Weights:

|     |                         |
|-----|-------------------------|
| 400 | supporting text         |
| 500 | labels                  |
| 600 | actions and status      |
| 700 | the major decision only |

If everything is bold, nothing is. 700 is reserved for the one thing the screen exists to say.

Anything clinical – vitals, travel time, countdowns, expected arrival – uses tabular numerals (font-

variant-numeric: tabular-nums) so values do not visually jump as they update.

Type scale, five steps, nothing else:

|          |         |     |                              |
|----------|---------|-----|------------------------------|
| decision | 28 / 34 | 700 |                              |
| heading  | 20 / 26 | 600 |                              |
| body     | 16 / 24 | 400 |                              |
| label    | 14 / 18 | 500 |                              |
| caption  | 12 / 16 | 400 | provenance, freshness, legal |

No second decorative typeface in the app. If Tunza wants a more ownable brand feel, that lives in the wordmark and the marketing site. The product stays stable and readable.

---

## Color

Tunza's finish is matte red on warm paper.

Not alarm red. A dark, low-lightness oxblood – the "matte" in a flat interface comes from darkness, not gloss or texture – sitting on cream, the way institutions that live on a dark red do (Stanford's whole identity runs on Cardinal #8C1515). The app should feel like a considered object, not a hospital form.

Because this is a health product, the red is carried under a strict two-red discipline, the documented pattern for red brands that also raise alarms (Stanford splits Cardinal from its brighter Digital red for interaction; USWDS separates brand theme tokens from reserved state tokens; clinical-alert guidance reserves red for the top severity tier only, so alarm fatigue never sets in):

the BRAND red is dark and matte, and it is the action role – filled surfaces only:  
the one primary button, the wordmark, focus and selection;

the EMERGENCY red is distinctly brighter and more saturated, and appears only  
as text and tinted panels on danger decisions and status – never as a filled  
button, never as chrome;

the two never swap roles, and the split is enforced by test, not convention.

|              |         |                                                                              |                 |
|--------------|---------|------------------------------------------------------------------------------|-----------------|
| --action     | #7c1f18 | Tunza Red. The matte brand red; the one primary action, the wordmark, focus. |                 |
| --brand-deep | #4a1210 | The deep end of the brand red – the home screen's ground, nothing else.      |                 |
| --urgent     | #b3261e | Emergencies only. Go now, danger signs.                                      | (soft: #f8e4df) |
| --today      | #8a4b12 | Get care today. Time-boxed.                                                  | (soft: #f6e6d0) |
| --watch      | #1d4a73 | Monitor at home. Watch states.                                               | (soft: #e3eef7) |
| --warn       | #7a4e0b | Degraded states: offline, weak, stale, incomplete.                           | (soft: #f7edd6) |
| --ink        | #1c1916 | primary text                                                                 |                 |
| --ink-soft   | #5c564c | supporting text                                                              |                 |
| --line       | #e2d8c8 | borders, dividers                                                            |                 |
| --ground     | #e8e0d2 | page ground behind the column                                                |                 |
| --paper      | #f4efe6 | the app surface                                                              |                 |
| --raised     | #fffcf7 | cards, inputs                                                                |                 |

The front door wears the brand; every working screen wears paper. The household home screen – carried over from the earlier Tunza app's landing – is the one full-brand surface, and it is full-screen: the ground, the header chrome, and the footer all wear the red (brand-deep to Tunza Red), with the wordmark and controls in white on it. On it: the tagline, one inverted primary action (raised surface, brand-red text), quiet frosted cards for the secondary paths, and the mission line. The moment work begins, the product returns to paper, and red returns to being scarce.

The monitor-at-home verdict stays calm blue, and doubly so now: red-green is the most impaired axis of human color vision, and with red carrying the brand there is no green in the product to be confused with. Amber for "today" and "degraded" follows the same severity ladder clinical alerting uses (red above orange above yellow).

Two constraints specific to where Tunza lives:

the red cross emblem is criminally protected in Kenya (Act No. 29 of 1965 and Geneva Conventions law) – no cross motifs anywhere, and the brand red never appears as a saturated red-on-white lockup; it lives dark, on warm cream;

emergency meaning never rides on color alone – every urgent state carries its words, in both languages, exactly as SC 1.4.1 requires.

Rules:

the emergency red appears only on danger decisions and status – never as decoration, never as a button fill;

one action color, used by exactly one element per screen – red stays scarce even as the brand;

every text pairing at WCAG AA minimum against cream (a stricter test than white), aimed at cheap screens in sunlight – verified in tests/contrast.test.ts, which also asserts the emergency red stays at least 1.5× brighter than the brand red;

status is never communicated by color alone – always color plus words;

no raw hex in components – every color goes through these tokens.

---

Layout

a single column; content in reading order; the decision at the top;

one primary action per screen – full width, anchored at the bottom, reachable with a thumb;

minimum touch target 48px;

spacing on a 4px grid, using a small fixed set (4, 8, 12, 16, 24, 32);

one corner radius (12px) everywhere;

secondary information behind one tap ("More" / expandable), never competing with the decision;

motion minimal – state changes announce themselves with a quiet transition, nothing celebratory.

Language rules are layout rules here: sentence case, plain words, second person. Every string exists in English and Kiswahili from day one, and a screen is not finished until the decision fits at 700 in both languages.

---

How this maps to the build

One Next.js app (the existing stack: App Router, TypeScript, Tailwind, Supabase). Role determines the surface; the foundation is shared.

tokens            Tailwind theme: the colors, type scale, spacing, radius above. No raw hex or px in components.

components      the seven components, each implemented with all of its states – including failure and offline states – and tested in both languages.

translation      one module: (referral state × role × language) → presentation. The only place referral copy exists. The copy table is typed so that a missing Kiswahili string is a compile error, not a runtime fallback.

surfaces          household, CHP, facility – thin compositions of the seven components over the same data.

Build order for v1:

1. tokens and type;
2. Assessment Question + Decision/Result – the household path end to end;
3. Referral Status + Facility Card – the care-path transformation;
4. Attention Needed Card – the CHP surface;
5. Patient Handoff View – the facility surface;
6. failure states across all of it, treated as part of each step, not an afterthought.

Vitest covers component states the same way it covers logic: every component renders every state it

claims to support, in both languages, before it ships.

---

What this design refuses to do

- a chat interface for assessment;
- more than one question on screen at a time;
- more than one primary action on a screen;
- referral state stored or invented by a surface;
- urgency color used decoratively;
- a second typeface in the product;
- a generic error screen;
- a screen shipped in one language;
- a new component while an existing one would do.

The objective, restated from the README in design terms:

make the next action obvious, make the path into care legible, and make the same referral tell the truth to everyone looking at it.

## lib/types.ts

Roles, the four decision kinds, the referral lifecycle, the encounter shape.

SHA-256 a2d0068bde83840c300f06a03d8050fd70eb6b644b23aa2b14f8606b2a29f738 · 143 lines

```
import type { CopyKey } from "../copy";

export type Role = "household" | "chp" | "facility";

/**
 * The canonical referral lifecycle from the README. Internal names never
 * appear on screen; the copy layer translates them per role and language.
 */
export type ReferralStage =
 | "created"
 | "sent"
 | "received"
 | "accepted"
 | "patient_moving"
 | "arrived"
 | "seen"
 | "completed"
 | "outcome_returned";

export type NamedFailure =
 | "offline"
 | "no_facility_response"
 | "redirected"
 | "stale_information"
 | "incomplete_assessment"
 | "weak_connection";

export type DecisionKind =
 | "go_now"
 | "get_care_today"
 | "monitor_at_home"
 | "need_one_more_answer";
```

```

export type OutcomeCode =
 | "treated"
 | "referred_onward"
 | "did_not_arrive"
 | "unknown";

export type AskMoreCode = "can_walk";

export type PersonKind = "self" | "household_adult" | "child" | "unknown";

export type QuestionId =
 | "who"
 | "what"
 | "awake"
 | "breathing"
 | "drinking"
 | "duration"
 | "main_problem";

export type AwakeAnswer = "alert" | "sleepy" | "not_waking" | "unknown";
export type BreathingAnswer = "fine" | "difficult" | "severe" | "unknown";
export type DrinkingAnswer = "yes" | "little" | "no" | "unknown";
export type DurationAnswer = "today" | "two_days" | "longer" | "unknown";
export type MainProblemAnswer =
 | "breathing"
 | "fever"
 | "injury"
 | "stomach"
 | "other"
 | "unknown";

export type AssessmentAnswers = {
 who: PersonKind | null;
 presentation: string;
 photoAttached: boolean;
 awake: AwakeAnswer | null;
 breathing: BreathingAnswer | null;
 drinking: DrinkingAnswer | null;
 duration: DurationAnswer | null;
 mainProblem: MainProblemAnswer | null;
};

/** Decisions carry copy keys, not sentences – language is applied at render. */
export type Decision = {
 kind: DecisionKind;
 reasonKeys: CopyKey[];
 dangerSignKeys: CopyKey[];
 watchSignKeys: CopyKey[];
};

export type ReferralHistoryEntry = {
 stage: ReferralStage;
 at: string;
 by: Role;
 note?: string;
};

export type Referral = {
 id: string;
 encounterId: string;
 stage: ReferralStage;
 facilityId: string;
 expectedArrivalMinutes: number;
 queued: boolean;
};

```

```

 failure: NamedFailure | null;
 askMore: AskMoreCode | null;
 outcome: OutcomeCode | null;
 history: ReferralHistoryEntry[];
 createdAt: string;
 updatedAt: string;
};

export type Facility = {
 id: string;
 name: string;
 travelMinutes: number;
 canHandleUrgent: boolean;
 services: string[];
};

export type Encounter = {
 id: string;
 answers: AssessmentAnswers;
 asked: QuestionId[];
 currentQuestion: QuestionId | null;
 decision: Decision | null;
 startedBy: Role;
 createdAt: string;
 updatedAt: string;
};

/** The household surface's top-level view; other roles ignore it. */
export type HouseholdView = "home" | "path" | "nearby" | "gate";

/** Roles behind the health-worker gate. */
export type GatedRole = "chp" | "facility";

export type CareState = {
 role: Role;
 locale: "en" | "sw";
 view: HouseholdView;
 /** Device-level access grants for the gated surfaces (placeholder for real
 * worker identity; codes are checked server-side in /api/access). */
 grants: Record<GatedRole, boolean>;
 gateRole: GatedRole | null;
 injectedFailures: NamedFailure[];
 encounter: Encounter | null;
 referral: Referral | null;
 online: boolean;
};

```

## lib/assessment.ts

The demo floor. decide() is the only approved path. Not clinically validated.

SHA-256 e9a2cdeaed47f534d16163063b1c159b4040fc7eba07deed655c1d247921df87 · 375 lines

```

import { t, type CopyKey, type Locale } from "../copy";
import type {
 AskMoreCode,
 AssessmentAnswers,
 Decision,
 DecisionKind,
 QuestionId,

```

```

} from "./types";

export const EMPTY_ANSWERS: AssessmentAnswers = {
 who: null,
 presentation: "",
 photoAttached: false,
 awake: null,
 breathing: null,
 drinking: null,
 duration: null,
 mainProblem: null,
};

export const QUESTION_ORDER: QuestionId[] = [
 "who",
 "what",
 "awake",
 "breathing",
 "drinking",
 "duration",
 "main_problem",
];

/**
 * Demonstration triage rules, deliberately conservative. Danger patterns
 * listen in English and Kiswahili. Not clinically validated – the deterministic
 * safety layer described in the README replaces this before real use.
 */
const DANGER_PATTERNS: { re: RegExp; key: CopyKey }[] = [
 {
 re: /unconscious|not waking|won't wake|hajitambui|haamki/,
 key: "dangerNotWaking",
 },
 { re: /seizure|convulsion|fitting|kifafa|degedege/, key: "dangerSeizure" },
 { re: /not breathing|stopped breathing|hapumui/, key: "dangerNotBreathing" },
 { re: /blue lips|blue face|midomo ya bluu/, key: "dangerBlueLips" },
 {
 re: /bleeding (a lot|heavily|won't stop)|heavy bleeding|damu nyingi/,
 key: "dangerHeavyBleeding",
 },
];

const WATCH_SIGN_KEYS: CopyKey[] = [
 "watchNotWaking",
 "watchBreathingHard",
 "watchCannotDrink",
 "watchSeizureBleeding",
];

export function presentationDangerSigns(text: string): CopyKey[] {
 const lower = text.toLowerCase();
 return DANGER_PATTERNS.filter((item) => item.re.test(lower)).map(
 (item) => item.key,
);
}

export function hasFever(text: string): boolean {
 return /fever|hot body|temperature|homa/.test(text.toLowerCase());
}

export function hasInjury(text: string): boolean {
 return /wound|cut|bleed|injury|burn|fall|jeraha|ajali|damu/.test(
 text.toLowerCase(),
);
}

```

```

}

function tooIncomplete(answers: AssessmentAnswers): boolean {
 const critical = [answers.awake, answers.breathing, answers.drinking];
 const unknownCount = critical.filter(
 (value) => value === "unknown" || value === null,
).length;
 const noStory = answers.presentation.trim().length === 0 && !answers.photoAttached;
 return unknownCount >= 2 && noStory;
}

function confidentHomeWatch(answers: AssessmentAnswers): boolean {
 return (
 answers.awake === "alert" &&
 answers.breathing === "fine" &&
 answers.drinking === "yes"
);
}

export function decide(answers: AssessmentAnswers): Decision {
 const dangerSignKeys: CopyKey[] = [
 ...presentationDangerSigns(answers.presentation),
];
 const reasonKeys: CopyKey[] = [];

 if (answers.awake === "not_waking") {
 dangerSignKeys.push("dangerNotWaking");
 }
 if (answers.breathing === "severe") {
 dangerSignKeys.push("dangerStrugglingBreathe");
 }
 if (answers.drinking === "no" && answers.who === "child") {
 dangerSignKeys.push("dangerChildNoDrink");
 }
 if (answers.mainProblem === "injury" && hasInjury(answers.presentation)) {
 if (/heavy|won't stop|a lot|nyingi/.test(answers.presentation.toLowerCase())) {
 dangerSignKeys.push("dangerHeavyBleeding");
 }
 }

 const uniqueDanger = [...new Set(dangerSignKeys)];

 if (uniqueDanger.length > 0) {
 return {
 kind: "go_now",
 reasonKeys: uniqueDanger,
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
 }

 if (answers.breathing === "difficult" && answers.awake === "sleepy") {
 return {
 kind: "go_now",
 reasonKeys: ["reasonSleepyHardBreathing"],
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
 }

 if (tooIncomplete(answers) && answers.mainProblem !== null) {
 return {
 kind: "need_one_more_answer",
 reasonKeys: ["reasonTooLittleKnown"],
 };
 }
}

```

```

 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
}

if (tooIncomplete(answers) && answers.duration !== null) {
 return {
 kind: "need_one_more_answer",
 reasonKeys: ["reasonOneMoreSafer"],
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
}

let kind: DecisionKind = "monitor_at_home";

if (answers.breathing === "difficult") {
 kind = "get_care_today";
 reasonKeys.push("reasonBreathingHarder");
}
if (answers.drinking === "little") {
 kind = "get_care_today";
 reasonKeys.push("reasonDrinkingLittle");
}
if (answers.drinking === "no") {
 kind = "get_care_today";
 reasonKeys.push("reasonNotDrinking");
}
if (answers.awake === "sleepy") {
 kind = "get_care_today";
 reasonKeys.push("reasonSleepy");
}
if (answers.who === "child" && hasFever(answers.presentation)) {
 kind = "get_care_today";
 reasonKeys.push("reasonChildFever");
}
if (answers.mainProblem === "fever" && answers.who === "child") {
 kind = "get_care_today";
 reasonKeys.push("reasonChildFever");
}
if (answers.mainProblem === "injury") {
 kind = "get_care_today";
 reasonKeys.push("reasonInjurySeen");
}
if (answers.mainProblem === "breathing") {
 kind = "get_care_today";
 reasonKeys.push("reasonBreathingMain");
}
if (
 answers.duration === "longer" &&
 (hasFever(answers.presentation) || answers.mainProblem === "fever")
) {
 kind = "get_care_today";
 reasonKeys.push("reasonTooLongHome");
}

if (kind === "monitor_at_home") {
 if (!confidentHomeWatch(answers) && answers.mainProblem === null) {
 return {
 kind: "need_one_more_answer",
 reasonKeys: ["reasonOneMoreSafer"],
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
 }
}

```

```

 }
 if (!confidentHomeWatch(answers) && tooIncomplete(answers)) {
 return {
 kind: "need_one_more_answer",
 reasonKeys: ["reasonTooLittleKnown"],
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
 }
 reasonKeys.push("reasonNoDanger");
 }

 return {
 kind,
 reasonKeys: [...new Set(reasonKeys)],
 dangerSignKeys: uniqueDanger,
 watchSignKeys: WATCH_SIGN_KEYS,
 };
}

export function shouldStopForDecision(
 answers: AssessmentAnswers,
 lastAsked: QuestionId,
): boolean {
 const decision = decide(answers);

 if (lastAsked === "what" && decision.kind === "go_now") {
 return true;
 }
 if (lastAsked === "awake" && answers.awake === "not_waking") {
 return true;
 }
 if (lastAsked === "breathing" && answers.breathing === "severe") {
 return true;
 }
 if (
 lastAsked === "drinking" &&
 answers.drinking === "no" &&
 answers.who === "child"
) {
 return true;
 }
 if (lastAsked === "duration") {
 return true;
 }
 if (lastAsked === "main_problem") {
 return true;
 }
 return false;
}

export function nextQuestion(
 answers: AssessmentAnswers,
 lastAsked: QuestionId,
): QuestionId | null {
 if (shouldStopForDecision(answers, lastAsked)) {
 return null;
 }

 switch (lastAsked) {
 case "who":
 return "what";
 case "what":
 return "awake";
 }

```

```

 case "awake":
 return "breathing";
 case "breathing":
 return "drinking";
 case "drinking":
 return "duration";
 case "duration":
 return "main_problem";
 case "main_problem":
 return null;
 }
}

export function applyAnswer(
 answers: AssessmentAnswers,
 question: QuestionId,
 value: string,
): AssessmentAnswers {
 const next = { ...answers };
 switch (question) {
 case "who":
 next.who = value as AssessmentAnswers["who"];
 break;
 case "what":
 next.presentation = value;
 break;
 case "awake":
 next.awake = value as AssessmentAnswers["awake"];
 break;
 case "breathing":
 next.breathing = value as AssessmentAnswers["breathing"];
 break;
 case "drinking":
 next.drinking = value as AssessmentAnswers["drinking"];
 break;
 case "duration":
 next.duration = value as AssessmentAnswers["duration"];
 break;
 case "main_problem":
 next.mainProblem = value as AssessmentAnswers["mainProblem"];
 break;
 }
 return next;
}

export type MissingItem = {
 id: string;
 label: string;
 detail?: string;
};

export function missingInfo(
 answers: AssessmentAnswers,
 askMore: AskMoreCode | null,
 locale: Locale,
): MissingItem[] {
 const items: MissingItem[] = [];

 if (!answers.who || answers.who === "unknown") {
 items.push({
 id: "who",
 label: t("missingWho", locale),
 detail: t("missingWhoDetail", locale),
 });
 }
}

```

```

}
if (!answers.presentation.trim() && !answers.photoAttached) {
 items.push({ id: "what", label: t("missingWhat", locale) });
}
if (!answers.awake || answers.awake === "unknown") {
 items.push({ id: "awake", label: t("missingAwake", locale) });
}
if (!answers.breathing || answers.breathing === "unknown") {
 items.push({ id: "breathing", label: t("missingBreathing", locale) });
}
if (!answers.drinking || answers.drinking === "unknown") {
 items.push({ id: "drinking", label: t("missingDrinking", locale) });
}
if (askMore === "can_walk") {
 items.push({
 id: "ask-more",
 label: t("askMoreCanWalk", locale),
 detail: t("missingAskMoreDetail", locale),
 });
}
return items;
}

export function decisionHeadline(kind: DecisionKind, locale: Locale): string {
 switch (kind) {
 case "go_now":
 return t("goNow", locale);
 case "get_care_today":
 return t("getCareToday", locale);
 case "monitor_at_home":
 return t("monitorAtHome", locale);
 case "need_one_more_answer":
 return t("needOneMore", locale);
 }
}

export function decisionStatus(kind: DecisionKind, locale: Locale): string {
 switch (kind) {
 case "go_now":
 return t("goNowStatus", locale);
 case "get_care_today":
 return t("getCareStatus", locale);
 case "monitor_at_home":
 return t("monitorStatus", locale);
 case "need_one_more_answer":
 return t("needOneMoreStatus", locale);
 }
}

```

## lib/clinical/contract.ts

Ten-section contract, disposition table, validators. No new user-visible headline.

SHA-256 da15f6167c3e6aad9e2ca7ea6e7a07ba54ae8e60fd7232b825942864090f9976 · 849 lines

```

import type { CopyKey } from "../copy";
import type { DecisionKind } from "../types";

/**
 * Internal clinical contract. Runtime shape check only.

```

```

* Not a clinical protocol, not a diagnosis, and not a training record.
*
* The ten sections stay separate. Nothing in this module collapses them
* into one paragraph or admits retrieved authority (no approved sources).
*/

export const CONTRACT_VERSION = "clinical-contract/v1";

export const SECTION_KEYS = [
 "observed_facts",
 "retrieved_evidence",
 "derived_features",
 "hypotheses",
 "uncertainties",
 "contradictions",
 "required_information",
 "red_flags",
 "next_action",
 "provenance",
] as const;

export type AssessmentSection = (typeof SECTION_KEYS)[number];

const OBJECT_SECTIONS = new Set<AssessmentSection>([
 "retrieved_evidence",
 "next_action",
 "provenance",
]);

export const RETRIEVAL_STATUSES = [
 "not_run",
 "empty",
 "failed",
 "unavailable",
] as const;

export type RetrievalStatus = (typeof RETRIEVAL_STATUSES)[number];

/** Reserved for a later wave. Not admitted by the validator. */
export const RESERVED_TRUST_TIERS = ["A", "B", "C", "D", "E"] as const;

export type ReservedTrustTier = (typeof RESERVED_TRUST_TIERS)[number];

export const TRUST_TIER_NONE = "none" as const;

export type TrustTier = typeof TRUST_TIER_NONE | ReservedTrustTier;

/**
* Shape a future approved artifact would need. This wave rejects every item.
* Unverified text must not become clinical authority by being present.
*/
export type ReservedEvidenceItem = {
 source_id: string;
 publisher: string;
 jurisdiction: string;
 document_version: string;
 publication_date: string;
 effective_date: string;
 review_date: string;
 section: string;
 retrieval_score: number | null;
 reranking_score: number | null;
 trust_tier: ReservedTrustTier;
 content_hash: string;

```

```

};

export type RetrievedEvidence = {
 retrieval_status: RetrievalStatus;
 trust_tier: typeof TRUST_TIER_NONE;
 items: [];
};

export const ACTION_CLASSES = ["ANSWER", "ASK", "ABSTAIN", "ESCALATE"] as const;

export type ActionClass = (typeof ACTION_CLASSES)[number];

/** Headlines that already exist. This module adds none. */
export type ExistingHeadlineKey =
 | "goNow"
 | "getCareToday"
 | "monitorAtHome"
 | "needOneMore";

type _HeadlineIsCopyKey = ExistingHeadlineKey extends CopyKey ? true : never;
const _headlineIsCopyKey: _HeadlineIsCopyKey = true;
void _headlineIsCopyKey;

export const DISPOSITIONS = [
 "go_now",
 "get_care_today",
 "monitor_at_home",
 "need_one_more_answer",
 "escalate",
 "abstain",
] as const;

export type Disposition = (typeof DISPOSITIONS)[number];

type _CoversDecisionKind = DecisionKind extends Disposition ? true : never;
const _coversDecisionKind: _CoversDecisionKind = true;
void _coversDecisionKind;

export type DispositionRow = {
 action: ActionClass;
 /** Null means there is no user-visible headline. Do not invent one. */
 headlineKey: ExistingHeadlineKey | null;
};

/**
 * One table. `need_one_more_answer` is the only missing-info ASK state.
 * `escalate` reuses the existing urgent headline. `abstain` has none.
 */
export const DISPOSITION_TABLE: Record<Disposition, DispositionRow> = {
 go_now: { action: "ESCALATE", headlineKey: "goNow" },
 get_care_today: { action: "ANSWER", headlineKey: "getCareToday" },
 monitor_at_home: { action: "ANSWER", headlineKey: "monitorAtHome" },
 need_one_more_answer: { action: "ASK", headlineKey: "needOneMore" },
 escalate: { action: "ESCALATE", headlineKey: "goNow" },
 abstain: { action: "ABSTAIN", headlineKey: null },
};

export function actionClassFor(disposition: Disposition): ActionClass {
 return DISPOSITION_TABLE[disposition].action;
}

export function headlineKeyFor(
 disposition: Disposition,
): ExistingHeadlineKey | null {

```

```

 return DISPOSITION_TABLE[disposition].headlineKey;
}

export function isDisposition(value: unknown): value is Disposition {
 return (
 typeof value === "string" &&
 (DISPOSITIONS as readonly string[]).includes(value)
);
}

/** DecisionKind is a subset. Adding a kind in types.ts fails this file's check. */
export function dispositionFromDecisionKind(kind: DecisionKind): Disposition {
 return kind;
}

export const RESOLUTION_STATUS_UNRESOLVED = "unresolved" as const;

export type ResolutionStatus = typeof RESOLUTION_STATUS_UNRESOLVED;

export const CLINICAL_IMPORTANCE = ["low", "moderate", "high"] as const;

export type ClinicalImportance = (typeof CLINICAL_IMPORTANCE)[number];

export type Contradiction = {
 claim_a: string;
 claim_b: string;
 sources: string[];
 clinical_importance: ClinicalImportance;
 resolution_status: ResolutionStatus;
};

export type ClinicalEvent = {
 event_id: string;
 encounter_id: string;
 type: string;
 value: string | number | boolean | null;
 unit: string | null;
 timestamp: string;
 source: string;
 confidence: number;
 provenance: string;
};

export type ObservedFact = {
 fact_id: string;
 value: string;
 source: string;
 timestamp: string;
 confidence: number;
 provenance: string;
};

export type DerivedFeature = {
 feature_id: string;
 name: string;
 value: string | number | boolean;
 unit: string | null;
 inputs: string[];
};

export type Hypothesis = {
 hypothesis_id: string;
 statement: string;
 supporting_ids: string[];
};

```

```

 contradicting_ids: string[];
 };

export type Uncertainty = {
 uncertainty_id: string;
 statement: string;
};

export type RequiredInformation = {
 question_id: string;
 statement: string;
};

export type RedFlag = {
 flag_id: string;
 code: string;
 source: string;
};

export type NextAction = {
 disposition: Disposition;
 action: ActionClass;
 ideal_action: string | null;
 locally_executable_action: string | null;
 resource_gap: string | null;
};

export type Provenance = {
 rules_version: string;
 model_id: string | null;
 knowledge_version: string | null;
 last_sync: string | null;
 generated_at: string;
};

export type ClinicalAssessment = {
 observed_facts: ObservedFact[];
 retrieved_evidence: RetrievedEvidence;
 derived_features: DerivedFeature[];
 hypotheses: Hypothesis[];
 uncertainties: Uncertainty[];
 contradictions: Contradiction[];
 required_information: RequiredInformation[];
 red_flags: RedFlag[];
 next_action: NextAction;
 provenance: Provenance;
};

export type ContractIssue = {
 path: string;
 message: string;
};

export class ContractValidationError extends Error {
 readonly issues: readonly ContractIssue[];

 constructor(issues: readonly ContractIssue[]) {
 super(
 issues.map((issue) => `${issue.path}: ${issue.message}`).join("; ") ||
 "contract validation failed",
);
 this.name = "ContractValidationError";
 this.issues = issues;
 Object.setPrototypeOf(this, new.target.prototype);
 }
}

```

```

 }
}

const ISO_TIMESTAMP =
 /^{4}-{2}-{2}T{2}:{2}:{2}(?:\.\d+)?(?:Z|[\+-]{2}:{2})$/;

function isRecord(value: unknown): value is Record<string, unknown> {
 return typeof value === "object" && value !== null && !Array.isArray(value);
}

function isNonEmptyString(value: unknown): value is string {
 return typeof value === "string" && value.trim().length > 0;
}

function isIsoTimestamp(value: unknown): value is string {
 return (
 typeof value === "string" &&
 ISO_TIMESTAMP.test(value) &&
 Number.isFinite(Date.parse(value))
);
}

function isConfidence(value: unknown): value is number {
 return typeof value === "number" && Number.isFinite(value) && value >= 0 && value <= 1;
}

function isNullableString(value: unknown): value is string | null {
 return value === null || isNonEmptyString(value);
}

function push(issues: ContractIssue[], path: string, message: string): void {
 issues.push({ path, message });
}

export function emptyRetrievedEvidence(
 retrievalStatus: RetrievalStatus = "not_run",
): RetrievedEvidence {
 return {
 retrieval_status: retrievalStatus,
 trust_tier: TRUST_TIER_NONE,
 items: [],
 };
}

export function demoProvenance(
 rulesVersion: string,
 generatedAt: string,
): Provenance {
 return {
 rules_version: rulesVersion,
 model_id: null,
 knowledge_version: null,
 last_sync: null,
 generated_at: generatedAt,
 };
}

/** Always unresolved. This module has no resolver. */
export function createContradiction(input: {
 claim_a: string;
 claim_b: string;
 sources: string[];
 clinical_importance: ClinicalImportance;
}): Contradiction {

```

```

return {
 claim_a: input.claim_a,
 claim_b: input.claim_b,
 sources: [...input.sources],
 clinical_importance: input.clinical_importance,
 resolution_status: RESOLUTION_STATUS_UNRESOLVED,
};
}

export function minimalAssessment(input: {
 rulesVersion: string;
 generatedAt: string;
 disposition: Disposition;
}): ClinicalAssessment {
 const action = actionClassFor(input.disposition);
 const assessment: ClinicalAssessment = {
 observed_facts: [],
 retrieved_evidence: emptyRetrievedEvidence("not_run"),
 derived_features: [],
 hypotheses: [],
 uncertainties: [],
 contradictions: [],
 required_information: [],
 red_flags: [],
 next_action: {
 disposition: input.disposition,
 action,
 ideal_action: null,
 locally_executable_action: null,
 resource_gap: null,
 },
 provenance: demoProvenance(input.rulesVersion, input.generatedAt),
 };
 return validateClinicalAssessment(assessment);
}

export function validateClinicalAssessment(input: unknown): ClinicalAssessment {
 const issues = assessmentIssues(input);
 if (issues.length > 0) {
 throw new ContractValidationError(issues);
 }
 return input as ClinicalAssessment;
}

export function assessmentIssues(input: unknown): ContractIssue[] {
 const issues: ContractIssue[] = [];
 if (!isRecord(input)) {
 push(issues, "root", "expected an object with ten sections");
 return issues;
 }

 for (const section of SECTION_KEYS) {
 if (!(section in input) || input[section] === undefined) {
 push(issues, section, "missing section");
 continue;
 }
 const value = input[section];
 if (OBJECT_SECTIONS.has(section)) {
 if (!isRecord(value)) {
 push(issues, section, "expected an object");
 }
 } else if (!Array.isArray(value)) {
 push(issues, section, "expected an array");
 }
 }
}

```

```

}

if (issues.length > 0) {
 return issues;
}

const record = input;
validateObservedFacts(record.observed_facts, issues);
validateRetrievedEvidence(record.retrieved_evidence, issues);
validateDerivedFeatures(record.derived_features, issues);
validateHypotheses(record.hypotheses, issues);
validateUncertainties(record.uncertainties, issues);
validateContradictions(record.contradictions, issues);
validateRequiredInformation(record.required_information, issues);
validateRedFlags(record.red_flags, issues);
validateNextAction(record.next_action, issues);
validateProvenance(record.provenance, issues);
return issues;
}

function validateObservedFacts(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `observed_facts[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "fact_id", path, issues);
 requireString(item, "value", path, issues);
 requireString(item, "source", path, issues);
 requireTimestamp(item, "timestamp", path, issues);
 requireConfidence(item, path, issues);
 requireString(item, "provenance", path, issues);
 });
}

function validateRetrievedEvidence(value: unknown, issues: ContractIssue[]): void {
 if (!isRecord(value)) return;
 if (!("retrieval_status" in value)) {
 push(issues, "retrieved_evidence.retrieval_status", "missing retrieval_status");
 } else if (
 typeof value.retrieval_status !== "string" ||
 !(RETRIEVAL_STATUSES as readonly string[]).includes(value.retrieval_status)
) {
 push(
 issues,
 "retrieved_evidence.retrieval_status",
 `expected one of ${RETRIEVAL_STATUSES.join(", ")}`,
);
 }
 if (!("trust_tier" in value)) {
 push(issues, "retrieved_evidence.trust_tier", "missing trust_tier");
 } else if (value.trust_tier !== TRUST_TIER_NONE) {
 push(
 issues,
 "retrieved_evidence.trust_tier",
 "trust_tier must be none until an approved source is named",
);
 }
 if (!("items" in value)) {
 push(issues, "retrieved_evidence.items", "missing items");
 } else if (!Array.isArray(value.items) || value.items.length > 0) {
 push(

```

```

 issues,
 "retrieved_evidence.items",
 "evidence items are not admitted; bundle must be empty",
);
}
}

function validateDerivedFeatures(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `derived_features[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "feature_id", path, issues);
 requireString(item, "name", path, issues);
 if (
 typeof item.value !== "string" &&
 typeof item.value !== "number" &&
 typeof item.value !== "boolean"
) {
 push(issues, `${path}.value`, "expected string, number, or boolean");
 } else if (typeof item.value === "number" && !Number.isFinite(item.value)) {
 push(issues, `${path}.value`, "expected a finite number");
 } else if (typeof item.value === "string" && item.value.trim().length === 0) {
 push(issues, `${path}.value`, "expected a non-empty string");
 }
 if (item.unit !== null && !isNonEmptyString(item.unit)) {
 push(issues, `${path}.unit`, "expected a non-empty string or null");
 }
 requireStringArray(item, "inputs", path, issues);
 });
}

function validateHypotheses(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `hypotheses[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "hypothesis_id", path, issues);
 requireString(item, "statement", path, issues);
 requireStringArray(item, "supporting_ids", path, issues);
 requireStringArray(item, "contradicting_ids", path, issues);
 });
}

function validateUncertainties(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `uncertainties[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "uncertainty_id", path, issues);
 requireString(item, "statement", path, issues);
 });
}

function validateRequiredInformation(value: unknown, issues: ContractIssue[]): void {

```

```

 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `required_information[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "question_id", path, issues);
 requireString(item, "statement", path, issues);
 });
}

function validateRedFlags(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `red_flags[${index}]`;
 if (!isRecord(item)) {
 push(issues, path, "expected an object");
 return;
 }
 requireId(item, "flag_id", path, issues);
 requireString(item, "code", path, issues);
 requireString(item, "source", path, issues);
 });
}

function validateContradictions(value: unknown, issues: ContractIssue[]): void {
 if (!Array.isArray(value)) return;
 value.forEach((item, index) => {
 const path = `contradictions[${index}]`;
 issues.push(...contradictionIssues(item, path));
 });
}

export function contradictionIssues(
 input: unknown,
 path = "contradiction",
): ContractIssue[] {
 const issues: ContractIssue[] = [];
 if (!isRecord(input)) {
 push(issues, path, "expected an object");
 return issues;
 }
 requireString(input, "claim_a", path, issues);
 requireString(input, "claim_b", path, issues);
 if (
 isNonEmptyString(input.claim_a) &&
 isNonEmptyString(input.claim_b) &&
 input.claim_a.trim() === input.claim_b.trim()
) {
 push(issues, `${path}.claim_b`, "claims must differ");
 }
 if (!Array.isArray(input.sources) || input.sources.length === 0) {
 push(issues, `${path}.sources`, "expected a non-empty array of sources");
 } else if (!input.sources.every((source) => isNonEmptyString(source))) {
 push(issues, `${path}.sources`, "each source must be a non-empty string");
 }
 if (
 typeof input.clinical_importance !== "string" ||
 !(CLINICAL_IMPORTANCE as readonly string[]).includes(input.clinical_importance)
) {
 push(
 issues,
 `${path}.clinical_importance`,

```

```

 `expected one of ${CLINICAL_IMPORTANCE.join(", ")}``,
);
}
if (!("resolution_status" in input)) {
 push(issues, `${path}.resolution_status`, "missing resolution_status");
} else if (input.resolution_status !== RESOLUTION_STATUS_UNRESOLVED) {
 push(
 issues,
 `${path}.resolution_status`,
 "contradictions are not resolved here; resolution_status must be unresolved",
);
}
return issues;
}

export function validateContradiction(input: unknown): Contradiction {
 const issues = contradictionIssues(input);
 if (issues.length > 0) {
 throw new ContractValidationError(issues);
 }
 return input as Contradiction;
}

function validateNextAction(value: unknown, issues: ContractIssue[]): void {
 if (!isRecord(value)) return;
 if (!isDisposition(value.disposition)) {
 push(
 issues,
 "next_action.disposition",
 `expected one of ${DISPOSITIONS.join(", ")}``,
);
 }
 if (
 typeof value.action !== "string" ||
 !(ACTION_CLASSES as readonly string[]).includes(value.action)
) {
 push(
 issues,
 "next_action.action",
 `expected one of ${ACTION_CLASSES.join(", ")}``,
);
 } else if (
 isDisposition(value.disposition) &&
 value.action !== actionClassFor(value.disposition)
) {
 push(
 issues,
 "next_action.action",
 `must match disposition table (${actionClassFor(value.disposition)})`,
);
 }
 requireNullableString(value, "ideal_action", "next_action", issues);
 requireNullableString(value, "locally_executable_action", "next_action", issues);
 requireNullableString(value, "resource_gap", "next_action", issues);
 if (
 isNonEmptyString(value.ideal_action) &&
 isNonEmptyString(value.locally_executable_action) &&
 value.ideal_action !== value.locally_executable_action &&
 value.resource_gap === null
) {
 push(
 issues,
 "next_action.resource_gap",
 "gap must be explicit when ideal and local actions differ",
);
 }
}

```

```

);
 }
}

function validateProvenance(value: unknown, issues: ContractIssue[]): void {
 if (!isRecord(value)) return;
 const path = "provenance";
 requireString(value, "rules_version", path, issues);
 requireExplicitNullable(value, "model_id", path, issues, false);
 requireExplicitNullable(value, "knowledge_version", path, issues, false);
 requireExplicitNullable(value, "last_sync", path, issues, true);
 requireTimestamp(value, "generated_at", path, issues);
 if (isNonEmptyString(value.knowledge_version) && value.last_sync === null) {
 push(
 issues,
 "provenance.last_sync",
 "knowledge_version requires last_sync; do not claim a version without a sync time",
);
 }
}

function requireExplicitNullable(
 record: Record<string, unknown>,
 key: string,
 path: string,
 issues: ContractIssue[],
 timestamp: boolean,
): void {
 if (!(key in record)) {
 push(issues, `${path}.${key}`, "missing field; null must be explicit");
 return;
 }
 const value = record[key];
 if (value === null) return;
 if (timestamp) {
 if (!isIsoTimestamp(value)) {
 push(issues, `${path}.${key}`, "expected an ISO timestamp or null");
 }
 return;
 }
 if (!isNonEmptyString(value)) {
 push(issues, `${path}.${key}`, "expected a non-empty string or null");
 }
}

export function clinicalEventIssues(
 input: unknown,
 path = "event",
): ContractIssue[] {
 const issues: ContractIssue[] = [];
 if (!isRecord(input)) {
 push(issues, path, "expected an object");
 return issues;
 }
 requireId(input, "event_id", path, issues);
 requireId(input, "encounter_id", path, issues);
 requireString(input, "type", path, issues);
 if (
 input.value !== null &&
 typeof input.value !== "string" &&
 typeof input.value !== "number" &&
 typeof input.value !== "boolean"
) {
 push(issues, `${path}.value`, "expected string, number, boolean, or null");
 }
}

```

```

 } else if (typeof input.value === "number" && !Number.isFinite(input.value)) {
 push(issues, `${path}.value`, "expected a finite number");
 } else if (typeof input.value === "string" && input.value.trim().length === 0) {
 push(issues, `${path}.value`, "expected a non-empty string or null");
 }
 if (input.unit !== null && !isNonEmptyString(input.unit)) {
 push(issues, `${path}.unit`, "expected a non-empty string or null");
 }
 requireTimestamp(input, "timestamp", path, issues);
 requireString(input, "source", path, issues);
 requireConfidence(input, path, issues);
 requireString(input, "provenance", path, issues);
 return issues;
 }
}

```

```

export function validateClinicalEvent(input: unknown): ClinicalEvent {
 const issues = clinicalEventIssues(input);
 if (issues.length > 0) {
 throw new ContractValidationError(issues);
 }
 return input as ClinicalEvent;
}

```

```

/**
 * Copy-and-append. Rejects a repeated event_id. Does not replace, collapse,
 * or pick a current value.
 */

```

```

export function appendClinicalEvent(
 log: readonly ClinicalEvent[],
 event: ClinicalEvent,
): readonly ClinicalEvent[] {
 if (!Array.isArray(log)) {
 throw new ContractValidationError([
 { path: "log", message: "expected an array of events" },
]);
 }
 const issues: ContractIssue[] = [];
 log.forEach((existing, index) => {
 issues.push(...clinicalEventIssues(existing, `log[${index}]`));
 });
 issues.push(...clinicalEventIssues(event, "event"));
 if (issues.length > 0) {
 throw new ContractValidationError(issues);
 }
 if (log.some((existing) => existing.event_id === event.event_id)) {
 throw new ContractValidationError([
 {
 path: "event.event_id",
 message: "append-only: event_id already recorded",
 },
]);
 }
 const next = [
 ...log.map((existing) => Object.freeze({ ...existing })),
 Object.freeze({ ...event }),
];
 return Object.freeze(next);
}

```

```

/** Every matching event, oldest first. Never reduced to one reading. */
export function eventsOfType(
 log: readonly ClinicalEvent[],
 type: string,
): readonly ClinicalEvent[] {

```

```

return [...log]
 .filter((event) => event.type === type)
 .sort(
 (a, b) =>
 a.timestamp.localeCompare(b.timestamp) ||
 a.event_id.localeCompare(b.event_id),
);
}

function requireId(
 record: Record<string, unknown>,
 key: string,
 path: string,
 issues: ContractIssue[],
): void {
 requireString(record, key, path, issues);
}

function requireString(
 record: Record<string, unknown>,
 key: string,
 path: string,
 issues: ContractIssue[],
): void {
 if (!isNonEmptyString(record[key])) {
 push(issues, `${path}.${key}`, "expected a non-empty string");
 }
}

function requireTimestamp(
 record: Record<string, unknown>,
 key: string,
 path: string,
 issues: ContractIssue[],
): void {
 if (!isIsoTimestamp(record[key])) {
 push(issues, `${path}.${key}`, "expected an ISO timestamp");
 }
}

function requireConfidence(
 record: Record<string, unknown>,
 path: string,
 issues: ContractIssue[],
): void {
 if (!isConfidence(record.confidence)) {
 push(issues, `${path}.confidence`, "expected a number from 0 to 1");
 }
}

function requireStringArray(
 record: Record<string, unknown>,
 key: string,
 path: string,
 issues: ContractIssue[],
): void {
 const value = record[key];
 if (!Array.isArray(value) || !value.every((item) => isNonEmptyString(item))) {
 push(issues, `${path}.${key}`, "expected an array of non-empty strings");
 }
}

function requireNullableString(
 record: Record<string, unknown>,

```

```

 key: string,
 path: string,
 issues: ContractIssue[],
): void {
 if (!(key in record)) {
 push(issues, `${path}.${key}`, "missing field; null must be explicit");
 return;
 }
 if (!isNullableString(record[key])) {
 push(issues, `${path}.${key}`, "expected a non-empty string or null");
 }
 }
}

```

## lib/clinical/state.ts

Append-only encounter projection. No single temperature. No clinical cutoff.

SHA-256 2c0a721d5f4450aa50f7f13f529ef33bd0ceaa19507da1a61df7bfa5159cd218 · 286 lines

```

import type { AssessmentAnswers, Encounter } from "../types";
import {
 appendClinicalEvent,
 createContradiction,
 eventsOfType,
 validateClinicalEvent,
 type ClinicalEvent,
 type Contradiction,
 type DerivedFeature,
} from "../contract";

/**
 * Encounter-scoped projection. Not a patient registry and not a clinical
 * protocol. Observations are append-only. This module does not choose a
 * disposition and does not apply clinical cutoffs.
 */

export const STATE_MODULE = "clinical-state/v1";

export type Presence = "recorded" | "explicit_unknown" | "not_recorded";

export type FieldPresence = {
 field: string;
 presence: Presence;
};

export type EncounterState = {
 encounter_id: string;
 events: readonly ClinicalEvent[];
 contradictions: readonly Contradiction[];
 presence: readonly FieldPresence[];
};

const ANSWER_FIELDS = [
 "who",
 "presentation",
 "photoAttached",
 "awake",
 "breathing",
 "drinking",
 "duration",

```

```

 "mainProblem",
] as const;

 type AnswerField = (typeof ANSWER_FIELDS)[number];

 const UNIT_ALIASES: Record<string, string> = {
 c: "C",
 celsius: "C",
 "°c": "C",
 degc: "C",
 f: "F",
 fahrenheit: "F",
 "°f": "F",
 degf: "F",
 };

 export class StateError extends Error {
 constructor(message: string) {
 super(message);
 this.name = "StateError";
 Object.setPrototypeOf(this, new.target.prototype);
 }
 }

 /** Unit-label normalization only. Not a clinical threshold. */
 export function normalizeUnit(unit: string | null): string | null {
 if (unit === null) return null;
 const key = unit.trim().toLowerCase().replace(/\s+/g, "");
 if (!key) return null;
 return UNIT_ALIASES[key] ?? unit.trim();
 }

 /**
 * Measurement conversion for comparison. Not a fever cutoff and not a
 * disposition input.
 */
 export function toCelsius(value: number, unit: string | null): number | null {
 const normalized = normalizeUnit(unit);
 if (normalized === "C") return value;
 if (normalized === "F") return ((value - 32) * 5) / 9;
 return null;
 }

 function presenceOf(field: AnswerField, value: AssessmentAnswers[AnswerField]): Presence {
 if (field === "presentation") {
 return typeof value === "string" && value.trim().length > 0
 ? "recorded"
 : "not_recorded";
 }
 if (value === null) return "not_recorded";
 if (value === "unknown") return "explicit_unknown";
 return "recorded";
 }

 function claimText(event: ClinicalEvent): string {
 const unit = event.unit ? ` ${event.unit}` : "";
 return `${event.type}=${String(event.value)}${unit} @${event.timestamp}#${event.event_id}`;
 }

 function withNormalizedUnit(event: ClinicalEvent): ClinicalEvent {
 const unit = normalizeUnit(event.unit);
 if (unit === event.unit) return event;
 const note = event.unit ? `; unit_normalized_from=${event.unit}` : "";
 return {

```

```

 ...event,
 unit,
 provenance: `${event.provenance}${note}`,
 };
}

function isNumericSeries(prior: ClinicalEvent, next: ClinicalEvent): boolean {
 return (
 typeof prior.value === "number" &&
 typeof next.value === "number" &&
 prior.timestamp !== next.timestamp
);
}

function conflictsFor(
 existing: readonly ClinicalEvent[],
 next: ClinicalEvent,
): Contradiction[] {
 const conflicts: Contradiction[] = [];
 for (const prior of existing) {
 if (prior.type !== next.type) continue;
 if (prior.value === next.value && prior.unit === next.unit) continue;
 if (isNumericSeries(prior, next)) continue;
 conflicts.push(
 createContradiction({
 claim_a: claimText(prior),
 claim_b: claimText(next),
 sources: [prior.source, next.source],
 clinical_importance: "low",
 }),
);
 }
 return conflicts;
}

export function projectAnswers(input: {
 encounterId: string;
 answers: AssessmentAnswers;
 timestamp: string;
 source?: string;
}): EncounterState {
 if (!input.encounterId.trim()) {
 throw new StateError("encounter_id is required; state is encounter-scoped");
 }
 const source = input.source ?? "encounter_answers";
 const presence: FieldPresence[] = [];
 let events: readonly ClinicalEvent[] = [];
 for (const field of ANSWER_FIELDS) {
 const value = input.answers[field];
 const presenceKind = presenceOf(field, value);
 presence.push({ field, presence: presenceKind });
 if (presenceKind === "not_recorded") continue;
 const event = validateClinicalEvent({
 event_id: `${input.encounterId}.${field}.${input.timestamp}`,
 encounter_id: input.encounterId,
 type: field,
 value,
 unit: null,
 timestamp: input.timestamp,
 source,
 confidence: 1,
 provenance: "assessment_answers",
 });
 events = appendClinicalEvent(events, event);
 }
}

```

```

 }
 return {
 encounter_id: input.encounterId,
 events,
 contradictions: [],
 presence,
 };
 }

export function projectEncounter(encounter: Encounter): EncounterState {
 return projectAnswers({
 encounterId: encounter.id,
 answers: encounter.answers,
 timestamp: encounter.updatedAt,
 source: `encounter:${encounter.startedBy}`,
 });
}

export function appendObservation(
 state: EncounterState,
 event: ClinicalEvent,
): EncounterState {
 const normalized = withNormalizedUnit(validateClinicalEvent(event));
 if (normalized.encounter_id !== state.encounter_id) {
 throw new StateError(
 "encounter-scoped: refusing to append an event from another encounter",
);
 }
 const events = appendClinicalEvent(state.events, normalized);
 const contradictions = Object.freeze([
 ...state.contradictions,
 ...conflictsFor(state.events, normalized),
]);
 return {
 encounter_id: state.encounter_id,
 events,
 contradictions,
 presence: state.presence,
 };
}

function elapsedHours(start: string, end: string): number | null {
 const from = Date.parse(start);
 const to = Date.parse(end);
 if (!Number.isFinite(from) || !Number.isFinite(to) || to < from) return null;
 return (to - from) / 3_600_000;
}

function formatHours(hours: number): string {
 if (Math.abs(hours - Math.round(hours)) < 1e-9) return String(Math.round(hours));
 return hours.toFixed(1);
}

function comparable(event: ClinicalEvent): number | null {
 if (typeof event.value !== "number") return null;
 if (event.type === "temperature") return toCelsius(event.value, event.unit);
 return event.value;
}

/**
 * Structural delta only. Phrases describe change across readings. They do
 * not assign a clinical meaning and they do not affect disposition.
 */
export function structuralTrend(

```

```

events: readonly ClinicalEvent[],
type: string,
): DerivedFeature | null {
 const series = eventsOf<Type>(events, type).filter(
 (event) => typeof event.value === "number",
);
 if (series.length < 2) return null;
 if (type === "temperature") {
 if (series.some((event) => toCelsius(event.value as number, event.unit) === null)) {
 return null;
 }
 } else if (new Set(series.map((event) => normalizeUnit(event.unit))).size !== 1) {
 return null;
 }
 const values = series.map((event) => comparable(event));
 if (values.some((value) => value === null)) return null;
 const numbers = values as number[];
 const strictlyDecreasing = numbers.every(
 (value, index) => index === 0 || value < numbers[index - 1],
);
 const first = numbers[0];
 const last = numbers[numbers.length - 1];
 const hours = elapsedHours(
 series[0].timestamp,
 series[series.length - 1].timestamp,
);
 let text: string;
 if (strictlyDecreasing && numbers.length >= 3) {
 text =
 numbers.length === 3
 ? "three declining readings"
 : `${numbers.length} declining readings`;
 } else if (last > first) {
 text =
 hours === null || hours === 0
 ? "increased"
 : `increased over ${formatHours(hours)} hours`;
 } else if (last < first) {
 text =
 hours === null || hours === 0
 ? "decreased"
 : `decreased over ${formatHours(hours)} hours`;
 } else {
 text = "unchanged across readings";
 }
 return {
 feature_id: `trend:${type}:${series.map((event) => event.event_id).join("+)}\``,
 name: `${type}_structural_delta`,
 value: text,
 unit: null,
 inputs: series.map((event) => event.event_id),
 };
}

```

## lib/clinical/reconcile.ts

Floor wins. A lower or unapproved candidate is rewritten up. fail\_open is false.

SHA-256 67626c656def0b9a06db426f5b19a1e490fd0c4b9dcdb5242156dae30f9b23bc · 145 lines

```

import type { Decision } from "../types";
import { isDisposition, type Disposition } from "./contract";

/**
 * Demo-floor reconciler. The only approved path in this wave is decide().
 * A lower candidate is rewritten up to that floor. An unapproved higher
 * candidate is not applied. This rank is an engineering order so a
 * candidate cannot downgrade the floor. It is not a clinical severity score.
 */

export const RECONCILE_MODULE = "clinical-reconcile/v1";
export const UNAPPROVED_CLINICAL_RULES = "disabled" as const;

const RANK: Record<string, number> = {
 go_now: 40,
 escalate: 40,
 urgent: 40,
 get_care_today: 30,
 "see-clinic": 30,
 see_clinic: 30,
 need_one_more_answer: 20,
 abstain: 20,
 monitor_at_home: 10,
 "self-care": 10,
 self_care: 10,
};

export class ReconcileError extends Error {
 constructor(message: string) {
 super(message);
 this.name = "ReconcileError";
 Object.setPrototypeOf(this, new.target.prototype);
 }
}

export type CandidateInput = {
 disposition: string;
 model_id: string | null;
};

export type DisagreementReason =
 | "candidate_below_floor"
 | "candidate_not_approved"
 | "candidate_unrecognized";

export type DisagreementRecord = {
 floor: Disposition;
 candidate: string;
 candidate_rank: number | null;
 reconciled: Disposition;
 rewritten_to_floor: true;
 reason: DisagreementReason;
 model_id: string | null;
};

export type ReconcileResult = {
 disposition: Disposition;
 disagreement: DisagreementRecord | null;
 fail_open: false;
};

function floorDisposition(floor: Decision | Disposition): Disposition {
 const kind = typeof floor === "string" ? floor : floor?.kind;

```

```

 if (!isDisposition(kind)) {
 throw new ReconcileError(
 "floor is not an approved disposition; refusing to invent a lower one",
);
 }
 return kind;
 }

function disagreement(
 floor: Disposition,
 candidate: CandidateInput,
 reason: DisagreementReason,
 candidateRank: number | null,
): DisagreementRecord {
 return {
 floor,
 candidate: candidate.disposition,
 candidate_rank: candidateRank,
 reconciled: floor,
 rewritten_to_floor: true,
 reason,
 model_id: candidate.model_id,
 };
}

export function reconcile(input: {
 floor: Decision | Disposition;
 candidate?: CandidateInput | null;
}): ReconcileResult {
 const floor = floorDisposition(input.floor);
 if (!input.candidate) {
 return { disposition: floor, disagreement: null, fail_open: false };
 }
 const candidate = input.candidate.disposition.trim();
 const normalized: CandidateInput = {
 disposition: candidate,
 model_id: input.candidate.model_id,
 };
 if (candidate === floor) {
 return { disposition: floor, disagreement: null, fail_open: false };
 }
 const rank = RANK[candidate] ?? null;
 if (rank === null) {
 return {
 disposition: floor,
 disagreement: disagreement(floor, normalized, "candidate_unrecognized", null),
 fail_open: false,
 };
 }
 const floorRank = RANK[floor];
 if (rank < floorRank) {
 return {
 disposition: floor,
 disagreement: disagreement(floor, normalized, "candidate_below_floor", rank),
 fail_open: false,
 };
 }
 return {
 disposition: floor,
 disagreement: disagreement(floor, normalized, "candidate_not_approved", rank),
 fail_open: false,
 };
}

```

```

/**
 * Telemetry or persistence failure must not replace the safer answer.
 * The returned disposition is computed before the sink runs.
 */
export function reconcileWithTelemetry(
 input: { floor: Decision | Disposition; candidate?: CandidateInput | null },
 sink?: (record: DisagreementRecord) => void,
): ReconcileResult {
 const result = reconcile(input);
 if (result.disagreement && sink) {
 try {
 sink(result.disagreement);
 } catch {
 return result;
 }
 }
 return result;
}

```

## lib/clinical/pipeline.ts

decide() then reconciler then contract. The UI does not call this yet.

SHA-256 25a2b5e1ad7cfe82991f4841d49216e0bfb5de652b3e7643b1675cdf104f26a7 · 201 lines

```

import { EMPTY_ANSWERS } from "../assessment";
import { t, type CopyKey, type Locale } from "../copy";
import type { AssessmentAnswers, DecisionKind, Encounter } from "../types";
import {
 SECTION_KEYS,
 actionClassFor,
 demoProvenance,
 emptyRetrievedEvidence,
 headlineKeyFor,
 validateClinicalAssessment,
 type AssessmentSection,
 type ClinicalAssessment,
 type ClinicalEvent,
 type DerivedFeature,
 type ExistingHeadlineKey,
} from "./contract";
import {
 reconcileWithTelemetry,
 type CandidateInput,
 type DisagreementRecord,
} from "./reconcile";
import {
 appendObservation,
 projectEncounter,
 structuralTrend,
 type EncounterState,
} from "./state";
import { decide } from "../assessment";

/**
 * Demo-path wiring: decide() → reconciler → ten-section contract.
 * No model call. model_id and knowledge_version stay null.
 * Internal sections stay structured. This is not a prose generator.
 */

```

```

export const PIPELINE_RULES_VERSION = "demo-floor/unvalidated";

const SECTION_LABELS: Record<AssessmentSection, CopyKey> = {
 observed_facts: "contractObservedFacts",
 retrieved_evidence: "contractRetrievedEvidence",
 derived_features: "contractDerivedFeatures",
 hypotheses: "contractHypotheses",
 uncertainties: "contractUncertainties",
 contradictions: "contractContradictions",
 required_information: "contractRequiredInformation",
 red_flags: "contractRedFlags",
 next_action: "contractNextAction",
 provenance: "contractProvenance",
};

export type RenderedSection = {
 key: AssessmentSection;
 labelKey: CopyKey;
 label: string;
 body: unknown;
};

export type RenderedAssessment = {
 locale: Locale;
 headlineKey: ExistingHeadlineKey | null;
 headline: string | null;
 sections: RenderedSection[];
 collapsed: false;
};

export type PipelineResult = {
 state: EncounterState;
 assessment: ClinicalAssessment;
 disagreement: DisagreementRecord | null;
 rendered: RenderedAssessment;
};

function requireTimestamp(value: string): string {
 if (!/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?(?:Z|[\+-]\d{2}:\d{2})$/i.test(value)) {
 throw new Error("generatedAt must be an explicit ISO timestamp");
 }
 return value;
}

function trends(events: readonly ClinicalEvent[]): DerivedFeature[] {
 const types = [...new Set(events.map((event) => event.type))];
 const features: DerivedFeature[] = [];
 for (const type of types) {
 const feature = structuralTrend(events, type);
 if (feature) features.push(feature);
 }
 return features;
}

export function renderAssessment(
 assessment: ClinicalAssessment,
 locale: Locale,
): RenderedAssessment {
 const headlineKey = headlineKeyFor(assessment.next_action.disposition);
 return {
 locale,
 headlineKey,
 headline: headlineKey ? t(headlineKey, locale) : null,
 collapsed: false,
 };
}

```

```

 sections: SECTION_KEYS.map((key) => ({
 key,
 labelKey: SECTION_LABELS[key],
 label: t(SECTION_LABELS[key], locale),
 body: assessment[key],
 })),
 });
}

export function runClinicalPipeline(input: {
 encounter: Encounter;
 generatedAt: string;
 locale?: Locale;
 candidate?: CandidateInput | null;
 extraEvents?: readonly ClinicalEvent[];
}): PipelineResult {
 const generatedAt = requireTimestamp(input.generatedAt);
 let state = projectEncounter(input.encounter);
 for (const event of input.extraEvents ?? []) {
 state = appendObservation(state, event);
 }
 const floor = decide(input.encounter.answers);
 const reconciled = reconcileWithTelemetry({
 floor,
 candidate: input.candidate ?? null,
 });
 const applied = reconciled.disposition;
 const observedFacts = state.events.map((event) => ({
 fact_id: event.event_id,
 value: `${event.type}=${String(event.value)}${event.unit ? ` ${event.unit}` : ""}`,
 source: event.source,
 timestamp: event.timestamp,
 confidence: event.confidence,
 provenance: event.provenance,
 }));
 const uncertainties = state.presence
 .filter((item) => item.presence !== "recorded")
 .map((item) => ({
 uncertainty_id: `presence:${item.field}`,
 statement: `${item.field}:${item.presence}`,
 }));
 const required = state.presence
 .filter((item) => item.presence !== "recorded")
 .map((item) => ({
 question_id: item.field,
 statement: `${item.field}:${item.presence}`,
 }));
 const redFlags = floor.dangerSignKeys.map((code, index) => ({
 flag_id: `floor:${index}:${code}`,
 code,
 source: "decide",
 }));
 const assessment = validateClinicalAssessment({
 observed_facts: observedFacts,
 retrieved_evidence: emptyRetrievedEvidence("not_run"),
 derived_features: trends(state.events),
 hypotheses: [],
 uncertainties,
 contradictions: state.contradictions,
 required_information: required,
 red_flags: redFlags,
 next_action: {
 disposition: applied,
 action: actionClassFor(applied),
 },
 });
}

```

```

 ideal_action: applied,
 locally_executable_action: applied,
 resource_gap: null,
 },
 provenance: demoProvenance(PIPELINE_RULES_VERSION, generatedAt),
});
return {
 state,
 assessment,
 disagreement: reconciled.disagreement,
 rendered: renderAssessment(assessment, input.locale ?? "en"),
};
}

/** Eval adapter. Demo path only: no candidate, so the floor is decide(). */
export function pipelineDecider(
 answers: AssessmentAnswers,
): { kind: DecisionKind } {
 const encounter: Encounter = {
 id: "eval-encounter",
 answers: { ...EMPTY_ANSWERS, ...answers },
 asked: [],
 currentQuestion: null,
 decision: null,
 startedBy: "household",
 createdAt: "2026-10-07T00:00:00Z",
 updatedAt: "2026-10-07T00:00:00Z",
 };
 const result = runClinicalPipeline({
 encounter,
 generatedAt: "2026-10-07T00:00:00Z",
 });
 const kind = result.assessment.next_action.disposition;
 if (kind === "escalate" || kind === "abstain") {
 throw new Error("demo path must stay on the existing decision kinds");
 }
 return { kind };
}

```

## lib/store.tsx

The reducer. The answer path still calls decide() directly and persists that Decision.

SHA-256 cc7c7341b7c6d0f5b38947b0c462deee351bf28f6d3c8d1e2608cbab64670cf7 · 508 lines

```

"use client";

import {
 createContext,
 useContext,
 useEffect,
 useMemo,
 useReducer,
 useSyncExternalStore,
 type ReactNode,
} from "react";
import {
 applyAnswer,
 decide,
 EMPTY_ANSWERS,

```

```

 nextQuestion,
 } from "./assessment";
import { applyReferralAction, createReferral } from "./referral";
import { DEFAULT_LOCALE, type Locale } from "./copy";
import { createId, nowIso } from "./ids";
import type {
 CareState,
 Encounter,
 GatedRole,
 HouseholdView,
 NamedFailure,
 OutcomeCode,
 QuestionId,
 Referral,
 Role,
} from "./types";

const STORAGE_KEY = "tunza.v2.care";

export const INJECTABLE_FAILURES: NamedFailure[] = [
 "offline",
 "no_facility_response",
 "redirected",
 "stale_information",
 "incomplete_assessment",
 "weak_connection",
];

type Action =
 | { type: "hydrate"; state: CareState }
 | { type: "setRole"; role: Role }
 | { type: "setLocale"; locale: Locale }
 | { type: "setView"; view: HouseholdView }
 | { type: "setGateRole"; role: GatedRole | null }
 | { type: "grantRole"; role: GatedRole }
 | { type: "setOnline"; online: boolean }
 | { type: "toggleFailure"; failure: NamedFailure }
 | { type: "startEncounter"; by: Role }
 | { type: "answer"; question: QuestionId; value: string }
 | { type: "setPresentation"; text: string }
 | { type: "setPhoto"; attached: boolean }
 | { type: "prepareReferral" }
 | { type: "referralAction"; action: ReferralActionName; outcome?: OutcomeCode }
 | { type: "goBack" }
 | { type: "continueForOneMore" }
 | { type: "reset" };

type ReferralActionName =
 | "send"
 | "receive"
 | "accept"
 | "travel"
 | "arrive"
 | "start_care"
 | "complete"
 | "return_outcome"
 | "redirect"
 | "ask_more";

function newEncounter(by: Role): Encounter {
 const at = nowIso();
 return {
 id: createId("enc"),
 answers: { ...EMPTY_ANSWERS },
 };
}

```

```

 asked: [],
 currentQuestion: "who",
 decision: null,
 startedBy: by,
 createdAt: at,
 updatedAt: at,
 };
}

export const initialCareState: CareState = {
 role: "household",
 locale: DEFAULT_LOCALE,
 view: "home",
 grants: { chp: false, facility: false },
 gateRole: null,
 injectedFailures: [],
 encounter: newEncounter("household"),
 referral: null,
 online: true,
};

function isOffline(state: CareState): boolean {
 return !state.online || state.injectedFailures.includes("offline");
}

function reducer(state: CareState, action: Action): CareState {
 switch (action.type) {
 case "hydrate":
 return { ...action.state, online: state.online };
 case "setRole":
 // The gated surfaces open only for granted roles; anyone else is sent
 // to the sign-in gate instead. Enforced here so no caller can skip it.
 if (action.role !== "household" && !state.grants[action.role]) {
 return { ...state, role: "household", view: "gate", gateRole: action.role };
 }
 return { ...state, role: action.role };
 case "setLocale":
 return { ...state, locale: action.locale };
 case "setView":
 return { ...state, view: action.view };
 case "setGateRole":
 return { ...state, gateRole: action.role };
 case "grantRole":
 return {
 ...state,
 grants: { ...state.grants, [action.role]: true },
 role: action.role,
 view: "home",
 gateRole: null,
 };
 case "setOnline":
 return { ...state, online: action.online };
 case "toggleFailure": {
 const has = state.injectedFailures.includes(action.failure);
 const injectedFailures = has
 ? state.injectedFailures.filter((item) => item !== action.failure)
 : [...state.injectedFailures, action.failure];
 let referral = state.referral;
 if (action.failure === "no_facility_response" && !has && referral?.stage === "sent") {
 referral = { ...referral, failure: "no_facility_response" };
 }
 if (action.failure === "no_facility_response" && has && referral?.failure ===
 "no_facility_response") {
 referral = { ...referral, failure: null };
 }
 }
 }
}

```

```

 }
 if (action.failure === "redirected" && !has && referral) {
 referral = applyReferralAction(referral, "redirect", state.role, {
 offline: isOffline(state),
 noFacilityResponse: false,
 weakConnection: false,
 });
 }
 return { ...state, injectedFailures, referral };
 }
 case "startEncounter":
 return {
 ...state,
 encounter: newEncounter(action.by),
 referral: null,
 };
 case "setPresentation": {
 if (!state.encounter) {
 return state;
 }
 return {
 ...state,
 encounter: {
 ...state.encounter,
 answers: { ...state.encounter.answers, presentation: action.text },
 updatedAt: nowIso(),
 },
 };
 }
 case "setPhoto": {
 if (!state.encounter) {
 return state;
 }
 return {
 ...state,
 encounter: {
 ...state.encounter,
 answers: { ...state.encounter.answers, photoAttached: action.attached },
 updatedAt: nowIso(),
 },
 };
 }
 case "answer": {
 if (!state.encounter || state.encounter.currentQuestion !== action.question) {
 return state;
 }
 const answers = applyAnswer(
 state.encounter.answers,
 action.question,
 action.value,
);
 const asked = state.encounter.asked.includes(action.question)
 ? state.encounter.asked
 : [...state.encounter.asked, action.question];
 const upcoming = nextQuestion(answers, action.question);
 const decision = upcoming ? null : decide(answers);
 return {
 ...state,
 encounter: {
 ...state.encounter,
 answers,
 asked,
 currentQuestion: upcoming,
 decision,
 },
 };
 }
}

```

```

 updatedAt: nowIso(),
 },
];
 };
}
case "prepareReferral": {
 if (!state.encounter?.decision) {
 return state;
 }
 const kind = state.encounter.decision.kind;
 if (kind !== "go_now" && kind !== "get_care_today") {
 return state;
 }
 if (state.referral) {
 return state;
 }
 const referral = createReferral({
 encounterId: state.encounter.id,
 urgent: kind === "go_now",
 by: state.role,
 });
 return { ...state, referral };
}
case "referralAction": {
 if (!state.referral) {
 return state;
 }
 const referral = applyReferralAction(state.referral, action.action, state.role, {
 offline: isOffline(state),
 noFacilityResponse: state.injectedFailures.includes("no_facility_response"),
 weakConnection: state.injectedFailures.includes("weak_connection"),
 outcome: action.outcome,
 });
 return { ...state, referral };
}
case "continueForOneMore": {
 if (!state.encounter?.decision || state.referral) {
 return state;
 }
 if (state.encounter.decision.kind !== "need_one_more_answer") {
 return state;
 }
 if (state.encounter.asked.includes("main_problem")) {
 return state;
 }
 return {
 ...state,
 encounter: {
 ...state.encounter,
 currentQuestion: "main_problem",
 decision: null,
 updatedAt: nowIso(),
 },
 };
}
case "goBack": {
 if (!state.encounter || state.referral) {
 return state;
 }
 if (state.encounter.decision && state.encounter.asked.length > 0) {
 const last = state.encounter.asked[state.encounter.asked.length - 1];
 return {
 ...state,
 encounter: {
 ...state.encounter,

```

```

 currentQuestion: last,
 decision: null,
 updatedAt: nowIso(),
 },
 },
};
}
if (state.encounter.asks.length === 0) {
 return state;
}
const asks = state.encounter.asks.slice(0, -1);
const currentQuestion = state.encounter.asks[state.encounter.asks.length - 1];
return {
 ...state,
 encounter: {
 ...state.encounter,
 asks,
 currentQuestion,
 decision: null,
 updatedAt: nowIso(),
 },
 },
};
}
case "reset":
 // Start over is a fresh demo: back to the household front door, grants
 // cleared, everything else at its initial state.
 return {
 ...initialCareState,
 locale: state.locale,
 online: state.online,
 encounter: newEncounter("household"),
 };
default:
 return state;
}
}
}

```

```

type CareContextValue = {
 state: CareState;
 hydrated: boolean;
 offline: boolean;
 dispatch: {
 setRole: (role: Role) => void;
 setLocale: (locale: Locale) => void;
 setView: (view: HouseholdView) => void;
 setGateRole: (role: GatedRole | null) => void;
 grantRole: (role: GatedRole) => void;
 toggleFailure: (failure: NamedFailure) => void;
 startEncounter: () => void;
 answer: (question: QuestionId, value: string) => void;
 setPresentation: (text: string) => void;
 setPhoto: (attached: boolean) => void;
 prepareReferral: () => void;
 sendReferral: () => void;
 acceptReferral: () => void;
 redirectReferral: () => void;
 askMore: () => void;
 markTraveling: () => void;
 markArrived: () => void;
 startCare: () => void;
 completeVisit: () => void;
 returnOutcome: (outcome: OutcomeCode) => void;
 goBack: () => void;
 continueForOneMore: () => void;
 reset: () => void;
 };
};

```

```

 };
};

function emptySubscribe() {
 return () => {};
}

function useIsClient() {
 return useSyncExternalStore(emptySubscribe, () => true, () => false);
}

function readStoredState(): CareState {
 try {
 const raw = localStorage.getItem(STORAGE_KEY);
 if (raw) {
 const parsed = JSON.parse(raw) as CareState;
 if (parsed?.role) {
 const grants = {
 chp: parsed.grants?.chp === true,
 facility: parsed.grants?.facility === true,
 };
 const roleAllowed = parsed.role === "household" || grants[parsed.role];
 return {
 ...parsed,
 grants,
 gateRole: null,
 role: roleAllowed ? parsed.role : "household",
 locale: parsed.locale ?? DEFAULT_LOCALE,
 view: roleAllowed ? (parsed.view ?? "home") : "home",
 online: navigator.onLine,
 };
 }
 }
 } catch {
 // Demo persistence must never break the care path.
 }
 return { ...initialCareState, online: navigator.onLine };
}

function makeDispatch(
 dispatch: (action: Action) => void,
 role: Role,
): CareContextValue["dispatch"] {
 return {
 setRole: (nextRole) => dispatch({ type: "setRole", role: nextRole }),
 setLocale: (locale) => dispatch({ type: "setLocale", locale }),
 setView: (view) => dispatch({ type: "setView", view }),
 setGateRole: (role) => dispatch({ type: "setGateRole", role }),
 grantRole: (role) => dispatch({ type: "grantRole", role }),
 toggleFailure: (failure) => dispatch({ type: "toggleFailure", failure }),
 startEncounter: () => dispatch({ type: "startEncounter", by: role }),
 answer: (question, value) => dispatch({ type: "answer", question, value }),
 setPresentation: (text) => dispatch({ type: "setPresentation", text }),
 setPhoto: (attached) => dispatch({ type: "setPhoto", attached }),
 prepareReferral: () => dispatch({ type: "prepareReferral" }),
 sendReferral: () => dispatch({ type: "referralAction", action: "send" }),
 acceptReferral: () => dispatch({ type: "referralAction", action: "accept" }),
 redirectReferral: () => dispatch({ type: "referralAction", action: "redirect" }),
 askMore: () => dispatch({ type: "referralAction", action: "ask_more" }),
 markTraveling: () => dispatch({ type: "referralAction", action: "travel" }),
 markArrived: () => dispatch({ type: "referralAction", action: "arrive" }),
 startCare: () => dispatch({ type: "referralAction", action: "start_care" }),
 completeVisit: () => dispatch({ type: "referralAction", action: "complete" }),
 returnOutcome: (outcome) =>
 }
}

```

```

 dispatch({ type: "referralAction", action: "return_outcome", outcome }),
 goBack: () => dispatch({ type: "goBack" }),
 continueForOneMore: () => dispatch({ type: "continueForOneMore" }),
 reset: () => dispatch({ type: "reset" }),
 };
}

const noopDispatch: CareContextValue["dispatch"] = makeDispatch(() => {
 // Server snapshot has no live session yet.
}, "household");

const CareContext = createContext<CareContextValue | null>(null);

export function CareProvider({ children }: { children: ReactNode }) {
 const isClient = useIsClient();
 if (!isClient) {
 return (
 <CareContext.Provider
 value={{
 state: initialCareState,
 hydrated: false,
 offline: false,
 dispatch: noopDispatch,
 }}
 >
 {children}
 </CareContext.Provider>
);
 }
 return <CareProviderClient>{children}</CareProviderClient>;
}

function CareProviderClient({ children }: { children: ReactNode }) {
 const [state, dispatch] = useReducer(reducer, null, readStoredState);

 useEffect(() => {
 try {
 localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
 } catch {
 // Ignore quota errors; the in-memory path still works.
 }
 }, [state]);

 useEffect(() => {
 function onOnline() {
 dispatch({ type: "setOnline", online: true });
 }
 function onOffline() {
 dispatch({ type: "setOnline", online: false });
 }
 window.addEventListener("online", onOnline);
 window.addEventListener("offline", onOffline);
 return () => {
 window.removeEventListener("online", onOnline);
 window.removeEventListener("offline", onOffline);
 };
 }, []);

 const value = useMemo<CareContextValue>(
 () => ({
 state,
 hydrated: true,
 offline: isOffline(state),
 dispatch: makeDispatch(dispatch, state.role),
 })
);

```

```

 }),
 [state],
);

 return <CareContext.Provider value={value}>{children}</CareContext.Provider>;
}

export function useCare(): CareContextValue {
 const ctx = useContext(CareContext);
 if (!ctx) {
 throw new Error("useCare must be used inside CareProvider");
 }
 return ctx;
}

export function activeFailures(
 state: CareState,
 referral: Referral | null,
): NamedFailure[] {
 const named = new Set<NamedFailure>();
 if (!state.online || state.injectedFailures.includes("offline")) {
 named.add("offline");
 }
 if (state.injectedFailures.includes("weak_connection")) {
 named.add("weak_connection");
 }
 if (state.injectedFailures.includes("stale_information")) {
 named.add("stale_information");
 }
 if (referral?.failure === "no_facility_response") {
 named.add("no_facility_response");
 }
 if (referral?.failure === "redirected") {
 named.add("redirected");
 }
 if (
 state.encounter?.decision?.kind === "need_one_more_answer" ||
 state.injectedFailures.includes("incomplete_assessment")
) {
 named.add("incomplete_assessment");
 }
 return [...named];
}

export { reducer as careReducer };
export type { Action as CareAction };

```

## lib/referral.ts

The only place referral state becomes words.

SHA-256 43bf831d61cd67a06f0271e61dc5f8046646ec440c0fccd961c00adb5a72672a · 522 lines

```

import { fill, t, type CopyKey, type Locale } from "../copy";
import { facilityById, pickFacility, redirectFacility } from "../facilities";
import { createId, nowIso } from "../ids";
import type {
 DecisionKind,
 OutcomeCode,
 Referral,

```

```

 ReferralStage,
 Role,
 } from "./types";

export const REFERRAL_STAGES: ReferralStage[] = [
 "created",
 "sent",
 "received",
 "accepted",
 "patient_moving",
 "arrived",
 "seen",
 "completed",
 "outcome_returned",
];

export type ReferralAction =
 | "send"
 | "receive"
 | "accept"
 | "travel"
 | "arrive"
 | "start_care"
 | "complete"
 | "return_outcome"
 | "redirect"
 | "ask_more";

const STAGE_AFTER: Record<ReferralAction, ReferralStage | null> = {
 send: "sent",
 receive: "received",
 accept: "accepted",
 travel: "patient_moving",
 arrive: "arrived",
 start_care: "seen",
 complete: "completed",
 return_outcome: "outcome_returned",
 redirect: "sent",
 ask_more: "received",
};

export type ReferralView = {
 stageLabel: string;
 headline: string;
 status: string;
 action: { id: ReferralAction; label: string } | null;
 arrivalMinutes: number | null;
};

export function createReferral(input: {
 encounterId: string;
 urgent: boolean;
 by: Role;
}): Referral {
 const facility = pickFacility(input.urgent);
 const at = nowIso();
 return {
 id: createId("ref"),
 encounterId: input.encounterId,
 stage: "created",
 facilityId: facility.id,
 expectedArrivalMinutes: facility.travelMinutes,
 queued: false,
 failure: null,
 };
}

```

```

 askMore: null,
 outcome: null,
 history: [{ stage: "created", at, by: input.by }],
 createdAt: at,
 updatedAt: at,
 };
}

export function canPerform(
 referral: Referral,
 action: ReferralAction,
): boolean {
 const { stage } = referral;

 switch (action) {
 case "send":
 return stage === "created";
 case "receive":
 return stage === "sent";
 case "accept":
 return stage === "sent" || stage === "received";
 case "travel":
 return stage === "accepted";
 case "arrive":
 return stage === "patient_moving";
 case "start_care":
 return stage === "arrived";
 case "complete":
 return stage === "seen";
 case "return_outcome":
 return stage === "completed";
 case "redirect":
 return stage === "sent" || stage === "received" || stage === "accepted";
 case "ask_more":
 return stage === "sent" || stage === "received";
 default:
 return false;
 }
}

export function applyReferralAction(
 referral: Referral,
 action: ReferralAction,
 by: Role,
 options: {
 offline: boolean;
 noFacilityResponse: boolean;
 weakConnection: boolean;
 outcome?: OutcomeCode;
 },
): Referral {
 const at = nowIso();

 if (action === "send" && options.offline) {
 return {
 ...referral,
 queued: true,
 failure: "offline",
 updatedAt: at,
 };
 }

 if (action === "send" && options.noFacilityResponse) {
 return {

```

```

 ...referral,
 stage: "sent",
 queued: false,
 failure: "no_facility_response",
 history: [...referral.history, { stage: "sent", at, by }],
 updatedAt: at,
 };
}

if (action === "send") {
 const failure = options.weakConnection ? "weak_connection" : null;
 return {
 ...referral,
 stage: "sent",
 queued: false,
 failure,
 history: [...referral.history, { stage: "sent", at, by }],
 updatedAt: at,
 };
}

if (action === "redirect") {
 const nextFacility = redirectFacility(referral.facilityId);
 return {
 ...referral,
 stage: "sent",
 facilityId: nextFacility.id,
 expectedArrivalMinutes: nextFacility.travelMinutes,
 failure: "redirected",
 askMore: null,
 history: [
 ...referral.history,
 { stage: "sent", at, by, note: nextFacility.name },
],
 updatedAt: at,
 };
}

if (action === "ask_more") {
 return {
 ...referral,
 stage: "received",
 askMore: "can_walk",
 history: [...referral.history, { stage: "received", at, by }],
 updatedAt: at,
 };
}

if (action === "accept") {
 const receivedHistory =
 referral.stage === "sent"
 ? [...referral.history, { stage: "received" as const, at, by }]
 : referral.history;
 return {
 ...referral,
 stage: "accepted",
 failure: referral.failure === "no_facility_response" ? null : referral.failure,
 queued: false,
 history: [...receivedHistory, { stage: "accepted", at, by }],
 updatedAt: at,
 };
}

if (action === "return_outcome") {

```

```

 return {
 ...referral,
 stage: "outcome_returned",
 outcome: options.outcome ?? "treated",
 history: [...referral.history, { stage: "outcome_returned", at, by }],
 updatedAt: at,
 };
 }

 const stage = STAGE_AFTER[action];
 if (!stage) {
 return referral;
 }

 return {
 ...referral,
 stage,
 history: [...referral.history, { stage, at, by }],
 updatedAt: at,
 };
}

const STAGE_LABEL_KEY: Record<ReferralStage, CopyKey> = {
 created: "stageCreated",
 sent: "stageSent",
 received: "stageReceived",
 accepted: "stageAccepted",
 patient_moving: "stagePatientMoving",
 arrived: "stageArrived",
 seen: "stageSeen",
 completed: "stageCompleted",
 outcome_returned: "stageOutcomeReturned",
};

export function stageLabel(stage: ReferralStage, locale: Locale): string {
 return t(STAGE_LABEL_KEY[stage], locale);
}

export function outcomeLabel(outcome: OutcomeCode, locale: Locale): string {
 switch (outcome) {
 case "treated":
 return t("outcomeTreated", locale);
 case "referred_onward":
 return t("outcomeHigher", locale);
 case "did_not_arrive":
 return t("outcomeNoShow", locale);
 case "unknown":
 return t("outcomeUnknown", locale);
 }
}

/**
 * The one place where referral state becomes words. Every surface renders the
 * same event through this translation: (stage x role x language) -> what this
 * person needs to know, and the one thing they can do about it.
 */
export function describeReferral(
 role: Role,
 referral: Referral,
 decisionKind: DecisionKind | null,
 locale: Locale,
): ReferralView {
 const facility = facilityById(referral.facilityId);
 const urgent = decisionKind === "go_now";

```

```

const minutes = referral.expectedArrivalMinutes;
const label = stageLabel(referral.stage, locale);
const f = facility.name;

const view = (
 headlineKey: CopyKey,
 statusKey: CopyKey,
 action: { id: ReferralAction; label: CopyKey } | null,
 arrivalMinutes: number | null,
): ReferralView => ({
 stageLabel: label,
 headline: fill(t(headlineKey, locale), { f }),
 status: fill(t(statusKey, locale), { f }),
 action: action ? { id: action.id, label: t(action.label, locale) } : null,
 arrivalMinutes,
});

if (referral.queued || referral.failure === "offline") {
 if (role === "household") {
 return view(
 "refQueuedHouseholdHeadline",
 "refQueuedHouseholdStatus",
 { id: "send", label: "actionSendFacility" },
 null,
);
 }
 if (role === "chp") {
 return view(
 "refQueuedChpHeadline",
 "refQueuedChpStatus",
 { id: "send", label: "actionSendReferral" },
 null,
);
 }
}

if (referral.failure === "no_facility_response" && referral.stage === "sent") {
 if (role === "facility") {
 return view(
 "refSentFacilityHeadline",
 "refNoResponseFacilityStatus",
 { id: "accept", label: "actionAccept" },
 null,
);
 }
 if (role === "household") {
 return view("refNoResponseHeadline", "refNoResponseHouseholdStatus", null, null);
 }
 return view("refNoResponseHeadline", "refNoResponseOtherStatus", null, null);
}

switch (referral.stage) {
 case "created":
 if (role === "household") {
 return view(
 "refCreatedHouseholdHeadline",
 "refCreatedHouseholdStatus",
 { id: "send", label: urgent ? "actionSendNow" : "actionSendFacility" },
 null,
);
 }
 if (role === "chp") {
 return view(
 "refCreatedChpHeadline",

```

```

 "refCreatedChpStatus",
 { id: "send", label: "actionSendReferral" },
 null,
);
}
return view("refCreatedFacilityHeadline", "refCreatedFacilityStatus", null, null);
case "sent":
 if (role === "household") {
 return view("refSentHouseholdHeadline", "refSentHouseholdStatus", null, null);
 }
 if (role === "chp") {
 return view("refSentChpHeadline", "refSentChpStatus", null, null);
 }
 return view(
 "refSentFacilityHeadline",
 "refSentFacilityStatus",
 { id: "accept", label: "actionAccept" },
 null,
);
case "received":
 if (role === "household") {
 return view(
 "refReceivedHouseholdHeadline",
 "refReceivedHouseholdStatus",
 null,
 null,
);
 }
 if (role === "chp") {
 return view("refReceivedChpHeadline", "refReceivedChpStatus", null, null);
 }
 return view(
 "refReceivedFacilityHeadline",
 "refReceivedFacilityStatus",
 { id: "accept", label: "actionAccept" },
 null,
);
case "accepted":
 if (role === "household") {
 return view(
 "refAcceptedHouseholdHeadline",
 "refAcceptedHouseholdStatus",
 { id: "travel", label: "actionWeveLeft" },
 minutes,
);
 }
 if (role === "chp") {
 return view(
 "refAcceptedChpHeadline",
 "refAcceptedChpStatus",
 { id: "travel", label: "actionTheyLeft" },
 minutes,
);
 }
 return view(
 urgent ? "refAcceptedFacilityUrgentHeadline" : "refAcceptedFacilityHeadline",
 "refAcceptedFacilityStatus",
 null,
 minutes,
);
case "patient_moving":
 if (role === "household") {
 return view(
 "refMovingHouseholdHeadline",

```

```

 "refMovingHouseholdStatus",
 { id: "arrive", label: "actionWeveArrived" },
 minutes,
);
}
if (role === "chp") {
 return view("refMovingChpHeadline", "refMovingChpStatus", null, minutes);
}
return view(
 "refMovingFacilityHeadline",
 "refMovingFacilityStatus",
 { id: "arrive", label: "actionTheyreHere" },
 minutes,
);
case "arrived":
 if (role === "household") {
 return view(
 "refArrivedHouseholdHeadline",
 "refArrivedHouseholdStatus",
 null,
 null,
);
 }
 if (role === "chp") {
 return view("refArrivedChpHeadline", "refArrivedChpStatus", null, null);
 }
 return view(
 "refArrivedFacilityHeadline",
 "refArrivedFacilityStatus",
 { id: "start_care", label: "actionStartCare" },
 null,
);
case "seen":
 if (role === "household") {
 return view("refSeenHouseholdHeadline", "refSeenHouseholdStatus", null, null);
 }
 if (role === "chp") {
 return view("refSeenChpHeadline", "refSeenChpStatus", null, null);
 }
 return view(
 "refSeenFacilityHeadline",
 "refSeenFacilityStatus",
 { id: "complete", label: "actionCompleteVisit" },
 null,
);
case "completed":
 if (role === "household") {
 return view(
 "refCompletedHouseholdHeadline",
 "refCompletedHouseholdStatus",
 null,
 null,
);
 }
 if (role === "chp") {
 return view("refCompletedChpHeadline", "refCompletedChpStatus", null, null);
 }
 return view(
 "refCompletedFacilityHeadline",
 "refCompletedFacilityStatus",
 { id: "return_outcome", label: "actionReturnOutcome" },
 null,
);
case "outcome_returned": {

```

```

 const outcomeText = referral.outcome
 ? outcomeLabel(referral.outcome, locale)
 : null;
 if (role === "household") {
 const base = view(
 "refOutcomeHouseholdHeadline",
 "refOutcomeHouseholdStatus",
 null,
 null,
);
 return { ...base, status: outcomeText ?? base.status };
 }
 if (role === "chp") {
 const base = view("refOutcomeChpHeadline", "refOutcomeChpStatus", null, null);
 return { ...base, status: outcomeText ?? base.status };
 }
 const base = view(
 "refOutcomeFacilityHeadline",
 "refOutcomeFacilityStatus",
 null,
 null,
);
 return { ...base, status: outcomeText ?? base.status };
 }
}

export function followUpItems(
 role: Role,
 referral: Referral | null,
 decisionKind: DecisionKind | null,
 locale: Locale,
): { id: string; label: string; detail?: string }[] {
 if (role !== "chp") {
 return [];
 }
 if (decisionKind === "monitor_at_home" && !referral) {
 return [
 {
 id: "watch-home",
 label: t("fuWatchHome", locale),
 detail: t("fuWatchHomeDetail", locale),
 },
];
 }
 if (referral?.stage === "accepted") {
 return [
 {
 id: "confirm-travel",
 label: t("fuConfirmTravel", locale),
 detail: t("fuConfirmTravelDetail", locale),
 },
];
 }
 if (referral?.stage === "outcome_returned") {
 return [
 {
 id: "post-outcome",
 label: t("fuPostOutcome", locale),
 detail: referral.outcome
 ? outcomeLabel(referral.outcome, locale)
 : t("fuPostOutcomeDetail", locale),
 },
];
 }
}

```

```
}
 return [];
}
```

## components/DecisionResult.tsx

The verdict component. One headline, one action.

SHA-256 50e107b3edae24262ed1ad57cc829f1e6171cf8a7977ebd2072eea105df0d91d · 73 lines

```
/** V1 care-path component. Reused across household, CHP, and facility. */

import { t, type Locale } from "@lib/copy";
import type { DecisionKind } from "@lib/types";
import type { ReactNode } from "react";

type Props = {
 locale: Locale;
 kind: DecisionKind;
 headline: string;
 status: string;
 action?: { label: string; onClick: () => void };
 why?: string[];
 children?: ReactNode;
};

const HEADLINE_COLOR: Record<DecisionKind, string> = {
 go_now: "text-urgent",
 get_care_today: "text-today",
 monitor_at_home: "text-watch",
 need_one_more_answer: "text-ink",
};

export function DecisionResult({
 locale,
 kind,
 headline,
 status,
 action,
 why,
 children,
}: Props) {
 return (
 <section className="flex flex-col gap-6">
 <header className="flex flex-col gap-3">
 <p className="text-caption font-medium uppercase tracking-[0.14em] text-ink-soft">
 {t("nextAction", locale)}
 </p>
 <h1
 className={`text-decision font-bold tracking-tight ${HEADLINE_COLOR[kind]}`}
 >
 {headline}
 </h1>
 <p className="text-body font-semibold text-ink">{status}</p>
 </header>

 {action ? (
 <button
 type="button"
 onClick={action.onClick}>
```

```

 className="min-h-14 rounded-2xl bg-action px-4 text-body font-semibold text-action-ink"
 >
 {action.label}
 </button>
) : null}

 {children}

 {why && why.length > 0 ? (
 <details className="rounded-2xl border border-line bg-raised px-4 py-3">
 <summary className="cursor-pointer text-label font-medium text-ink">
 {t("whyThis", locale)}
 </summary>
 <ul className="mt-3 flex flex-col gap-2 text-label text-ink-soft">
 {why.map((reason) => (
 <li key={reason}>{reason}
))}

 </details>
) : null}
 </section>
);
}

```

## components/surfaces/HouseholdSurface.tsx

Household renders the stored Decision, not the pipeline.

SHA-256 9ca3625c201d79fe39de194bf8b46638948118e63c2e3c299771cf274294037f · 143 lines

```

"use client";

import { AssessmentFlow } from "@components/AssessmentFlow";
import { CarePath } from "@components/CarePath";
import { DecisionResult } from "@components/DecisionResult";
import { HomeScreen } from "@components/HomeScreen";
import { NearbyFacilities } from "@components/NearbyFacilities";
import { RoleGate } from "@components/RoleGate";
import { Warning } from "@components/Warning";
import { decisionHeadline, decisionStatus } from "@lib/assessment";
import { t } from "@lib/copy";
import { warningCopy } from "@lib/failures";
import { stageLabel } from "@lib/referral";
import { activeFailures, useCare } from "@lib/store";

export function HouseholdSurface() {
 const { state, dispatch } = useCare();
 const encounter = state.encounter;
 const referral = state.referral;
 const locale = state.locale;

 if (state.view === "home") {
 const continueDetail = referral
 ? `${t("referralEyebrow", locale)} · ${stageLabel(referral.stage, locale)}`
 : encounter?.decision
 ? decisionHeadline(encounter.decision.kind, locale)
 : encounter && encounter.asks.length > 0
 ? t("homeCardAssessTitle", locale)
 : null;
 return (

```

```

 <HomeScreen
 locale={locale}
 continueDetail={continueDetail}
 onStart={() => dispatch.setView("path")}
 onNearby={() => dispatch.setView("nearby")}
 onContinue={() => dispatch.setView("path")}
 onHealthWorker={() => {
 dispatch.setGateRole(null);
 dispatch.setView("gate");
 }}
 />
);
}

if (state.view === "gate") {
 return <RoleGate />;
}

if (state.view === "nearby") {
 return (
 <div className="flex flex-col gap-4">
 <button
 type="button"
 onClick={() => dispatch.setView("home")}
 className="self-start text-label font-medium text-ink-soft"
 >
 {t("back", locale)}
 </button>
 <h1 className="text-decision font-semibold tracking-tight text-ink">
 {t("nearbyTitle", locale)}
 </h1>
 <NearbyFacilities locale={locale} />
 </div>
);
}

if (referral && encounter) {
 return <CarePath role="household" />;
}

if (encounter?.decision && !encounter.currentQuestion) {
 return <DecisionView />;
}

return <AssessmentFlow />;
}

function DecisionView() {
 const { state, dispatch } = useCare();
 const decision = state.encounter?.decision;
 const locale = state.locale;
 if (!decision) {
 return null;
 }

 const failures = activeFailures(state, null);
 const askedMain = state.encounter?.asked.includes("main_problem") ?? false;
 const action =
 decision.kind === "go_now" || decision.kind === "get_care_today"
 ? { label: t("prepareFacility", locale), onClick: dispatch.prepareReferral }
 : decision.kind === "need_one_more_answer" && !askedMain
 ? { label: t("answerTheQuestion", locale), onClick: dispatch.continueForOneMore }
 : undefined;

```

```

const incomplete = warningCopy("incomplete_assessment", locale);
const offlineCopy = warningCopy("offline", locale);
const danger = warningCopy("danger_sign", locale);
const watch = warningCopy("watch_sign", locale);

return (
 <DecisionResult
 locale={locale}
 kind={decision.kind}
 headline={decisionHeadline(decision.kind, locale)}
 status={decisionStatus(decision.kind, locale)}
 action={action}
 why={decision.reasonKeys.map((key) => t(key, locale))}
 >
 {failures.includes("incomplete_assessment") ? (
 <Warning
 named="incomplete_assessment"
 eyebrow={incomplete.eyebrow}
 title={incomplete.title}
 body={incomplete.body}
 />
) : null}
 {failures.includes("offline") ? (
 <Warning
 named="offline"
 eyebrow={offlineCopy.eyebrow}
 title={offlineCopy.title}
 body={offlineCopy.body}
 />
) : null}
 {decision.dangerSignKeys.length > 0 ? (
 <Warning
 named="danger_sign"
 eyebrow={danger.eyebrow}
 title={danger.title}
 body={decision.dangerSignKeys.map((key) => t(key, locale)).join(" · ")}
 />
) : null}
 {decision.kind === "monitor_at_home" ? (
 <Warning
 named="watch_sign"
 eyebrow={watch.eyebrow}
 title={watch.title}
 body={decision.watchSignKeys.map((key) => t(key, locale)).join(" · ")}
 />
) : null}
 </DecisionResult>
);
}

```

## components/surfaces/ChpSurface.tsx

CHP uses the same DecisionResult.

SHA-256 652906d914bae7973485bf432cf2800acca21f236ee5b07fdc8358b35ce377f2 · 95 lines

```

"use client";

import { AssessmentFlow } from "@components/AssessmentFlow";
import { AttentionNeeded } from "@components/AttentionNeeded";

```

```

import { CarePath } from "@components/CarePath";
import { DecisionResult } from "@components/DecisionResult";
import { Warning } from "@components/Warning";
import { decisionHeadline, decisionStatus, missingInfo } from "@lib/assessment";
import { t } from "@lib/copy";
import { warningCopy } from "@lib/failures";
import { followUpItems } from "@lib/referral";
import { activeFailures, useCare } from "@lib/store";

export function ChpSurface() {
 const { state, dispatch } = useCare();
 const encounter = state.encounter;
 const referral = state.referral;
 const locale = state.locale;

 if (!encounter) {
 return (
 <AttentionNeeded
 title={t("encounterTitle", locale)}
 items={[
 {
 id: "none",
 label: t("noEncounter", locale),
 detail: t("noEncounterDetail", locale),
 },
]}
 action={{ label: t("startEncounter", locale), onClick: dispatch.startEncounter }}
 />
);
 }

 if (referral) {
 return <CarePath role="chp" />;
 }

 if (encounter.decision && !encounter.currentQuestion) {
 const missing = missingInfo(encounter.answers, null, locale);
 const followUps = followUpItems("chp", null, encounter.decision.kind, locale);
 const failures = activeFailures(state, null);
 const kind = encounter.decision.kind;
 const askedMain = encounter.asked.includes("main_problem");
 const action =
 kind === "go_now" || kind === "get_care_today"
 ? { label: t("createReferral", locale), onClick: dispatch.prepareReferral }
 : kind === "need_one_more_answer" && !askedMain
 ? { label: t("answerTheQuestion", locale), onClick: dispatch.continueForOneMore }
 : undefined;
 const incomplete = warningCopy("incomplete_assessment", locale);

 return (
 <div className="flex flex-col gap-4">
 <DecisionResult
 locale={locale}
 kind={kind}
 headline={decisionHeadline(kind, locale)}
 status={decisionStatus(kind, locale)}
 action={action}
 why={encounter.decision.reasonKeys.map((key) => t(key, locale))}
 >
 {failures.includes("incomplete_assessment") ? (
 <Warning
 named="incomplete_assessment"
 eyebrow={incomplete.eyebrow}
 title={incomplete.title}
 />
) : null}
 </div>
);
 }
}

```

```

 body={incomplete.body}
 />
) : null}
 </DecisionResult>
 <AttentionNeeded title={t("missingInformation", locale)} items={missing} />
 <AttentionNeeded title={t("whoNeedsFollowUp", locale)} items={followUps} />
</div>
);
}

const missing = missingInfo(encounter.answers, null, locale);
return (
 <div className="flex flex-col gap-4">
 <AssessmentFlow />
 {encounter.asks.length > 0 ? (
 <details className="rounded-2xl border border-line bg-raised px-4 py-3">
 <summary className="cursor-pointer text-label font-medium text-ink">
 {t("missingInformation", locale)}
 </summary>
 <div className="mt-3">
 <AttentionNeeded items={missing} />
 </div>
 </details>
) : null}
 </div>
);
}

```

## components/surfaces/FacilitySurface.tsx

Facility speaks referral state, not a second clinical paragraph.

SHA-256 7bb2637971117ac311d51c8799ff3005f1630d1e462e3e97c5cea6bcec74ccc4 · 48 lines

```

"use client";

import { AssessmentQuestion } from "@components/AssessmentQuestion";
import { AttentionNeeded } from "@components/AttentionNeeded";
import { CarePath } from "@components/CarePath";
import { t } from "@lib/copy";
import { useCare } from "@lib/store";
import type { OutcomeCode } from "@lib/types";

export function FacilitySurface() {
 const { state, dispatch } = useCare();
 const referral = state.referral;
 const locale = state.locale;

 if (!referral) {
 return (
 <AttentionNeeded
 title={t("incomingTitle", locale)}
 items={[
 {
 id: "none",
 label: t("noIncoming", locale),
 detail: t("noIncomingDetail", locale),
 },
]}
 />
);
 }
}

```

```

);
 }

 if (referral.stage === "completed" && !referral.outcome) {
 return (
 <AssessmentQuestion
 locale={locale}
 question={t("outcomeQuestion", locale)}
 choices={[
 { id: "treated", label: t("outcomeTreated", locale) },
 { id: "referred_onward", label: t("outcomeHigher", locale) },
 { id: "did_not_arrive", label: t("outcomeNoShow", locale) },
]}
 dontKnowLabel={t("dontKnow", locale)}
 onDontKnow={() => dispatch.returnOutcome("unknown")}
 onChoose={(id) => dispatch.returnOutcome(id as OutcomeCode)}
 />
);
 }

 return <CarePath role="facility" />;
}

```

## components/CarePath.tsx

Referral presentation goes through describeReferral.

SHA-256 75ce8f94c75430458223d844a74468fe27fc9e12505ce10b650bef60dc0d4f7e · 181 lines

```

"use client";

import { AttentionNeeded } from "@components/AttentionNeeded";
import { FacilityCard } from "@components/FacilityCard";
import { PatientHandoff } from "@components/PatientHandoff";
import { ReferralStatus } from "@components/ReferralStatus";
import { Warning } from "@components/Warning";
import { missingInfo } from "@lib/assessment";
import { t } from "@lib/copy";
import { warningCopy } from "@lib/failures";
import { facilityById } from "@lib/facilities";
import { handoffFacts, whyComing } from "@lib/handoff";
import {
 describeReferral,
 followUpItems,
 type ReferralAction,
} from "@lib/referral";
import { activeFailures, useCare } from "@lib/store";
import type { Role } from "@lib/types";

export function CarePath({ role }: { role: Role }) {
 const { state, dispatch } = useCare();
 const referral = state.referral;
 const encounter = state.encounter;
 const locale = state.locale;

 if (!referral || !encounter) {
 return null;
 }

 const decisionKind = encounter.decision?.kind ?? null;

```

```

const view = describeReferral(role, referral, decisionKind, locale);
const facility = facilityById(referral.facilityId);
const failures = activeFailures(state, referral);
const missing = missingInfo(encounter.answers, referral.askMore, locale);
const followUps = followUpItems(role, referral, decisionKind, locale);
const stale = failures.includes("stale_information");
const known = handoffFacts(encounter.answers, stale, encounter.updatedAt, locale);
const canHandle = decisionKind !== "go_now" || facility.canHandleUrgent;

const performAction: Partial<Record<ReferralAction, () => void>> = {
 send: dispatch.sendReferral,
 accept: dispatch.acceptReferral,
 travel: dispatch.markTraveling,
 arrive: dispatch.markArrived,
 start_care: dispatch.startCare,
 complete: dispatch.completeVisit,
};

const onAction = view.action ? performAction[view.action.id] : undefined;
const action =
 view.action && onAction
 ? { label: view.action.label, onClick: onAction }
 : undefined;

const facilityStatus = canHandle
 ? role === "facility"
 ? t("facilityWeCanHandle", locale)
 : t("facilityCanTake", locale)
 : t("facilityMayNot", locale);

const handoff = (
 <PatientHandoff
 locale={locale}
 why={whyComing(encounter.answers, decisionKind, locale)}
 known={known}
 missing={missing.map((item) => item.label)}
 />
);

const facilityCard = (
 <FacilityCard
 locale={locale}
 name={facility.name}
 travelMinutes={facility.travelMinutes}
 canHandle={canHandle}
 services={facility.services}
 statusLabel={facilityStatus}
 />
);

const warnings = failures.map((named) => {
 const copy = warningCopy(named, locale);
 return (
 <Warning
 key={named}
 named={named}
 eyebrow={copy.eyebrow}
 title={copy.title}
 body={copy.body}
 />
);
});

const secondaryActions =

```

```

role === "facility" &&
(referral.stage === "sent" || referral.stage === "received") ? (
 <details className="rounded-2xl border border-line bg-raised px-4 py-3">
 <summary className="cursor-pointer text-label font-medium text-ink">
 {t("otherActions", locale)}
 </summary>
 <div className="mt-3 flex flex-col gap-2">
 <button
 type="button"
 onClick={dispatch.redirectReferral}
 className="min-h-12 rounded-xl border border-line px-3 text-left text-body font-medium"
 >
 {t("redirect", locale)}
 </button>
 <button
 type="button"
 onClick={dispatch.askMore}
 className="min-h-12 rounded-xl border border-line px-3 text-left text-body font-medium"
 >
 {t("askMore", locale)}
 </button>
 </div>
 </details>
) : null;

return (
 <div className="flex flex-col gap-4">
 <ReferralStatus
 locale={locale}
 role={role}
 stage={referral.stage}
 stageLabel={view.stageLabel}
 headline={view.headline}
 status={view.status}
 arrivalMinutes={view.arrivalMinutes}
 action={action}
 >
 {warnings}
 </ReferralStatus>

 {role === "facility" ? (
 <>
 {handoff}
 {facilityCard}
 {secondaryActions}
 {missing.length > 0 ? (
 <AttentionNeeded title={t("missingInformation", locale)} items={missing} />
) : null}
 </>
) : null}

 {role === "household" ? (
 <>
 {facilityCard}
 <details className="rounded-2xl border border-line bg-raised px-4 py-3">
 <summary className="cursor-pointer text-label font-medium text-ink">
 {t("moreDetail", locale)}
 </summary>
 <div className="mt-3 flex flex-col gap-3">{handoff}</div>
 </details>
 </>
) : null}

 {role === "chp" ? (

```

```

<>
 <AttentionNeeded
 title={t("needsAction", locale)}
 items={([...followUps, ...missing]}
 emptyLabel={
 followUps.length === 0 && missing.length === 0
 ? t("nothingWaiting", locale)
 : undefined
 }
 />
 {facilityCard}
 <details className="rounded-2xl border border-line bg-raised px-4 py-3">
 <summary className="cursor-pointer text-label font-medium text-ink">
 {t("moreDetail", locale)}
 </summary>
 <div className="mt-3">{handoff}</div>
 </details>
</>
) : null}
</div>
);
}

```

## evals/gates.ts

Release machinery. Promotion is blocked. This file cannot authorize a release.

SHA-256 41c4487ad7df83b5e2de5129e3bfce76e9bbb48cc47a7218fe0707590148d237 · 354 lines

```

import { readFileSync } from "node:fs";
import { resolve, sep } from "node:path";
import type { AssessmentAnswers, DecisionKind } from "../lib/types";
import { actionClassFor } from "../lib/clinical/contract";
import {
 contentHash,
 loadSuite,
 type ManifestEntry,
 type Suite,
} from "../runner";

/**
 * Fail-closed release machinery. This module cannot authorize a release.
 * Missing metrics and unsigned clinical thresholds block promotion.
 * Expected labels are read from fixtures. They are not derived here.
 */

export const ACTION_DISTRIBUTION_KEYS = [
 "ANSWER",
 "ASK",
 "ABSTAIN",
 "ESCALATE",
] as const;

export type DistributionKey = (typeof ACTION_DISTRIBUTION_KEYS)[number];

export type GateStatus = "pass" | "fail" | "blocked";

export type GateRow = {
 id: string;
 status: GateStatus;

```

```

 reason: string;
};

export type PromotionDecision = {
 promotion: "blocked";
 release_authorization: "none";
 clinical_effectiveness: "not_established";
 rows: GateRow[];
 block_reasons: string[];
 distribution: Record<DistributionKey, number> | null;
};

export type GateFamily = {
 status?: "measured" | "not_measurable";
 value?: number;
 numerator?: number;
 denominator?: number;
 cases?: { language?: string; matched?: boolean }[];
};

export type AdversarialClass =
 | "look_alike"
 | "paraphrase_bypass"
 | "colloquial_misspelling_codeswitch"
 | "injection_inert";

export type AdversarialEntry = {
 id: string;
 class: AdversarialClass;
 language: "en" | "sw" | "und";
 lineage: "SYNTHETIC";
 execution: "inert_data_not_executed";
 training_set: false;
 input: Partial<AssessmentAnswers>;
 expected: DecisionKind | null;
 label_status: "engineering_assertion_unverified" | "withheld_not_a_clinical_label";
 note: string;
 provenance: ManifestEntry["provenance"];
 content_hash: string;
};

export type AdversarialCatalog = {
 purpose: "engineering_fixtures_only";
 training_set: false;
 label_limit: string;
 entries: AdversarialEntry[];
};

const BLANK: AssessmentAnswers = {
 who: null,
 presentation: "",
 photoAttached: false,
 awake: null,
 breathing: null,
 drinking: null,
 duration: null,
 mainProblem: null,
};

const REQUIRED_UNMEASURED = [
 "clinical_correctness",
 "calibration",
 "retrieval",
 "multilingual_generation",

```

```

 "medication_safety",
] as const;

function isRecord(value: unknown): value is Record<string, unknown> {
 return typeof value === "object" && value !== null && !Array.isArray(value);
}

export function hashWithoutContentHash(value: unknown): string {
 if (!isRecord(value)) return contentHash(value);
 const rest: Record<string, unknown> = {};
 for (const key of Object.keys(value)) {
 if (key === "content_hash") continue;
 rest[key] = value[key];
 }
 return contentHash(rest);
}

export function loadAdversarialSuite(evalsRoot: string): Suite {
 return loadSuite(evalsRoot, "adversarial/manifest.json");
}

export function loadAdversarialCatalog(evalsRoot: string): AdversarialCatalog {
 const path = resolve(evalsRoot, "adversarial/catalog.json");
 if (!path.startsWith(resolve(evalsRoot) + sep)) {
 throw new Error("Adversarial catalog outside evals");
 }
 const catalog = JSON.parse(readFileSync(path, "utf8")) as AdversarialCatalog;
 if (catalog.purpose !== "engineering_fixtures_only" || catalog.training_set !== false) {
 throw new Error("Adversarial catalog must stay an engineering fixture, not a training set");
 }
 if (!catalog.label_limit.includes("unverified")) {
 throw new Error("Adversarial catalog is missing the unverified-authorship limit");
 }
 const seen = new Set<string>();
 for (const entry of catalog.entries) {
 if (seen.has(entry.id)) throw new Error(`Duplicate adversarial id: ${entry.id}`);
 seen.add(entry.id);
 if (entry.lineage !== "SYNTHETIC" || entry.execution !== "inert_data_not_executed") {
 throw new Error(`Adversarial entry is not inert synthetic data: ${entry.id}`);
 }
 if (entry.training_set !== false) {
 throw new Error(`Adversarial entry must not be a training row: ${entry.id}`);
 }
 if (entry.provenance.label_derivation !== "explicit_assertion_not_runtime_output" ||
 entry.provenance.human_authorship !== "unverified" ||
 entry.provenance.clinical_review !== "not_reviewed") {
 throw new Error(`Adversarial provenance is not an unverified assertion: ${entry.id}`);
 }
 if (hashWithoutContentHash(entry) !== entry.content_hash) {
 throw new Error(`Adversarial hash mismatch: ${entry.id}`);
 }
 if (entry.class === "injection_inert" && !JSON.stringify(entry.input).includes("INERT_FIXTURE")) {
 throw new Error(`Injection fixture is missing its inert marker: ${entry.id}`);
 }
 }
 return catalog;
}

export type AdversarialObservation = {
 id: string;
 class: AdversarialClass;
 expected: DecisionKind | null;
 observed: DecisionKind;
 matched: boolean | null;
};

```

```

 executed: false;
 };

export function observeAdversarial(
 entries: readonly AdversarialEntry[],
 decider: (answers: AssessmentAnswers) => { kind: DecisionKind },
): AdversarialObservation[] {
 return entries.map((entry) => {
 const observed = decider({ ...BLANK, ...entry.input }).kind;
 return {
 id: entry.id,
 class: entry.class,
 expected: entry.expected,
 observed,
 matched: entry.expected === null ? null : observed === entry.expected,
 executed: false,
 };
 });
}

export function distributionFromKinds(
 kinds: readonly DecisionKind[],
): Record<DistributionKey, number> {
 const counts: Record<DistributionKey, number> = {
 ANSWER: 0,
 ASK: 0,
 ABSTAIN: 0,
 ESCALATE: 0,
 };
 for (const kind of kinds) {
 counts[actionClassFor(kind)] += 1;
 }
 return counts;
}

function sliceRates(family: GateFamily | undefined): Record<string, { numerator: number; denominator: number }> {
 const slices: Record<string, { numerator: number; denominator: number }> = {};
 for (const item of family?.cases ?? []) {
 const key = item.language ?? "und";
 const slice = slices[key] ?? { numerator: 0, denominator: 0 };
 slice.denominator += 1;
 if (item.matched) slice.numerator += 1;
 slices[key] = slice;
 }
 return slices;
}

function measuredValue(family: GateFamily | undefined): number | null {
 if (!family || family.status !== "measured" || typeof family.value !== "number") return null;
 return family.value;
}

export function evaluateReleaseGate(input: {
 baseline: Record<string, GateFamily | undefined>;
 challenger: Record<string, GateFamily | undefined>;
 distribution: Partial<Record<DistributionKey, number>> | null;
 clinical_thresholds_signed: boolean;
}): PromotionDecision {
 const rows: GateRow[] = [];
 const blockReasons: string[] = [];

 const baselineRecall = measuredValue(input.baseline.red_flag_recall);
 const challengerRecall = measuredValue(input.challenger.red_flag_recall);

```

```

if (baselineRecall === null || challengerRecall === null) {
 rows.push({
 id: "red_flag_recall_gte_baseline",
 status: "blocked",
 reason: "red-flag recall is missing or not measured",
 });
 blockReasons.push("missing_red_flag_recall");
} else if (challengerRecall + 1e-12 < baselineRecall) {
 rows.push({
 id: "red_flag_recall_gte_baseline",
 status: "fail",
 reason: "challenger red-flag recall is below the frozen baseline",
 });
 blockReasons.push("red_flag_recall_regression");
} else {
 rows.push({
 id: "red_flag_recall_gte_baseline",
 status: "pass",
 reason: "challenger red-flag recall is at least the frozen baseline",
 });
}

const baselineSlices = sliceRates(input.baseline.red_flag_recall);
const challengerSlices = sliceRates(input.challenger.red_flag_recall);
const sliceKeys = Object.keys(baselineSlices);
let sliceFailed = false;
if (sliceKeys.length === 0) {
 rows.push({
 id: "no_safety_critical_slice_regression",
 status: "blocked",
 reason: "baseline has no language slices to compare",
 });
 blockReasons.push("missing_slices");
} else {
 for (const key of sliceKeys) {
 const before = baselineSlices[key];
 const after = challengerSlices[key];
 if (!after || after.denominator < before.denominator || after.numerator < before.numerator) {
 sliceFailed = true;
 }
 }
 rows.push({
 id: "no_safety_critical_slice_regression",
 status: sliceFailed ? "fail" : "pass",
 reason: sliceFailed
 ? "a safety-critical language slice regressed or disappeared"
 : "language slices did not regress against the frozen baseline",
 });
 if (sliceFailed) blockReasons.push("slice_regression");
}

const distribution = input.distribution;
const distributionComplete = distribution !== null &&
 ACTION_DISTRIBUTION_KEYS.every((key) => typeof distribution[key] === "number");
rows.push({
 id: "disposition_distribution_reported",
 status: distributionComplete ? "pass" : "blocked",
 reason: distributionComplete
 ? "ANSWER/ASK/ABSTAIN/ESCALATE counts are present, including zeros"
 : "disposition distribution is missing a required key",
});
if (!distributionComplete) blockReasons.push("missing_distribution");

rows.push({

```

```

 id: "clinical_thresholds_signed",
 status: input.clinical_thresholds_signed ? "pass" : "blocked",
 reason: input.clinical_thresholds_signed
 ? "a signature flag was supplied; this module still cannot authorize release"
 : "clinical thresholds are unsigned",
 });
 if (!input.clinical_thresholds_signed) blockReasons.push("unsigned_clinical_threshold");

 const missingMetrics = REQUIRED_UNMEASURED.filter((family) => {
 const baseline = input.baseline[family];
 const challenger = input.challenger[family];
 return !baseline || baseline.status !== "measured" || !challenger || challenger.status !==
"measured";
 });
 rows.push({
 id: "required_metrics_present",
 status: missingMetrics.length === 0 ? "pass" : "blocked",
 reason: missingMetrics.length === 0
 ? "required metric families are measured"
 : `missing or not_measurable: ${missingMetrics.join(", ")}`,
 });
 if (missingMetrics.length > 0) blockReasons.push("missing_metric");

 return {
 promotion: "blocked",
 release_authorization: "none",
 clinical_effectiveness: "not_established",
 rows,
 block_reasons: blockReasons.length > 0 ? blockReasons : ["release_not_authorized_by_this_module"],
 distribution: distributionComplete ? distribution as Record<DistributionKey, number> : null,
 };
}

export type ChampionComparison = {
 input_identity: "identical_frozen_inputs";
 input_count: number;
 agreement_count: number;
 disagreement_count: number;
 promotion: "blocked";
 release_authorization: "none";
};

export function compareChampionChallenger<T>(
 inputs: readonly T[],
 champion: (input: T) => string,
 challenger: (input: T) => string,
) : ChampionComparison {
 let agreement = 0;
 for (const input of inputs) {
 if (champion(input) === challenger(input)) agreement += 1;
 }
 return {
 input_identity: "identical_frozen_inputs",
 input_count: inputs.length,
 agreement_count: agreement,
 disagreement_count: inputs.length - agreement,
 promotion: "blocked",
 release_authorization: "none",
 };
}

export function assertNoReleaseAuthorization(decision: PromotionDecision): void {
 if (decision.promotion !== "blocked" || decision.release_authorization !== "none") {
 throw new Error("release gate failed closed check");
 }
}

```

```

}
if (decision.clinical_effectiveness !== "not_established") {
 throw new Error("clinical effectiveness must stay unestablished");
}
}

```

## vault/architecture.md

Module map as written in the repo vault.

SHA-256 ec884e3b547fdd5120744ebbcbeed887387ae25fccdb7180a5a5409fe9832876 · 72 lines

### # Architecture

One Next.js App Router app (v16, React 19, Tailwind 4, Vitest). Everything in v1 is client-side and deterministic – no backend yet.

### ## Load-bearing modules

- ``lib/copy.ts`` – every user-visible string, EN + SW, one typed table. ``sw`` is ``Record<CopyKey, string>``, so a missing translation is a compile error. ``t(key, locale)`` requires an explicit locale. Templates use ``{f}``, ``{m}``, ``{who}`` via ``fill()``.
- ``lib/referral.ts`` – the canonical lifecycle from the README: ``created`` → ``sent`` → ``received`` → ``accepted`` → ``patient_moving`` → ``arrived`` → ``seen`` → ``completed`` → ``outcome_returned``. ``describeReferral(role, referral, kind, locale)`` is the ONE place referral state becomes words: (state × role × language) → headline, status, stage label, and a ``**semantic action id**`` (never matched by button text). Outcomes are codes.
- ``lib/assessment.ts`` – deterministic demo triage rules (conservative, hears EN + SW danger keywords, abstains via ``need_one_more_answer``). Emits copy keys, not sentences. ``**Not clinically validated**`` – the README's deterministic safety layer replaces this before real use.
- ``lib/store.tsx`` – reducer + context; state: role, locale, view (household home/path/nearby), injectedFailures, encounter, referral, online. Persists to ``localStorage`` ``tunza.v2.care``.
- ``lib/failures.ts`` – ``warningCopy(named, locale)``: eyebrow/title/body for the six failure states + danger/watch.
- ``components/`` – the seven contract components + ``HomeScreen``, ``NearbyFacilities``, ``CareShell``, ``DemoChrome`` (demo harness: role switcher, failure injection, language toggle, start over; wordmark navigates home for household).
- ``lib/voice.ts`` – one tap-to-speak path with a designed degrade chain: server Whisper (``/api/transcribe``, needs ``OPENAI_API_KEY``) → on-device SpeechRecognition (``lib/speech.ts``) → "type instead" message. Capability probed via ``GET`` ``/api/transcribe`` (cached).
- ``lib/places.ts`` + ``lib/geohash.ts`` + ``app/api/facilities`` – real nearby facilities (Google Places New, needs ``GOOGLE_PLACES_API_KEY``), ported from the old app's privacy pattern: precise coords never leave the device; the stateless proxy sees only the precision-5 geohash cell center; distances ranked client-side. Nearby view falls back to the demo list with an honest note on any failure (no geolocation, denied, unanswered prompt via a 12s watchdog, offline, 503-unconfigured, zero results).
- ``app/api/access`` + ``components/RoleGate.tsx`` – the health-worker gate: POST checks role+code server-side (env ``CHP_ACCESS_CODE`` / ``FACILITY_ACCESS_CODE``; demo codes CHP-DEMO / FACILITY-DEMO until set, and GET reports demo mode so the screen can say so). Grants live in state (``grants``), enforced in the store reducer; wordmark → home from any role; Start over clears grants. Placeholder for real worker identity.
- ``.env.example`` – ``OPENAI_API_KEY``, ``GOOGLE_PLACES_API_KEY``, ``CHP_ACCESS_CODE``, ``FACILITY_ACCESS_CODE``; all optional, everything

degrades by design without them. The repo is PUBLIC unless flipped: keys live only in ``.env.local`` / deploy env, never committed.

## Quality gates (all must pass before any push)

``npx tsc --noEmit` · `npm test` (66) · `npm run lint` · `npm run build``

Tests include: copy parity + placeholder equality, canonical lifecycle, locked per-role phrases in both languages, component renders in both languages, and ``tests/contrast.test.ts`` which parses ``globals.css`` tokens and enforces WCAG pairs plus the two-red luminance split.

## Verification habit

Big UI changes get a real-browser pass: Playwright-core (temp devDependency, uninstalled before commit) driving Chromium at ``/opt/pw-browsers/chromium`` through the full journey in both languages. Scripts live in the session scratchpad, not the repo.

## The old app (``evanmotovich1-web/medical-triage``, private)

Next 14 + Supabase + Anthropic triage + Whisper voice + Google Places + web push. Source of ports (home screen already taken). Its i18n table (``lib/i18n.ts``) is the reference for existing EN/SW copy.

## vault/decisions.md

Dated decisions, including why wave 1 is a contract and not a model.

SHA-256 c166b008cfe2238a1b213a7f9f5ee038f7e0663a5d991dbbd9d8ca8d09c486e4 · 60 lines

# Decisions

Dated, newest first. Each entry: what was decided, and why it holds.

## 2026-10-06 – Wave 1 is an engineering contract, not a model  
The parent decision on the eval labels is: accept test-sourced labels for engineering-only regression, with human authorship explicitly unverified. They are not reviewed clinical labels. Training, retrieval ingestion, and guideline-threshold encoding stay closed until ``vault/wave2-ledger.md`` says their evidence exists. The demo floor in ``lib/assessment.ts`` remains the only approved path in this slice, and it is still not a clinical protocol. No push and no release follow from the green engineering gates.

## 2026-08-27 – CHP and facility go behind an access gate  
Per Evan: the wordmark always returns to the home page; from home a regular person can only enter the household path, and CHP/facility require signing in. Implemented as a server-checked access-code gate (``/api/access``; codes in env, never shipped to the client; well-known demo codes CHP-DEMO / FACILITY-DEMO work until real codes are set, and the sign-in screen says so). Grants persist per device; Start over clears them; the reducer refuses gated roles without a grant so no caller can skip the gate. This is UX-level gating and the placeholder for real worker identity/sign-up – not account security. The demo role switcher was removed from the Demo panel (it was a backdoor around the gate).

## 2026-08-26 – The front door wears the brand  
The household landing is the old app's home screen rebuilt in the matte finish – the ONE full-brand surface (``--brand-deep`` → ``--action``). Per Evan's review, it is full-screen red: the header chrome, footer, and the ground

behind the column all wear it (white chrome text; the Demo panel opens on a paper card). Everything after it wears paper, keeping red scarce. Old copy reused verbatim from `medical-triage/lib/i18n.ts`; History slot became a live Continue card fed by the referral machine; "Are you a doctor?" became "Are you a health worker?" → CHP surface.

## 2026-08-26 – Two reds, one discipline (matte red finish)  
Brand/action = dark oxblood `#7c1f18`; emergency = brighter `#b3261e`,  $\geq 1.5\times$  luminance apart (test-enforced), same warm hue family. Verified precedents: Stanford Cardinal vs Digital red split, USWDS theme-vs-state token roles, UCSF/OSHA "red = top severity tier only". Green left the product entirely. Emergency red never fills a button; brand red never marks danger.

## 2026-08-26 – Monitor verdict is watch-blue, not green  
"Stay home" must read as "keep watching", never "all clear" – and red-green is the most impaired color-vision axis. Doubly right once red became brand.

## 2026-08-26 – Canonical referral states win  
The README's lifecycle (with `seen`, `patient\_moving`) replaced the ad-hoc `prepared/traveling` set from the first bot build. Internal names never appear on screen; `describeReferral` is the single translation point and returns semantic action ids (English-label dispatch was a latent i18n bug).

## 2026-08-26 – Language is compile-enforced  
`sw: Record<CopyKey, string>` + `t(key, locale)` with required locale: a screen cannot ship in one language by accident. Decisions carry copy keys so language applies at render, not at decision time.

## 2026-08-26 – One codebase, not parallel builds  
The Cursor agent's v1 was merged into the contract branch and aligned, rather than building a rival – per the contract's own "one foundation" rule.

## vault/capability-map.md

What exists, what is missing, and which README claims are not code.

SHA-256 5d12d4a47322bd9ded6098ccc774e4156aab050f447b76fe05e89ae1fada9fcc · 242 lines

# Tunza capability map – wave 1 baseline audit

Dated 2026-10-07. Branch `clinical-contract-wave1` (from `main` @ `2a42af2`).  
Status labels: EXISTING / PARTIAL / MISSING / UNSAFE / DUPLICATED / DEPRECATED.  
Evidence is file:line in one of two repos:

- **\*\*PUBLIC\*\*** = `/Users/evanmotovich/code/Tunza` (paths relative to repo root)
- **\*\*PRIVATE\*\*** = `/Users/evanmotovich/code/medical-triage` (the "clone model" lane)

Code is truth. README claims are reconciled in §17. No PHI, no keys, no verbatim guideline text appear in this document.

## 0. What the "clone model" actually is

**\*\*There are no owned weights, no adapter, no fine-tune artifacts, and no training dataset in either tree.\*\*** The "existing Tunza clone model" is the PRIVATE lane: a system prompt in front of Anthropic's API (`app/api/triage/route.ts:28-61`, `:130`). It is a hosted-model integration, not a clone. Every directive section that assumes an evolvable model (fine-tuning §19-23, distillation, quantization, champion weights) has no substrate to act on yet. This is a measured finding, not an opinion.

The PUBLIC app has no model at all: its decisions come from the deterministic function ``decide()`` (``lib/assessment.ts:91``), explicitly commented `"**Not clinically validated**"` (``lib/assessment.ts:33``).

#### ## 1. Architecture – EXISTING (public), DUPLICATED at product level

- PUBLIC: one Next.js 16 App Router app, React 19, client-side and deterministic, no backend beyond three stateless routes (``vault/architecture.md``, ``package.json:13-17`` – runtime deps are only `next/react/react-dom`).
- Three roles on one referral object: household, CHP, facility (``lib/types.ts:3`` Role; grants enforced in the store reducer).
- PRIVATE: a second, older full app (Next 14 + Supabase + Anthropic + web push). Two triage products exist. Wave 1 treats PRIVATE as read-only; any merge is a later adapter task (see ``vault/wave1-contract.md``).

#### ## 2. Inference paths – EXISTING (public, deterministic) / EXISTING-UNSAFE (private)

- PUBLIC: ``decide(answers) → Decision`` (``lib/assessment.ts:91``), pure and synchronous. Question flow ``nextQuestion``/``shouldStopForDecision`` (``lib/assessment.ts`` after ``decide``), fixed order ``QUESTION_ORDER`` (``lib/assessment.ts:21``).
- PRIVATE: ``app/api/triage/route.ts`` – measured gaps, all UNSAFE:
  - model hardcoded `"claude-sonnet-4-5"` (``:130``); only ``max_tokens: 1024`` (``:131``); `**no timeout / AbortSignal anywhere in the route**`.
  - response ``JSON.parse(...)`` as `TriageResponse`` – a cast, not runtime validation (``:147``).
  - on parse failure it logs the raw model text (``console.error`` at ``:149``) – model output mixed with patient-submitted symptoms is a log-exposure risk.
  - ``redFlags`` is optional in the type (``lib/types.ts:37``) while the prompt demands it always (``route.ts:61``); nothing enforces the contract at runtime.
  - persistence is best-effort: insert failure is logged and the response is still returned (``:177-182``); inference can finish with no durable record.
  - No deterministic floor sits under the model: nothing stops the private lane answering ``self-care`` in a presentation where the public rules would say ``go_now``. The two paths are not connected.

#### ## 3. Retrieval paths – MISSING (both repos)

No retrieval, RAG, reranker, evidence bundle, or knowledge-source code exists in either tree. Nothing to extend; wave 1 only reserves the contract slot (empty evidence section with explicit ``retrieval_status``).

#### ## 4. Datasets – MISSING (both repos)

No training or evaluation datasets exist. The PRIVATE ``cases`` table (``supabase/migrations/0001_initial_schema.sql:7-18``) is operational storage of individual submissions, not a dataset: a case row is not a training example, carries no label provenance, no review status, and no data-rights basis for reuse. eCHIS access, where it exists, is not a training license.

#### ## 5. Fine-tuning infrastructure – MISSING

No trainer, no adapter, no dataset writer, no lineage metadata anywhere. The 2026-09-10 "strong base + SFT + locked evals" intent was never executed. Wave 1 adds `**no**` training entrypoint (see ``vault/wave1-contract.md``).

#### ## 6. Evaluation infrastructure – PARTIAL (public engineering tests only)

- PUBLIC: 7 vitest files (``tests/``): copy parity EN/SW, canonical referral lifecycle, per-role locked phrases, component renders in both languages, WCAG contrast token parsing, assessment cases, places/facilities, API routes. ``vitest.config.ts`` includes ``tests/**/*.test.{ts,tsx}``.

These are regression tests, not clinical evaluation: no frozen corpora, no metric families, no slice reporting, no release gates, no champion/challenger.

- PRIVATE: no evaluation code beyond the build.
- MISSING everywhere: the directive's eval families (clinical correctness, safety, triage, abstention, grounding, retrieval, multilingual, longitudinal, contradiction, medication safety, robustness, security, latency, cost).

#### ## 7. Prompts / system instructions – PARTIAL, DUPLICATED vocabulary

- PRIVATE system prompt lives inline in the route (`route.ts:28-61`): conservative-escalation instruction (`:34`), vitals handling (`:39`), language rule (`:41`), JSON shape with three-level urgency `self-care | see-clinic | urgent` (`:59`) and mandatory red flags (`:61`).
- PUBLIC needs no prompt (deterministic), but the **\*urgency** vocabularies disagree: private 3-level enum vs public 4 `DecisionKind`s (`lib/types.ts:28-32`). Not the same scale. A disposition mapping table is required before the lanes ever meet; wave 1 freezes it inside the contract module.

#### ## 8. Patient-context handling – PARTIAL

- PUBLIC: `AssessmentAnswers` = 7 questions, last-write-wins (`lib/types.ts:65-80`). No per-field source, timestamp, confidence, or provenance; conflicting values overwrite silently. `Encounter` (`lib/types.ts:114`) holds one answer snapshot.
- PRIVATE: one submission snapshot per case; `vitals` is a single `jsonb` blob (`0001\_initial\_schema.sql:15`), so a 37.4→39.2 C rise cannot be computed from storage. The route builds its prompt from the current request only; no longitudinal history is loaded.
- No `PatientState` with append-only events exists anywhere. Wave 1 adds the event/state layer, encounter-scoped.

#### ## 9. Persistence / storage – PARTIAL, DUPLICATED

- PUBLIC: browser localStorage `tunza.v2.care` (`lib/store.tsx:33`, read `:348`, write `:436`). Device-local, single-state, no event log.
- PRIVATE: Supabase Postgres, 5 migrations (cases, nearby\_cases, doctors, doctor\_notifications, push\_subscriptions), own-row RLS.
- Neither is an append-only clinical event store; neither supports longitudinal reasoning. Two competing persistence models remain until an adapter decision is made (wave 2+, ticketed).

#### ## 10. API boundaries – EXISTING (public), degrade-by-design

- `app/api/access` – worker gate; env codes `CHP\_ACCESS\_CODE` / `FACILITY\_ACCESS\_CODE`, well-known demo codes otherwise, and GET reports demo mode so the UI can say so (`app/api/access/route.ts:8-22,25`).
- `app/api/facilities` – stateless Google Places proxy; receives only the coarse geohash cell center, mandatory field mask, key never leaves the server, raw provider errors never forwarded (`app/api/facilities/route.ts:8-9,19-24`).
- `app/api/transcribe` – Whisper proxy; 503 when `OPENAI\_API\_KEY` absent, capability probe via GET (`app/api/transcribe/route.ts:10,29-32,41`).
- `env.example`: every key optional; each route degrades by design.
- PRIVATE: `triage`, `transcribe`, `facilities`, `push-dispatch` routes; the triage boundary's gaps are in §2.

#### ## 11. Observability – MISSING (public), UNSAFE (private)

- PUBLIC: none beyond tests; no telemetry, no disagreement records.
- PRIVATE: `console.error` only – including raw model text at `route.ts:149` (see §2). No structured telemetry, no version stamps on results, no rule/model disagreement capture.

- Wave 1 introduces provenance on every result (`rules\_version`, `model\_id` null on demo path, `knowledge\_version` null, `last\_sync` null, `generated\_at`) and disagreement records in the reconciler.

## ## 12. Safety mechanisms – PARTIAL, with one UNSAFE junction

- PUBLIC floor: conservative demo rules; danger keywords EN/SW (`lib/assessment.ts:36-46`), abstention via `need\_one\_more\_answer` (`lib/assessment.ts:134,143,195,203`), stop-early on danger (`shouldStopForDecision`), on-screen disclaimer copy in both languages. Named failure states (`lib/types.ts:20-26`, `lib/failures.ts`) are honest UI degrade states.
- UNSAFE junction: the private model path has no floor under it (§2) and no output validation; the lanes are unreconciled.
- Keyword rules are bypassable by paraphrase and by any language outside the EN/SW patterns. They are a floor, not a clinical protocol. Do not encode IMCI/ETAT numeric thresholds in wave 1 – that would fake clinical approval (see `vault/wave1-contract.md` deferrals).

## ## 13. Clinical rules – PARTIAL (demo only, unversioned)

- One unversioned rule set (`lib/assessment.ts` DANGER\_PATTERNS + `decide` branches). No rule-pack versioning, no effective dates, no clinical review trail, no change log. Marked not clinically validated in-file (`:33`) and in `vault/backlog.md` ("clinical review required before real use").
- Wave 1 adds `rules\_version` provenance and a reconciler that makes the floor non-downgradable; it does **not** author new clinical content.

## ## 14. Model / version management – MISSING

- PRIVATE: hardcoded model string (`route.ts:130`); no registry, no champion/challenger, no reproducibility metadata (base model, adapter, data versions, config).
- PUBLIC: nothing to manage yet.
- Wave 1 lays the provenance fields; registries are wave 2+.

## ## 15. Authentication / authorization boundaries – PARTIAL

- PUBLIC: worker gate is a placeholder for real identity – demo codes until env set (`app/api/access/route.ts`), grants held in client state, enforced in the reducer; "Start over" clears grants.
- PRIVATE: anonymous Supabase sessions; middleware refreshes cookies and **fails open** (passthrough) if Supabase is unconfigured (`middleware.ts:4-12`) – deliberate availability choice, acceptable only because persistence, not clinical access, hangs off it.
- No clinician credential verification exists in either tree (README's "verified-clinician workflows" is not code; see §17).

## ## 16. Duplicated / competing primitives

```
Primitive	PUBLIC	PRIVATE	Disposition
Triage decision	`decide()` 4 kinds (`lib/types.ts:28-32`)	prompt 3 levels (`route.ts:59`)	
freeze disposition table in wave-1 contract; adapter maps private onto it			
Persistence	localStorage `tunza.v2.care`	Supabase `cases`	keep separate this wave; event-store
design must fit both later			
i18n	`lib/copy.ts` typed EN/SW (`:362,691`)	`lib/i18n.ts`	PUBLIC table is authoritative
pattern; missing SW string is a compile error			
Facility data	demo list (`lib/facilities.ts:3-4` "Clearly fake") + Places nearby	Places port	
(origin of the pattern)	nearby = Places via location-blind proxy; referral stays demo until		
capability truth exists			
Voice	degrade chain server→on-device-type	server Whisper	PUBLIC chain is authoritative
```

Nothing is DEPRECATED in the public tree. The private lane as a whole is a

deprecated-in-waiting surface: it is the adapter target, not the trunk.

### ## 17. README claims vs code (reconciliation)

Imported from the fusion-harness fact sheet

(`prompts/tunza-vision-2026-09-23/fact-sheet.md`, verified against this tree) and re-verified here. The public README's "What exists today" list includes capabilities the public code does not have; several describe the private lane or intent:

- Optional vitals: no `vitals` match in public `\*.ts/tsx` - MISSING in public (private has the jsonb blob only).
- Case history for signed-in users, community case signals, outbreak signal, verified-clinician workflows, clinician notifications, web push, PWA: all README bullets without public-code support (`vault/backlog.md` lists them as backlog).
- Image input is a boolean attachment; the rules do not read the image.
- Referral uses demo facilities, not live Places results.
- The deterministic safety layer replacing the demo rules is backlog, with clinical review required.

Rule for wave 1 and beyond: **code is truth**; the README is intent. The capability map, not the README, is the baseline future waves are measured against.

### ## 18. Hygiene observations (no action this wave)

- PRIVATE contains a stray accidental file named  
`.env.local moto@Evans-MacBook-Pro medical-triage % nano .env.local`  
(a pasted shell command that became a filename). Not read, not touched. Delete it in the adapter wave; it must never be committed anywhere.
- No PHI or credentials were read or copied during this audit. `.env.local` files in both repos were not opened.

## vault/wave1-contract.md

Wave-1 scope, deferrals, and the files that must not be edited.

SHA-256 4ae6ae98b3f4b11748768629a7d64924b84015a9ff31caf5d2a7ca7c63eadec8 · 163 lines

### # Wave 1 contract - clinical contract spine

Dated 2026-10-07. Branch `clinical-contract-wave1` (from `main` @ `2a42af2`). This document is the agreement every wave-1 task works under. The audit that justifies it is `vault/capability-map.md`.

#### ## 1. Repository and branch decision

- All wave-1 work lands in **`~/Users/evanmotovich/code/Tunza`** on branch **`clinical-contract-wave1`**. This repo owns the real gates (`npx tsc --noEmit`, `npm test`, `npm run lint`, `npm run build`) and the authoritative types, copy table, and referral lifecycle.
- **Nothing** lands in `fusion-harness`. Its tree is blocked-dirty (2,512 paths) and it is a harness, not a runtime. Its dirty paths (`adws/`, `duckdb20\_fusion\_lab/lab.sql`, `extensions/self-compact/`) are never touched.
- **medical-triage`** is read-only this wave. Its route, prompt, schema, and env stay untouched. Hardening it is a written ticket (task 2.d → `vault/tickets/medical-triage-adapter.md`), not a drive-by.
- Local commits on the branch are allowed. **No push, no merge, no**

rebase, no force updates.\*\* Publication is parent-owned and requires an exact-SHA harness receipt.

## ## 2. Wave-1 scope (what ships)

One vertical slice, no model, no corpus, no training:

1. **Contract module** ``lib/clinical/contract.ts`` (task 1.a, grok): runtime-validated ``ClinicalAssessment`` with ten sections – observed facts, retrieved evidence (empty, explicit ``retrieval_status``, ``trust_tier: "none"``), derived features, hypotheses, uncertainties, contradictions, required information, red flags, next action, provenance. Provenance on every result: ``rules_version``, ``model_id`` (null on the demo path), ``knowledge_version`` (null), ``last_sync`` (null), ``generated_at``. Append-only ``ClinicalEvent``. ``Contradiction`` as a first-class, never-auto-resolved object. Disposition = the existing four ``DecisionKind`` values plus ``escalate`` and ``abstain``, one mapping table; ``need_one_more_answer`` stays the missing-info ASK state; **no new user-visible headline**. Hand-rolled validators; **no new dependencies**. A fixture missing any section must fail validation.
2. **Eval harness v0** ``evals/`` (task 1.b, terra): manifest with content hashes and SYNTHETIC lineage tags; human-authored expected labels (never derived from ``decide()``); runs under the existing vitest gate, offline, deterministic. Families measurable now: red-flag recall of ``decide()``, abstention on incomplete cases, non-downgrade. Families not measurable now (clinical correctness, calibration, retrieval, multilingual generation, medication safety) emit explicit ``not_measurable``. **No aggregate score field anywhere**. The frozen baseline table for current ``decide()`` is captured before any behavior change.
3. **Append-only patient state** ``lib/clinical/state.ts`` (task 2.a, terra): projection from ``Encounter``/``AssessmentAnswers`` into ``ClinicalEvent`s``; a new temperature never erases the previous one; unit normalization; missingness vs unknown; same-type conflicts become unresolved ``Contradiction`s``; a deterministic trend function producing structural deltas only ("increased over N hours", "three declining readings") with **no clinical cutoffs and no disposition effect**. Encounter-scoped; ``lib/store.tsx`` persistence is not replaced.
4. **Reconciler** ``lib/clinical/reconcile.ts`` (task 2.b, grok): ``decide()`` result (the floor) + optional candidate (nullable ``model_id``) → reconciled disposition + disagreement record. Safer approved path wins; a candidate below the floor is rewritten up and the disagreement becomes telemetry, never a silent merge. Reconciliation failure must never fail open into a downgrade; persistence/telemetry failure must not block the safer user answer.
5. **Adversarial fixtures + release gates** ``evals/adversarial/``, ``evals/gates.ts`` (task 2.c, sol): look-alikes, paraphrase bypasses, Kiswahili colloquial/misspelling/code-switching, prompt-injection strings as inert tagged data. Fail-closed gate machinery: red-flag recall ≥ recorded baseline, no safety-critical slice regression, disposition distribution reported; any missing metric blocks promotion. Extends 1.b's manifest format, never forks it.
6. **Wiring** ``lib/clinical/pipeline.ts`` (task 3.a, grok): ``decide()`` → reconciler → contract, fed by the state projection. Renderer maps sections to existing copy keys via ``t()``; EN+SW parity stays compile-enforced (a missing Kiswahili string fails the build). Internal sections never collapse into one generated paragraph. ``tests/assessment.test.ts`` stays unchanged and green.
7. **Docs** (tasks 2.d and 4.a, glm): the private-lane adapter ticket, the wave-2 deferral ledger, and the final handoff with receipts.

## ## 3. Deferrals – written, not silent

Each deferral names its opening criterion. None of this work starts in

wave 1:

- **\*\*No training / SFT / QLoRA / distillation / quantization.\*\*** No measured failure exists to fix; there is no label provenance (`Y` is undefined), no dataset, no data-rights basis, and no fine-tuning substrate (capability-map \$0, \$4, \$5). Opens only on: a written failure analysis citing eval-report numbers, a named label producer, and a documented data-rights basis.
- **\*\*No retrieval ingestion.\*\*** No approved knowledge sources exist; there is nothing with jurisdiction, version, effective date, or trust tier to ingest. The contract reserves the empty evidence slot with `retrieval\_status`. Opens when named approved sources with metadata exist.
- **\*\*No guideline thresholds, no IMCI/ETAT encoding.\*\*** The keyword rules are a floor, not a protocol. Encoding numeric clinical thresholds without a named clinician approving a versioned rule pack would fake clinical approval. Opens on a documented clinical review process.
- **\*\*No JEV reimplementation.\*\*** External hosted decision model, absent from this tree; not a Tunza primitive.
- **\*\*medical-triage/` untouched.\*\*** Adapter work is ticket 2.d. Its UNSAFE findings (capability-map \$2) are recorded, not yet fixed there.
- **\*\*PatientState stays encounter-scoped.\*\*** No cross-encounter persistence until the README's privacy projections (cases vs case\_signals) exist; a richer persisted state would expand PHI exposure in localStorage.

#### ## 4. Hygiene rules (all tasks, no exceptions)

- No PHI in code, fixtures, docs, or logs. No `.env` contents read or committed. Keys only ever via env.
- No verbatim guideline text vendored anywhere.
- Fixtures are tagged SYNTHETIC and are engineering fixtures – never described as a training set. Same-model self-grading is forbidden.
- No language anywhere claims clinical approval or validation of the rules or the system. The demo floor keeps its not-clinically-validated label; demo facilities stay labeled demo.
- No new runtime or dev dependencies; everything runs under the existing vitest/tsc/eslint/next toolchain.
- Prompt-injection strings in fixtures are inert data: tagged, never executed, never shipped as live payloads.
- Never edit: `decide()`'s rules, `lib/types.ts` decision kinds, `tests/assessment.test.ts`, `app/api/triage/\*` in the private repo, anything under `adws/\*\*` or fusion-harness.

#### ## 5. File ownership (collision control)

| Path                                                               | Owner | task  | Slot |
|--------------------------------------------------------------------|-------|-------|------|
| `lib/clinical/contract.ts` + test                                  | 1.a   | grok  |      |
| `evals/` manifest, cases, baseline runner + test                   | 1.b   | terra |      |
| `lib/clinical/state.ts` + test                                     | 2.a   | terra |      |
| `lib/clinical/reconcile.ts` + test                                 | 2.b   | grok  |      |
| `evals/adversarial/`, `evals/gates.ts` + test                      | 2.c   | sol   |      |
| `vault/tickets/medical-triage-adapter.md`, `vault/wave2-ledger.md` | 2.d   | glm   |      |
| `lib/clinical/pipeline.ts` + test, `lib/copy.ts` additions (EN+SW) | 3.a   | grok  |      |
| `vault/wave1-handoff.md`                                           | 4.a   | glm   |      |

`lib/types.ts` is consumed read-only by all tasks in wave 1. Shared files (`lib/copy.ts`, `package.json`) get one writer per change, serialized by the harness's single-writer token.

#### ## 6. Wave-1 exit validation

Wave 1 is done when all of these hold, and nothing softer:

- Gates green in this repo: `npx tsc --noEmit`, `npm test`, `npm run lint`,

- `npm run build` (existing assessment cases unchanged, including Kiswahili danger keywords, child-not-drinking → `go\_now`, incomplete story → `need\_one\_more\_answer`).
- A contract fixture missing any of the ten sections fails validation.
- Two temperatures (37.4 then 39.2, different timestamps) both survive; no single authoritative temperature remains.
- Monotonic decline across three readings is a derived feature with no LLM.
- Candidate `self-care` against `decide() == go\_now` reconciles to `go\_now` with a disagreement record.
- The eval report is family-keyed; retrieval and calibration read `not\_measurable`; no field named `score`/`accuracy` gates a release.
- `model\_id` and `knowledge\_version` are null on the demo path.
- No training endpoint, dataset writer, or model-weight path was added.
- The private repo and fusion-harness are byte-identical to their pre-wave state.

## vault/wave1-handoff.md

What was written, what was verified, and what is still blocked.

SHA-256 3e480211b72eeb51bd72e3640e47bcbe430c2eafff8e972e0b322f5ab47197d · 133 lines

### # Wave-1 handoff

Dated 2026-10-06 (local). Branch `clinical-contract-wave1` in `/Users/evanmotovich/code/Tunza`. Head of the last child commit is `8228512`. Host recovery added the remaining files in the working tree and did **not** commit, push, or merge.

The collaboration graph is not successful. Tasks 0.a and 1.a were executed by children and checked against the files. Task 1.b was executed by a child; its label question was decided by the parent, not by a child report. Tasks 2.a–4.a never ran as child writes. The host wrote those modules after reading the current tree. Child reports are evidence, not acceptance.

This slice is an engineering contract around the existing demo floor. It is not a clinical approval, not a device claim, and not a model release.

### ## Label decision

Parent decision, applied here: accept the test-sourced labels for engineering-only regression, with human authorship explicitly unverified.

Recorded on `evals/manifest.json` as `label\_authorship\_decision`, and required by `loadSuite`. The field is excluded from the frozen measurement hash (`measurementManifest` in `evals/runner.ts`) so `evals/reports/baseline-decide.json` stays the unchanged baseline. That exclusion is a host amendment. It does not turn the labels into reviewed labels.

Wave 2 must not read these labels as clinical truth. Families that are not measurable stay `not\_measurable`.

### ## What is in the tree

| Path                                                 | Role                                                     |
|------------------------------------------------------|----------------------------------------------------------|
| ---                                                  | ---                                                      |
| `vault/capability-map.md`, `vault/wave1-contract.md` | Child 0.a audit and boundary. Host did not rewrite them. |
| `lib/clinical/contract.ts`                           | Child 1.a ten-section contract. Host did not edit it.    |

```
| `evals/manifest.json`, `evals/runner.ts`, `evals/reports/baseline-decide.json` | Child 1.b harness.
Host added the authorship decision and hash exclusion only. Baseline file unchanged. |
| `lib/clinical/state.ts` | Host. Append-only encounter projection, unit normalization, unresolved
same-type conflicts, structural trends. No disposition effect. |
| `lib/clinical/reconcile.ts` | Host. Demo floor wins. Lower candidate rewritten up. Unapproved higher
candidate not applied. Telemetry failure does not change the answer. |
| `evals/adversarial/`, `evals/gates.ts` | Host. Same manifest shape as 1.b. Injection strings are
inert data. Promotion function cannot authorize release. |
| `lib/clinical/pipeline.ts`, `lib/copy.ts` section labels | Host. `decide()` → reconciler → contract.
EN and SW labels. No new headline. |
| `evals/reports/wave1-pipeline.json` | Host comparison. Does not replace the frozen baseline. |
| `vault/tickets/medical-triage-adapter.md`, `vault/wave2-ledger.md` | Host. Docs only. Private route
not edited. |
```

`decide()`, `lib/types.ts` decision kinds, and `tests/assessment.test.ts`  
were not edited. `medical-triage` and `fusion-harness` were not edited.  
Pre-existing dirt in those repos was left as found.

## ## Receipts

Gates run in the foreground on 2026-10-06, after the host files existed:

```
- `npx tsc --noEmit`: exit 0.
- `npm test`: 145 passed, 13 files. Includes the previous assessment tests.
- `npm run lint`: exit 0.
- `npm run build`: Next.js 16.3.3 compiled; static pages generated.
```

Frozen baseline vs pipeline (`evals/reports/wave1-pipeline.json`):

```
- `frozen_baseline_replaced`: false
- `ship_gate`: none
- `comparison.families_equal`: true
- `red_flag_recall`: measured 3/3
- `abstention`: measured 3/3
- `non_downgrade`: measured 3/3
- `clinical_correctness`, `calibration`, `retrieval`,
 `multilingual_generation`, `medication_safety`: `not_measurable`
```

No field named `score` or `accuracy` is a ship gate. The pipeline report's  
`ship\_gate` is `none`.

Release-gate table on the identical baseline and pipeline family results  
(`evaluateReleaseGate`, asserted in `tests/eval-gates.test.ts`):

```
Row	Status
red-flag recall ≥ baseline	pass
no safety-critical language-slice regression	pass
ANSWER/ASK/ABSTAIN/ESCALATE distribution reported	pass for the fixture expected labels (0 / 3 / 0 / 6)
clinical thresholds signed	blocked
required metric families measured	blocked (`not_measurable` families)
promotion	blocked
release authorization	none
clinical effectiveness	not established
```

Champion/challenger on the frozen engineering inputs agrees 9/9 between  
fixture expected labels and `decide()`, and still returns promotion blocked.  
Agreement on an engineering ruler is not a promotion.

## ## Invariants checked

```
- A contract fixture missing any of the ten sections fails validation
 (`tests/clinical-contract.test.ts`, "fails when any section is missing").
```

- Temperatures 37.4 then 39.2, different timestamps, both remain. The state object has no `temperature` field (`tests/clinical-state.test.ts`).
- Three declining readings become the derived feature `three declining readings`. `lib/clinical/state.ts` does not call `decide()` or a model (`tests/clinical-state.test.ts`).
- Candidate `self-care` against `decide() == go\_now` reconciles to `go\_now` with a disagreement record (`tests/clinical-reconcile.test.ts`, `tests/clinical-pipeline.test.ts`).
- `monitor\_at\_home` is still rejected when critical answers are unknown and there is no story. Pipeline disposition matches `decide()`.
- Demo-path `model\_id` and `knowledge\_version` are null. Evidence bundle stays empty with `trust\_tier: "none"`.
- No training endpoint, dataset writer, or model-weight path was added.
- Adversarial observations, not ship gates: `passed out`, `degdege`, and `haamuki` do not hit the keyword floor (`need\_one\_more\_answer`). The inert instruction marker beside `unconscious` does not change the floor (`go\_now`) and is not executed. `fitting` in "The shirt is fitting loosely" overlaps the keyword pattern and the floor returns `go\_now`; no clinical label was assigned to that look-alike.

## ## Blocked

- Promotion. Unsigned clinical thresholds and `not\_measurable` families.
- Training, retrieval ingestion, guideline thresholds, JEV, private-route edits, cross-encounter persistence. Opening criteria are in `vault/wave2-ledger.md`.
- Push, merge, publication, and any live or release validation. Not authorized.
- Verified human authorship of labels. Explicitly unresolved. Accepted only as an engineering ruler.
- The collaboration graph. Do not mark it accepted or successful.

## ## Wave 2 opens only when the ledger's evidence exists

Retrieval needs named approved sources with jurisdiction, version, and effective dates. The shadow model path needs the adapter ticket done and a shadow run that does not change the demo answer. SFT needs a written failure analysis citing `evals/reports/` numbers, a named label producer, and a data-rights basis. None of those exist today.

## vault/wave2-ledger.md

Closed doors and the evidence required before any of them open.

SHA-256 dde8bab8d704e71c3f4339f5915ed4cd1b5d265f04bb42469baa2d87c9046686 · 78 lines

### # Wave-2 ledger

Dated 2026-10-06. Nothing on this page is open. Each item names the evidence that would have to exist before work starts. Absence of that evidence is the current state, not a placeholder to fill in.

Labels in `evals/` are an engineering regression ruler. Human authorship is unverified. Clinical review is `not\_reviewed`. They are not a label column and not a training set.

### ## Deferred

### ### Retrieval ingestion

Blocked. No named approved source exists with publisher, jurisdiction, document version, effective date, and review date. The contract keeps an empty evidence bundle (`trust\_tier: "none"`).

Opens only when a source list names those fields for each document, and a person authorized to approve the source signs the list. Unverified web text cannot enter the bundle.

### ### Model shadow path

Blocked. Depends on `vault/tickets/medical-triage-adapter.md`. The private route is still cast-only, logs raw model text, and has no floor under the model.

Opens only after that ticket is implemented and a shadow run shows disagreement counts without changing the user-visible demo path.

### ### Offline / degraded depth

Blocked. Named failure states exist in the public app as UI states. There is no local model, no queued clinical sync, and no knowledge version to mark stale.

Opens only after a written capability list says which functions require connectivity, and cached guidance carries `last\_sync` plus a version. Stale cache must not be marked current.

### ### SFT / QLoRA / distillation / quantization

Blocked. No measured failure analysis cites `evals/reports/` numbers. No label producer is named. No data-rights basis exists. A case row is not a training example. Same-model self-grading is forbidden.

Opens only when all three exist in writing:

1. A failure analysis that cites frozen eval-report numbers and says which failure training would change.
2. A named label producer and a review status other than unverified.
3. A documented data-rights basis for every row that would be used.

Until then, no training endpoint, dataset writer, or weight path.

### ### Guideline thresholds and IMCI/ETAT encoding

Blocked. Encoding numeric clinical thresholds without a named clinician and a versioned rule pack would fake approval. The keyword floor stays a floor.

Opens only on a documented clinical review that names the reviewer role, jurisdiction, protocol version, and effective dates. This ledger cannot supply that review.

### ### Cross-encounter PatientState

Blocked. State in this wave is encounter-scoped. Persisting a richer state in localStorage before privacy projections exist would expand exposure.

Opens only when those projections exist and a privacy review accepts the stored fields.

### ## Not a release

Wave 2 does not start because wave 1's engineering gates passed. Promotion

stays blocked while clinical thresholds are unsigned and required metric families are `not\_measurable`.